



National University of Computer and Emerging Sciences, Lahore



NIDPS: Network Intrusion Detection and Prevention System

Syed Raza Ali Rizvi 23L-3021 BS(SE)
Eishal Fatima Qadri 23L-3027 BS(SE)
Ryyan Choudhary 23L-2542 BS(SE)

Supervisor: Dr. Faisal Aslam

Final Year Project
October 10, 2026

Anti-Plagiarism Declaration

This is to declare that the above publication was produced under the:

Title: NIDPS: Network Intrusion Detection and Prevention System

is the sole contribution of the author(s), and no part hereof has been reproduced as it is the basis (cut and paste) that can be considered Plagiarism. All referenced parts have been used to argue the idea and cited properly. I/We will be responsible and liable for any consequence if a violation of this declaration is determined.

Date: 10th October 2026

Name: Syed Raza Ali Rizvi

Signature: 

Name: Eishal Fatima Qadri

Signature: 

Name: Ryyan Choudhary

Signature: 

Author's Declaration

This states Authors' declaration that the work presented in the report is their own, and has not been submitted/presented previously to any other institution or organization.

Abstract

Our project presents a Network Intrusion Detection and Prevention System, which protects small networks by combining signature-based methods and machine learning. The proposed solution uses the Suricata tool for known attack detection, applies Isolation Forest to identify anomalous behavior, and classifies the attacks with Random Forest and XGBoost algorithms. The analysis of encrypted traffic is accomplished by extracting flow-based features and TLS fingerprints, thus avoiding the need for traffic decryption. Experimental validation of the system was performed on multiple public intrusion detection datasets and real traffic with specific emphasis on detection accuracy, generalization capability, system overhead, and automated blocking of attacks on a low-resource servers.

Executive Summary

The increasing use of IoT devices has made small networks more exposed to cyber security threats. Most home and small-office routers provide basic network protection, but they lack advanced intrusion detection and behavioural analysis. This project proposes a Network Intrusion Detection and Prevention System (NIDPS) that combines multiple detection techniques to provide better protection against both known and previously unseen attacks.
The proposed system runs on a Linux-based device acting as the wireless access point of the protected network. All client traffic passes through this device before reaching the ISP gateway. The NIDPS follows a three-layer detection approach. Suricata is used for signature-based detection of known attacks, Isolation Forest is used to identify abnormal network behaviour, and Random Forest or XGBoost is used to classify known attack types. The outputs of these layers are combined to decide whether traffic should be passed, reported as suspicious, or blocked.
The system is designed to detect attacks such as SSH brute force, port scanning, command-and-control beaconing, ARP spoofing, DNS spoofing, data exfiltration, DDoS, and lateral movement. Since most network traffic in modern networks is encrypted, the system uses features of flows and TLS fingerprints such as JA3 and JA3S instead of decrypting user traffic. This allows encrypted connections to be analysed while reducing privacy and performance concerns.
A major part of the project is the evaluation of machine learning models across different datasets. CIC-IDS-2017, CSE-CIC-IDS2018 and CIC-IoT-2023 are used for training and cross-dataset testing, while traffic collected from the actual deployment environment is used as an additional test set. The same flow extraction approach was used for all data sets and the production network to minimize differences in the training set and traffic under study.
When the system detects suspicious activity with a certain level of confidence, it can be configured to block the host that initiated the attack using iptables and ipset. Moreover, protection mechanisms such as temporary blocks, allow-lists or rate-limits can be applied. A watchdog and configurable failure handling mechanism are also present, which ensures network state integrity in case one of the monitoring components fails.
The project is evaluated using both detection and system performance measures. These include precision, recall, F1-score, false positive rate, forwarding latency, throughput, detection latency, blocking time, resource usage and recovery behaviour. The overall aim is to develop a practical and affordable NIDPS which combines signature-based detection, machine learning, automated prevention and failure handling for small networks.

Table of Contents

List of Figures **ix**

List of Tables **xi**

1 Introduction **1**
1.1 Purpose of this Document 2
1.2 Intended Audience 2
1.3 Definitions, Acronyms, and Abbreviations 2
1.4 Conclusion 4

2 Project Vision **5**
2.1 Problem Domain Overview 5
2.2 Problem Statement 5
2.3 Problem Elaboration 6
2.4 Goals and Objectives 6
2.5 Project Scope 7
2.6 Sustainable Development Goal (SDG) 8
2.7 Constraints 8
2.8 Business Opportunity 9
2.9 Stakeholders Description/ User Characteristics 9
2.10 Conclusion 10

3 Literature Review / Related Work **11**
3.1 Definitions, Acronyms, and Abbreviations 11
3.2 Detailed Literature Review 12
3.2.1 Performance Comparison of Intrusion Detection Systems and Application of Machine Learning to Snort 12
3.2.2 A Novel Hybrid Intrusion Detection Method Integrating Anomaly Detection with Misuse Detection 13
3.2.3 Collaborative and Hybrid Network Intrusion Detection Framework Based on Suricata and Isolation Forest 14
3.2.4 Self-Healing Hybrid Intrusion Detection System: An Ensemble Machine Learning Approach 15
3.2.5 Comparative Evaluation of Machine Learning and Signature-Based NIDS for Multi-Class and Binary Threat Detection 16

3.2.6 Real-Time Hybrid Signature and Machine Learning Intrusion Detection . . . 16
3.2.7 Improving Detection Accuracy of Network Intrusions Using a Hybrid NIDS Based on Isolation Forest and Random Forest . . . 17
3.2.8 Extended Isolation Forest for Intrusion Detection in Zeek Data . . . 18
3.2.9 An Optimized Isolation Forest Based Intrusion Detection System for Heterogeneous and Streaming Data in IIoT Networks . . . 19
3.2.10 Improved Isolation Forest with Sparse Auto-Encoder Feature Extraction . . . 19
3.2.11 An Improved X-Means and Isolation Forest Based Methodology for Network Traffic Anomaly Detection . . . 20
3.2.12 Intrusion Detection: A Comparison Study of Machine Learning Models Using Unbalanced Dataset . . . 21
3.2.13 Enhanced Intrusion Detection Systems Performance with UNSW-NB15 Data Analysis . . . 22
3.2.14 An Optimized Gradient Boosting Framework for IoT Intrusion Detection on the CICIoT2023 Dataset . . . 22
3.2.15 Cross-Dataset Generalisation of Deep Learning Intrusion Detection Models . . . 24
3.3 Literature Review Summary Table . . . 24
3.4 Data Set Summary Table . . . 26
3.5 Comparison with Closest Existing Systems . . . 27
3.6 Conclusion . . . 28

4 Software Requirement Specifications 29
4.1 List of Features . . . 29
4.2 Functional Requirements . . . 31
4.2.1 NIDPS-1: Inline Wireless Access Point . . . 31
4.2.2 NIDPS-2: Signature-Based Detection (Layer 1) . . . 31
4.2.3 NIDPS-3: TLS Fingerprints and Flow Extraction . . . 32
4.2.4 NIDPS-4: Anomaly Detection (Layer 2) . . . 32
4.2.5 NIDPS-5: Classification of Attacks (Layer 3) . . . 32
4.2.6 NIDPS-6: Decision Fusion . . . 32
4.2.7 NIDPS-7: Dynamic Blocking . . . 33
4.2.8 NIDPS-8: Recovery Mechanism . . . 33
4.2.9 NIDPS-9: Monitoring . . . 33
4.2.10 NIDPS-10: Configurations . . . 34
4.2.11 NIDPS-11: Logging and Storage . . . 34
4.2.12 NIDPS-12: Training and Versioning . . . 34
4.2.13 NIDPS-13: Auto Maintenance . . . 35
4.3 Quality Attributes . . . 35
4.4 Non-Functional Requirements . . . 36

4.4.1 Performance . . . 36
4.4.2 Capacity . . . 37
4.4.3 Resource Usage . . . 37
4.4.4 Reliability and Availability . . . 37
4.4.5 Accuracy . . . 37
4.4.6 Security . . . 37
4.4.7 Privacy . . . 38
4.4.8 Usability . . . 38
4.4.9 Maintainability . . . 38
4.4.10 Extensibility and Reusability . . . 38
4.4.11 Portability . . . 38
4.5 Assumptions . . . 38
4.6 Use Cases . . . 39
4.7 Hardware and Software Requirements . . . 51
4.7.1 Hardware Requirements . . . 51
4.7.2 Software Requirements . . . 52
4.8 Graphical User Interface . . . 53
4.8.1 Login Screen . . . 53
4.8.2 Overview Screen . . . 54
4.8.3 Network Traffic Screen . . . 54
4.8.4 Intrusion Alerts Screen . . . 55
4.8.5 Blocked Hosts and Allowlist Screen . . . 56
4.8.6 Detection and Prevention Configuration Screen . . . 56
4.8.7 System Health Screen . . . 59
4.9 Database Design . . . 60
4.9.1 ER Diagram . . . 60
4.9.2 Data Dictionary . . . 61
4.10 Risk Analysis . . . 66
4.11 Conclusion . . . 68

5 Proposed Approach and Methodology 69
5.1 Overview of the Proposed Framework . . . 69
5.2 Datasets . . . 70
5.2.1 Public Benchmark Datasets . . . 70
5.2.2 Self-Collected Real-World Dataset . . . 71
5.2.3 Feature Harmonisation Across Datasets . . . 71
5.2.4 Pre-processing . . . 72
5.3 Traffic Acquisition and Flow Construction . . . 73
5.3.1 Inline Packet Path . . . 73
5.3.2 Flow Assembly Algorithm . . . 73
5.3.3 TLS Fingerprint Extraction (JA3/JA3S) . . . 75
5.4 Layer 1: Signature-Based Detection . . . 76

5.4.1 Suricata Configuration . . . 76
5.4.2 Custom Rules for In-Scope Attacks . . . 76
5.4.3 ARP Spoofing Monitor . . . 77
5.5 Layer 2: Unsupervised Anomaly Detection . . . 77
5.5.1 Isolation Forest . . . 77
5.5.2 Adaptive Threshold Calibration . . . 78
5.5.3 Data Exfiltration Score . . . 79
5.5.4 C2 Beaconing Score . . . 79
5.6 Layer 3: Supervised Attack Classification . . . 79
5.6.1 Random Forest . . . 80
5.6.2 XGBoost . . . 80
5.6.3 Training and Model Selection . . . 80
5.6.4 Confidence Handling . . . 81
5.7 Decision Fusion . . . 81
5.7.1 Verdict Algorithm . . . 81
5.7.2 Host Evidence Accumulation . . . 83
5.7.3 Timing of ML Verdicts . . . 83
5.8 Automated Prevention . . . 83
5.8.1 Blocking Mechanism . . . 83
5.8.2 Blocking Procedure . . . 84
5.9 Fault Tolerance and Failure Handling . . . 84
5.9.1 Health Monitoring . . . 84
5.9.2 Failure Modes . . . 85
5.9.3 State Machine . . . 85
5.10 Evaluation Methodology . . . 85
5.10.1 Detection Metrics . . . 86
5.10.2 Cross-Dataset Generalisation . . . 86
5.10.3 System Performance on Constrained Hardware . . . 87
5.11 Conclusion . . . 88

6 High-Level and Low-Level Design 89
6.1 System Overview . . . 89
6.2 Design Considerations . . . 89
6.2.1 Assumptions and Dependencies . . . 90
6.2.2 General Constraints . . . 90
6.2.3 Goals and Guidelines . . . 91
6.2.4 Development Methods . . . 92
6.3 System Architecture . . . 92
6.3.1 Subsystem Architecture . . . 92
6.4 Architectural Strategies . . . 96
6.4.1 Layered Defence with a Short Fast Path . . . 96
6.4.2 Metadata-Only Analysis of Encrypted Traffic . . . 96

6.4.3 Offline Training, Online Inference . . . 96
6.4.4 Local Services Connected by an Event Bus . . . 97
6.4.5 Kernel-Enforced Blocking and Failure Semantics . . . 97
6.4.6 Choice of Languages and Products . . . 97
6.4.7 Error Detection and Recovery . . . 98
6.4.8 Data Storage, Persistence and Retention . . . 99
6.4.9 Future Extension . . . 99
6.5 Domain Model/Class Diagram . . . 99
6.6 Policies and Tactics . . . 99
6.6.1 Rule and Threshold Management . . . 99
6.6.2 Model Packaging and Versioning . . . 101
6.6.3 Engineering Trade-offs . . . 101
6.6.4 Coding Guidelines and Conventions . . . 101
6.6.5 Requirements Traceability . . . 101
6.6.6 Testing Plan . . . 102
6.6.7 Hierarchical Organisation of the Source Code . . . 102
6.6.8 Build and Deployment . . . 103
6.6.9 Maintenance . . . 103
6.7 Conclusion . . . 103

7 Implementation and Test Cases 105
7.1 Implementation . . . 105
7.1.1 Platform and Tools . . . 105
7.1.2 Implementation Status . . . 106
7.1.3 Communication Between Services . . . 106
7.1.4 Shared Foundations . . . 107
7.1.5 Flow Extraction Service . . . 107
7.1.6 Layer 1: Signature Detection and ARP Monitoring . . . 109
7.1.7 Layer 2: Anomaly Detection Service . . . 110
7.1.8 Self-Collected Dataset Tooling . . . 111
7.1.9 Components Under Development . . . 112
7.1.10 Setup and Deployment . . . 112

9 Experimental Results and Discussion 114
9.1 Experimental Setup . . . 114
9.2 Signature-Based Detection: Suricata Against the Self-Collected Attacks . . . 115
9.3 Anomaly Detection: Isolation Forest Against the Self-Collected Attacks . . . 117
9.4 Discussion . . . 117
9.5 Limitations of the Current Experimental Work . . . 118
9.6 Conclusion . . . 118

List of Figures

2.1 The figures display the orange SDG 9 icon (Industry, Innovation, and Infrastructure) and blue SDG 16 icon (Peace, Justice, and Strong Institutions), representing resilient tech frameworks and safe digital environments. 8

4.1 Use case diagram of the NIDPS. 45
4.2 Navigation flow of the Network Administrator. 53
4.3 Login screen. 53
4.4 Overview screen. 54
4.5 Network Traffic screen. 55
4.6 Intrusion Alerts screen with the Alert Evidence panel. 55
4.7 Blocked Hosts and Allowlist screen. 56
4.8 Configuration screen – Detection Layers tab. 57
4.9 Configuration screen – Detection Thresholds tab. 58
4.10 Configuration screen – Prevention and Failure tab. 58
4.11 Configuration screen – Rules, Calibration and Models tab. 59
4.12 System Health screen. 59
4.13 Entity-relationship diagram of the NIDPS database. 60

5.1 Proposed three-layer NIDPS pipeline. The inline packet path is represented by the solid line, whereas the dotted line represents the analysis and the supervision path. 70
5.2 Decision-fusion flow for one flow record. 82
5.3 Watchdog state machine for each supervised component. 86

6.1 System architecture of the NIDPS. 93
6.2 Packet path through the data plane. 94
6.3 Offline training pipeline and model hand-over to the device. 95
6.4 Class diagram of the NIDPS runtime. 100

List of Tables

1.1 Definitions, acronyms and abbreviations. 3

3.1 Summary of the reviewed literature. 24
3.2 Datasets used in the reviewed studies. 26
3.3 Comparison of the closest existing systems with the proposed NIDPS. . . 27

4.1 Features of the proposed NIDPS. 30
4.2 Quality attributes of the NIDPS. 35
4.3 Use case UC-01: Log In to Dashboard. 39
4.4 Use case UC-02: View Live Network Statistics. 40
4.5 Use case UC-03: Review Alerts and Evidence. 41
4.6 Use case UC-04: Detect and Block Malicious Host. 42
4.7 Use case UC-05: Release Blocked Host. 43
4.8 Use case UC-06: Manage Allowlist. 44
4.9 Use case UC-07: Configure Failure Mode. 46
4.10 Use case UC-08: Enable or Disable Detection Layers. 46
4.11 Use case UC-09: Adjust Detection Thresholds. 47
4.12 Use case UC-10: Calibrate Anomaly Threshold. 48
4.13 Use case UC-11: Recover from Component Failure. 49
4.14 Use case UC-12: Deploy Model Package. 50
4.15 Hardware requirements. 51
4.16 Software requirements. 52
4.17 Data dictionary for ADMIN_USER. 61
4.18 Data dictionary for HOST. 61
4.19 Data dictionary for FLOW_RECORD. 61
4.20 Data dictionary for EVIDENCE. 62
4.21 Data dictionary for VERDICT. 63
4.22 Data dictionary for BLOCK_ENTRY. 63
4.23 Data dictionary for ALLOWLIST_ENTRY. 64
4.24 Data dictionary for MODEL_PACKAGE. 64
4.25 Data dictionary for CALIBRATION_STATE. 65
4.26 Data dictionary for CONFIG_SETTING. 65
4.27 Data dictionary for COMPONENT_EVENT. 65
4.28 Data dictionary for AUDIT_LOG. 66
4.29 Risk analysis. 67

5.1 Public benchmark datasets used for training and cross-evaluation. 70
5.2 Unified attack taxonomy used for Layer 3. 72
5.3 Flow timeout parameters. 74
5.4 Custom Layer 1 rules for the in-scope attacks. 76
5.5 Configurable failure modes. 85
5.6 Cross-dataset evaluation matrix of macro F1-scores, to be filled from experiments. 87
5.7 System performance measures. 87

6.1 General constraints and their impact on the design. 91
6.2 Subsystem responsibilities. 93
6.3 Product and language choices. 98
6.4 Requirements traceability matrix. 102

7.1 Platform and tools used by the implementation. 105
7.2 Implementation status of the NIDPS components. 106
7.3 Environment variables read by the services. 113

9.1 Attack scripts used in the experiments (Section 7.1.8.1). 115
9.2 Suricata's response to each self-collected attack. 116
9.3 Isolation Forest and host-behaviour results on the four attacks Suricata missed. 117

Chapter 1

Introduction

Computer networks enable the connection of devices, exchange of information and access to online services, but at the same time they create threats for malevolent individuals to exploit. A **network intrusion** happens when an unauthorized entity attempts to gain access, use, interrupt or destroy a network or the devices connected to it. Intrusions can be external, originating from outside the network, for example, when an intruder scans the network for open ports or attempts to access the network using weak credentials, or internal, coming from within the network through a compromised device or user.

Network intrusions can take different forms, varying from the attempts to gain unauthorized access like in the case of brute-force attacks, Denial-of-Service attacks that cause service disruption, interception of traffic through spoofing, transferring data out of the network after gaining access to a host computer, or attacking other compromised nodes through command-and-control methods. The variety of intrusion methods requires multiple approaches to detecting malicious activity since relying on a single method may be ineffective.

Intrusion Detection and Prevention Systems are used to monitor network traffic and identify such malicious behaviour. Traditional systems commonly use predefined signatures or rules to recognize known attacks. These methods can be effective when an attack pattern is already known, but they may struggle with new attacks or variations that do not match an existing signature. Machine learning offers another approach by learning patterns in network traffic and identifying suspicious or abnormal behaviour. However, its effectiveness can depend heavily on the data used to train the model and the environment in which it is deployed.

Another challenge is that much of modern network communication is encrypted. While encryption protects user privacy, it also limits the amount of traffic content that can be directly inspected. This makes network security systems increasingly dependent on other information, such as traffic patterns, flow behaviour and connection metadata. These challenges create a need for intrusion detection systems that can combine multiple detection techniques and remain practical for real network environments.

The aim of this project is to develop a **Network Intrusion Detection and Prevention System (NIDPS)** for small networks. The project combines different detection approaches so that known attacks, abnormal behaviour and suspicious network activity can be identified and acted upon. The system is designed to operate on a Linux-based device placed directly in the network path, while also considering practical issues such as encrypted traffic, limited hardware resources and reliability.

The remainder of this report is organized as follows. **Chapter 2** contains the project scope, goals, and objectives. **Chapter 3** reviews the background research and related work on

network intrusion detection and machine learning. **Chapter 4** presents the Software Requirements Specification (SRS) of the proposed system. **Chapter 5** explains the proposed approach and methodology used for detection, prevention and evaluation. **Chapter 6** describes the high-level and low-level design of the system.

1.1 Purpose of this Document

The purpose of this document is to present the design, development and evaluation of our Final Year Project, a **Network Intrusion Detection and Prevention System (NIDPS)** for small networks. The project aims to provide a practical security solution that can monitor network traffic and detect malicious or abnormal activity and respond to identified threats using a combination of signature-based detection and machine learning.
The primary goal is to assess the protection level provided by this approach while maintaining its viability for limited hardware. The research also examines how well machine learning models trained on public intrusion detection datasets perform when tested on different datasets and on real deployment traffic.
The main research questions concern whether a combination of diverse detection techniques helps in improving the recognition of both normal and abnormal activities, whether the models are capable of generalizing on different network environments, and whether the overall system can be operated efficiently without affecting the performance of the normal network significantly
This report covers the project scope, related research, software requirements, methodology and system design.

1.2 Intended Audience

This report is mainly directed to the **Final Year Project evaluation panel, project advisor, faculty members and other academic reviewers** who will assess the technical quality, design, implementation and evaluation of the project.
In addition, it may be useful for **students, researchers and developers working in the areas of network security, intrusion detection or machine learning**, as it describes the proposed NIDPS architecture, detection approaches, evaluation methodology and practical implementation considerations.
The report is written with enough technical detail to explain the system clearly while keeping the discussion understandable for readers with a general background in computer science and cyber security.

1.3 Definitions, Acronyms, and Abbreviations

Table 1.1: Definitions, acronyms and abbreviations.

Term	Definition
NIDPS	Network Intrusion Detection and Prevention System
IDS	Intrusion Detection System
IPS	Intrusion Prevention System
ML	Machine Learning
RF	Random Forest
XGBoost	Extreme Gradient Boosting
IoT	Internet of Things
C2	Command and Control
DoS	Denial of Service
DDoS	Distributed Denial of Service
TLS	Transport Layer Security
JA3	TLS client fingerprint generated from properties of the ClientHello message
JA3S	TLS server fingerprint generated from properties of the ServerHello message
ARP	Address Resolution Protocol
DNS	Domain Name System
DHCP	Dynamic Host Configuration Protocol
NAT	Network Address Translation
IP	Internet Protocol
TCP	Transmission Control Protocol
UDP	User Datagram Protocol
ISP	Internet Service Provider
AP	Access Point
PCAP	Packet Capture file containing recorded network traffic
NFQUEUE	Linux Netfilter queue used to pass packets from the kernel to a user-space application for inspection
iptables	Linux firewall utility used to configure packet filtering and rules for controlling traffic
ipset	Linux utility used to maintain sets of IP addresses for firewall matching and blocking
Suricata	Open-source network IDS/IPS used for signature-based traffic inspection
Isolation Forest	Unsupervised machine learning algorithm used to detect abnormal or unusual network traffic

Term	Definition
Flow	A group of related packets identified using network properties such as source and destination addresses, ports and protocol
Signature-Based Detection	Detection of attacks by matching network activity against predefined rules or known attack patterns
Anomaly Detection	Detection of activity that differs significantly from established normal network behaviour
Zero-Day Attack	An attack that exploits a previously unknown or unaddressed vulnerability or behaviour
API	Application Programming Interface
F1-Score	Evaluation metric that combines precision and recall into a single value
False Positive	Legitimate network activity that is incorrectly identified as malicious
Fail-Open	Failure mode in which network traffic is allowed to continue when a security component becomes unavailable
Fail-Closed	Failure mode in which network traffic is blocked when a security component becomes unavailable

1.4 Conclusion

The proposed NIDPS aims to improve the security of small networks by combining multiple intrusion detection techniques within a single system. The project focuses on the detection of both known and abnormal network behavior with regard to practical aspects like encrypted traffic or the limitations of the available hardware and system reliability. Chapter 1 introduced the background, intention and general direction of the project. The following chapters elaborate on the related research, software requirements, proposed methodology and system design.

Chapter 2

Project Vision

This chapter provides an overview of the proposed Network Intrusion Detection and Prevention System. It mainly focuses on describing the purpose of the system, the problem that the system aims to solve, project objectives, scope, major constraints, identifying stakeholders, and the benefits of the project. Since this project includes both research and development, it focuses on building a working NIDPS as well as evaluating how well the selected intrusion detection techniques perform in different network environments.

2.1 Problem Domain Overview

Our system implements a detection pipeline with three layers to tackle diverse attack vectors. The first layer uses Suricata with the Emerging Threats ruleset to capture known malicious activity via fixed signatures. The second layer uses an Isolation Forest model to flag baseline traffic anomalies that lack pre-existing rules. The third layer applies Random Forest or XGBoost classifiers to categorize flagged events into specific threat types. Instead of relying on a single indicator, the system aggregates findings across all three layers before taking action, either allowing the connection, logging an alert, or initiating a block.

To protect user privacy and minimize processing overhead, the system avoids decrypting packet payloads. Instead, it evaluates connection behavior through flow statistics and TLS parameters, specifically JA3 and JA3S fingerprinting. When malicious activity is confirmed, an automated response module updates local firewall configurations using iptables and ipset to block the origin host immediately. A web-based management interface provides administrators with system telemetry, security logs, and a list of currently blocked endpoints.

The research component focuses on model adaptability across changing network conditions. Machine learning models often suffer performance degradation when deployed outside their initial training data. To evaluate cross-environment resilience, we test our models on benchmark datasets as well as live traffic captured directly from our access point deployment.

2.2 Problem Statement

Existing intrusion detection approaches have their limitations. Signature based systems are effective against known attacks, but they may miss new attacks or modified versions of existing ones. Machine learning approaches can identify unusual traffic, but their performance

may drop when they are used on a network that is different from the dataset on which they were trained.
The problem addressed by this project is therefore the lack of an affordable and practical solution for small networks that can combine known-attack detection, anomaly detection, and attack classification, while still run efficiently on limited hardware. The system must also be able to deal with encrypted traffic, avoid excessive false positives, and continue behaving safely if one of its components fails.

2.3 Problem Elaboration

The overall problem can be divided into smaller problems that need to be handled together.
First, no single detection method works for everything. Signature tools like Suricata catch known attacks instantly, but they miss brand new threats. On the flip side, machine learning spots weird behavior, but it can trigger false alarms on normal traffic. Using just one approach leaves major security gaps, so the system needs a multi-layer setup.
Second, machine learning models often struggle when moved to a new environment. A model might get perfect scores on one dataset, but completely fail on a different network with different devices and user habits. To test real-world reliability, we evaluate our models across multiple datasets, like CIC-IDS-2017, CSE-CIC-IDS2018, and CIC-IoT-2023, as well as live traffic from our own access point.
Third, most internet traffic is encrypted. Trying to decode network traffic is too taxing on the hardware and invasive to user privacy. Instead, the system reads connection behaviors, traffic patterns, and TLS handshake plaintexts (JA3 and JA3S).
Fourth, automatic blocking can backfire if the system makes a mistake. While blocking bad actors via iptables and ipset stops attacks fast, false alarms can accidentally lock out real users. To prevent this, the response engine includes safeguards like allowlists, temporary blocks, and adjustable threat thresholds.
Finally, the system runs on low power hardware. If the NIDPS uses too much CPU or memory, it will slow down internet speeds for everyone on the network. To keep things fast and stable, we use lightweight ML models, process data off-path, and set up fail-safes so a system crash doesn't knock out the entire internet connection.

2.4 Goals and Objectives

The main goal of this project is to build and evaluate a practical hybrid NIDPS that protects small networks using both signature matching and machine learning.
The main objectives include:

- **Hardware & Deployment:** Implement an inline NIDPS between client devices and the ISP gateway, complete with a watchdog and auto-recovery rules to prevent network downtime.

- **Multi-Layer Detection:** Combine Suricata (using Emerging Threats rules) for known attacks, Isolation Forest for unseen traffic anomalies, and Random Forest/XGBoost for classifying specific threats.
- **Encrypted Traffic Analysis:** Assess encrypted connections using flow statistics and TLS fingerprints (JA3/JA3S) without decrypting user payloads.
- **Automated Mitigation:** Automatically block confirmed threats using Linux firewall rules (iptables/ipset).
- **Cross-Environment Research:** Train and test models across CIC-IDS-2017, CSE-CIC-IDS2018, and CIC-IoT-2023, and benchmark their performance on live traffic from our actual access point.
- **Management & Telemetry:** Build a web dashboard for live monitoring, alerts, and system controls, measuring throughput, latency, CPU/memory usage, and detection accuracy under real traffic.

2.5 Project Scope

The system will operate directly as a wireless access point positioned between connected devices and the ISP gateway. Core features include:

- **Multi-Layer Detection:** Combines Suricata to catch known attack rules, Isolation Forest to spot unusual traffic spikes, and Random Forest or XGBoost to figure out the exact attack type.
- **Encrypted Traffic Analysis:** Looks at connection patterns and unencrypted TLS signatures to check if a stream is safe, completely skipping payload decryption to keep user privacy intact.
- **Active Response:** Proactively take actions such as dropping the offending IP from a network by triggering automated responses while providing a intuitive web interface to triage and manage alerts and blocked devices.
- **Research and Testing:** Evaluate the accuracy of the framework against several benchmark datasets and network traffic captures from our infrastructure while maintaining memory and CPU footprint under reasonable limits for small scale hardware.

Key Deliverables

- A working NIDPS access point prototype with automated prevention and fail-safe controls
- The complete ML model training, testing, and cross-dataset evaluation pipeline
- A web management dashboard
- Practical performance benchmarks (latency, resource usage, detection accuracy)

- Full system design and research documentation.

Out of Scope

To maintain a clear focus, the project will not:

- Function as an enterprise-grade security suite or support enterprise-scale hosting.
- Monitor traffic from devices bypassing the access point.
- Decrypt user traffic payloads.
- Cover every existing exploit category or run continuous online model retraining on the live device.

2.6 Sustainable Development Goal (SDG)



Figure 2.1: The figures display the orange SDG 9 icon (Industry, Innovation, and Infrastructure) and blue SDG 16 icon (Peace, Justice, and Strong Institutions), representing resilient tech frameworks and safe digital environments.

Our project mainly focuses on **SDG 9: Industry, Innovation and Infrastructure**. Modern digital services rely heavily on secure and stable network connections. By adapting advanced threat detection for low-cost, resource-constrained hardware, this project offers an accessible way to make small scale networks more resilient against cyberattacks.
It also contributes to **SDG 16: Peace, Justice and Strong Institutions** by supporting safe online spaces where users are protected against unauthorized access, data disclosure, and attacks on their network.

2.7 Constraints

Building and running this system comes with a few limitations:

- **Hardware Constraints:** Low-cost hardware is unable to withstand the demands of resource-heavy software. Memory consumption, as well as processor utilization, must be kept at a minimum, which rules out the use of heavy ML models.

- **Network Latency:** Being in the middle of the track means that any additional delay introduced by the system will add up to the overall latency of the Internet. Traffic processing must be fast to keep connections smooth.
- **Encrypted Traffic:** Since packet contents are hidden, reading payloads is off the table. The ML side has to rely only on connection patterns and TLS details.
- **Data Limits:** Public security datasets do not cover every real-world scenario. That is why testing across multiple datasets and live traffic is necessary.
- **Coverage Gaps:** The system can only monitor devices that connect directly through its access point. Any device using a different network stays invisible.
- **Blocking Mistakes:** Too aggressive blocking prevents legitimate users, while too lenient allows threats through, requires careful adjustment and default safety settings.

2.8 Business Opportunity

Most small offices, research labs, and home users cannot afford costly corporate security tools. Moreover, they rarely have a competent IT staff that would be able to constantly monitor and adjust complicated security software. This makes an affordable Linux-based solution that uses standard security hardware attractive. It makes all devices connected to it secure without any additional software. By only implementing the traffic control at the regional access point level, it makes all the devices connected to it safe and secure without any additional hassle. While this project is centered on research and a working prototype, it builds a solid base for a practical product aimed at smaller networks.

2.9 Stakeholders Description/ User Characteristics

Network Owners and Admins: The primary users of the system which involve configuration of alerts, blocked devices, and system preferences. Since these are not security experts, the system is designed to provide informative feedback and employ safety defaults.
Regular Network Users: These are the individuals and devices which access the WI-FI network. They are not direct users of the dashboard, but benefit from its existence, as it helps to maintain and optimize the network.
The Project Team: This team is responsible for the development of the system, including the coding, training, and testing of algorithms.
Advisors and Evaluators: Faculty members who review the details of the research, evaluate the code, and ensure that the project's objectives are met.
Future Researchers: Potential students and developers who will review the project's details and possibly adopt it in their own research.

2.10 Conclusion

The contents of chapter 2 provided an overview of the proposed NIDPS project, which included the problem statement, major aspects and issues, and objectives of the intended solution. In this chapter, we have discussed the project scope, key objectives, constraints, stakeholders, and potential benefits for business, and specified the relevant Sustainable Development Goal. Thus, this chapter sets the course and boundaries of the project, providing the basis for the requirements, methods, design, and implementation that will be elaborated in subsequent chapters.

Chapter 3

Literature Review / Related Work

This chapter reviews recent research related to Network Intrusion Detection and Prevention Systems (NIDPS). It focuses on the detection techniques, datasets, machine learning models, deployment approaches and evaluation methods used in previous studies. Each study is summarised, critically analysed and related to the proposed system, and the chapter ends with summary tables of the reviewed literature, the datasets they use and the systems closest to the proposed NIDPS.

3.1 Definitions, Acronyms, and Abbreviations

The important definitions, acronyms and abbreviations used in this chapter are listed below:

- **AUC:** Area Under the (ROC) Curve
- **CIC:** Canadian Institute for Cybersecurity
- **DT:** Decision Tree
- **EIF:** Extended Isolation Forest
- **F1-score:** Harmonic mean of precision and recall
- **FPR:** False Positive Rate
- **IDS:** Intrusion Detection System
- **IIoT:** Industrial Internet of Things
- **IoT:** Internet of Things
- **iForest / IF:** Isolation Forest
- **IPS:** Intrusion Prevention System
- **LSTM:** Long Short-Term Memory (recurrent neural network)
- **ML:** Machine Learning
- **NIDPS:** Network Intrusion Detection and Prevention System
- **NIDS:** Network Intrusion Detection System
- **OC-SVM:** One-Class Support Vector Machine
- **RF:** Random Forest
- **SVM:** Support Vector Machine

- **XGBoost:** Extreme Gradient Boosting
- **Zeek:** Open-source network security monitor that produces connection logs

3.2 Detailed Literature Review

This section covers notable literature on hybrid signature and machine learning intrusion detection, unsupervised anomaly detection with Isolation Forest, supervised attack classification with tree ensembles, and the deployment and generalisation of ML-based detectors. Each item is presented with a summary, a critical analysis and its relationship to the proposed NIDPS.

3.2.1 Performance Comparison of Intrusion Detection Systems and Application of Machine Learning to Snort

Snort and Suricata are compared on a 10 Gbps network, and machine learning is added to Snort to reduce its false alarms. An SVM optimised with the firefly algorithm lowers the false positive rate to 8.6%.

3.2.1.1 Summary of the Research Item

Shah and Issac [1] compared the two most widely used open-source signature engines, Snort and Suricata, running on identical machines at a traffic rate of 10 Gbps. Suricata handled the high traffic speed with a lower packet drop rate because of its multi-threaded design, but used more computing resources, while Snort showed slightly higher detection accuracy. Since Snort produced a large number of false positive alarms, the authors developed an adaptive Snort plug-in and compared several learning algorithms to filter its alerts. SVM was selected, and the best configuration, an SVM optimised with the firefly algorithm, achieved a false positive rate of 8.6% and a false negative rate of 2.2%.

3.2.1.2 Critical Analysis of the Research Item

The strengths and weaknesses of this work are:

- It gives a practical, measured comparison of Snort and Suricata under heavy load, which is useful when choosing the signature engine for an inline system.
- It shows clearly that machine learning can reduce the false alarms of a signature engine.
- The SVM models were evaluated offline; SVMs are relatively heavy, and the work does not show the approach running on live traffic on low-cost hardware.
- The system only detects and does not block traffic, and it does not consider encrypted traffic.

3.2.1.3 Relationship to the Proposed Research Work

This work supports the use of Suricata as the first layer of the proposed NIDPS because it copes better with high traffic rates, and it supports the idea of using machine learning to reduce the false positives of a signature engine. Unlike this work, the proposed system uses lightweight tree models that classify flows in milliseconds and acts on the verdict by blocking the offending host.

3.2.2 A Novel Hybrid Intrusion Detection Method Integrating Anomaly Detection with Misuse Detection

A C4.5 decision tree (misuse detection) splits the normal training data into subsets, and a separate one-class SVM is trained on each subset for anomaly detection. Evaluated on NSL-KDD.

3.2.2.1 Summary of the Research Item

Kim, Lee and Kim [2] proposed a hierarchical hybrid intrusion detection method in which a misuse detection model and an anomaly detection model are combined in a decomposed structure. A C4.5 decision tree is first trained as the misuse detector, and it is also used to divide the normal training data into smaller, more uniform subsets. A one-class SVM is then trained on each subset, so every anomaly model describes one region of normal behaviour more precisely and indirectly benefits from the knowledge of known attacks. On the NSL-KDD dataset the method detected both known and unknown attacks better than conventional methods while keeping false positives low, and the anomaly model needed only about 50% of the training time and 60% of the testing time of the conventional models.

3.2.2.2 Critical Analysis of the Research Item

The strengths and weaknesses of this work are:

- The decomposition of normal traffic gives more accurate anomaly models and noticeably lower training and testing time.
- It shows that relying on a single model to do all the work produces more false positives than letting each component handle the traffic it is best at.
- The evaluation uses only NSL-KDD, a dataset derived from 1998 traffic that does not represent modern or encrypted traffic.
- It is an offline detection method; no prevention or deployment on a real network is shown.

3.2.2.3 Relationship to the Proposed Research Work

The proposed NIDPS follows the same principle of combining misuse (signature) detection with anomaly detection so that each part handles the traffic it suits best. It replaces the

decision tree with Suricata rules and a Random Forest/XGBoost classifier, uses Isolation Forest instead of one-class SVM for cheaper anomaly scoring, and evaluates on modern datasets and live traffic.

3.2.3 Collaborative and Hybrid Network Intrusion Detection Framework Based on Suricata and Isolation Forest

Suricata detects known attacks and an Isolation Forest flags anomalies that may be unknown attacks. Several NIDS instances share alerts through a central log.

3.2.3.1 Summary of the Research Item

Chiba et al. [3] proposed a hybrid and collaborative NIDS framework. Suricata performs signature-based detection of known attacks and was chosen over Snort because it is multi-threaded and divides its workload across processing resources. Traffic that does not match any signature is examined by an Isolation Forest, which detects anomalies that may indicate unknown attacks. Since Isolation Forest does not depend on distance or density calculations, the authors argue that it keeps the computational cost low. The NIDS instances are collaborative: alerts are stored in a central log so that an attack detected by one sensor is also recognised by the others.

3.2.3.2 Critical Analysis of the Research Item

The strengths and weaknesses of this work are:

- The pairing of Suricata with Isolation Forest is lightweight and matches the needs of edge devices.
- Sharing confirmed attacks between sensors lets the whole network benefit from one detection.
- It is mainly a design proposal: there is no attack-classification stage and only limited experimental evaluation.
- It does not describe automated blocking, failure handling or the analysis of encrypted traffic.

3.2.3.3 Relationship to the Proposed Research Work

This is one of the works closest to the proposed NIDPS, which also places Suricata first and sends traffic to an Isolation Forest. The proposed system extends this idea into a complete three-layer implementation by adding a supervised attack classifier, decision fusion, automated blocking and a watchdog, all running on low-cost hardware for home and small-office networks.

3.2.4 Self-Healing Hybrid Intrusion Detection System: An Ensemble Machine Learning Approach

A C5 decision tree acts as the signature stage, an LSTM detects anomalies, and features of detected anomalies are turned into new signatures that are fed back into the first stage.

3.2.4.1 Summary of the Research Item

Kushal et al. [4] proposed a three-stage self-healing hybrid IDS. In the first stage, a C5 decision tree trained on UNSW-NB15 classifies packets as normal or attack and acts as the signature-based detector. Packets that it does not match are passed to the second stage, an LSTM recurrent neural network trained on ADFA-LD, which flags deviations from normal behaviour. In the third stage, the attributes of anomalous packets are extracted and added to the training data of the C5 stage, so the system learns new signatures and can detect similar attacks earlier. The C5 classifier reached a true positive rate of about 97% with a false positive rate of about 8%, while the LSTM reached a detection rate of about 90% with a false alarm rate of 17% and an AUC of 0.936.

3.2.4.2 Critical Analysis of the Research Item

The strengths and weaknesses of this work are:

- Turning detected anomalies into new signatures is an interesting way to keep the detector up to date without manual work.
- The two stages were evaluated separately on different datasets, and the authors note that the performance of the integrated system still needs to be assessed.
- The deep learning stage is computationally expensive and has a high false alarm rate, so wrongly learned signatures could classify dangerous traffic as normal or block benign traffic.
- Real-time testing on live networks is left as future work.

3.2.4.3 Relationship to the Proposed Research Work

The proposed NIDPS shares the goal of adapting to the local network, but keeps a fixed, verified set of models on the device and adapts only the anomaly threshold through calibration on normal traffic. This keeps the system adaptable while avoiding the cost and risk of learning new signatures online.

3.2.5 Comparative Evaluation of Machine Learning and Signature-Based NIDS for Multi-Class and Binary Threat Detection

Snort is compared with Random Forest, XGBoost and Decision Tree on UNSW-NB15. The ensemble models clearly outperform Snort in accuracy and false positive rate.

3.2.5.1 Summary of the Research Item

Wen-Udeoji, Muyeba and Mohammadi [5] built an empirical framework to compare the signature-based Snort NIDS with three machine learning models, Random Forest, XGBoost and Decision Tree, for both binary and multi-class threat detection on the UNSW-NB15 dataset while keeping its real class distribution. Random Forest reached 87% accuracy compared with 56.21% for Snort, with a false positive rate of 25% against 62.5% for Snort. Decision Tree and XGBoost reached 80.2% and 82.84% accuracy respectively. The authors conclude that the results support hybrid strategies that combine signature-based and machine learning detection.

3.2.5.2 Critical Analysis of the Research Item

The strengths and weaknesses of this work are:

- It is a recent, direct comparison of a signature engine with tree-based ML models on the same data.
- Keeping the natural class distribution gives a more realistic picture than artificially balanced data.
- Only one dataset is used, and the hybrid approach that the authors recommend is not implemented or evaluated.
- Real-time performance and resource use are not reported.

3.2.5.3 Relationship to the Proposed Research Work

The results support the choice of tree ensembles (Random Forest and XGBoost) for Layer 3 of the proposed NIDPS and the decision to keep the signature engine and the ML models as separate layers whose outputs are fused, rather than relying on either one alone.

3.2.6 Real-Time Hybrid Signature and Machine Learning Intrusion Detection

A signature engine is combined with deep learning models for real-time detection, and the decisions of both detection stages are combined after each stage has produced its own output.

3.2.6.1 Summary of the Research Item

Naik et al. [6] presented a real-time hybrid intrusion detection approach in which a signature-based engine detects known attacks and deep learning models detect anomalous behaviour. Each detection stage produces its own decision, and their experiments showed that combining these decisions at the end (late fusion) gives better results than passing the output of one stage as input to the other.

3.2.6.2 Critical Analysis of the Research Item

The strengths and weaknesses of this work are:

- It shows the benefit of late fusion of independent detector outputs in a real-time setting.
- Deep learning models need more processing power, which makes the approach harder to run inline on low-cost hardware.
- Automated prevention, encrypted traffic and cross-dataset evaluation are not addressed.

3.2.6.3 Relationship to the Proposed Research Work

The proposed NIDPS also uses late fusion across its three layers, but with optimised tree models instead of deep learning. Each layer's output is recorded, so the system can show which layer made the decision and why before deciding whether to pass, alert or block.

3.2.7 Improving Detection Accuracy of Network Intrusions Using a Hybrid NIDS Based on Isolation Forest and Random Forest

Suricata is combined with an Isolation Forest that flags anomalous traffic and a Random Forest that identifies the attack type. It reaches 96.33% accuracy on CIC-IDS-2017.

3.2.7.1 Summary of the Research Item

Wang and Avrianto [7] proposed a hybrid NIDS in which Suricata performs real-time packet inspection, an Isolation Forest flags anomalous traffic and a Random Forest identifies the family of the attack. The models were trained on pre-processed captured traffic and evaluated on CIC-IDS-2017 and on simulated attack data. The hybrid model reached 96.33% accuracy on CIC-IDS-2017 and 99.86% on the simulated data, and compared with standalone Suricata it detected more unknown threats with fewer false positives.

3.2.7.2 Critical Analysis of the Research Item

The strengths and weaknesses of this work are:

- It is the closest published design to the proposed system and shows that Isolation Forest and Random Forest work well together.
- The evaluation on CIC-IDS-2017 covers a limited subset of attacks, mainly DoS and port scanning.
- The very high accuracy is reported on simulated data, which may not reflect real network traffic.
- The system detects but does not block traffic, and failure handling is not considered.

3.2.7.3 Relationship to the Proposed Research Work

This is the most similar approach to the proposed NIDPS. The proposed system keeps the Isolation Forest and Random Forest pairing, but places Suricata inline in IPS mode as the first layer, adds automated blocking and failure handling, and is evaluated on three public datasets together with a self-collected dataset.

3.2.8 Extended Isolation Forest for Intrusion Detection in Zeek Data

The hyperparameters of the Extended Isolation Forest are tuned to detect malicious connections in Zeek logs. The simplest configuration (extension level 0) performs best.

3.2.8.1 Summary of the Research Item

Moomtaheen et al. [8] tuned the hyperparameters of the Extended Isolation Forest (EIF), a variant of Isolation Forest, to detect malicious traffic in Zeek connection logs. The authors created a new dataset, UWF-ZeekDataFall22, and used Hadoop for storage and Apache Spark for training. The model reached accuracies of 82.3% for the Resource Development tactic, 82.21% for Reconnaissance and 78.3% for Discovery. The best results came from extension level 0, which is equivalent to the standard Isolation Forest.

3.2.8.2 Critical Analysis of the Research Item

The strengths and weaknesses of this work are:

- It uses real Zeek connection logs labelled with MITRE ATT&CK tactics, which is closer to real deployments than older benchmarks.
- It shows that the extended variant gives no benefit over the basic Isolation Forest on this data.
- Accuracy around 80% is moderate, so Isolation Forest alone is not enough for reliable detection.
- The big-data pipeline (Hadoop and Spark) is not suited to small, resource-constrained devices.

3.2.8.3 Relationship to the Proposed Research Work

This study supports the decision to use the standard Isolation Forest in Layer 2 of the proposed NIDPS rather than a more complex variant, and to combine it with a signature layer and a supervised classifier because its accuracy on its own is moderate.

3.2.9 An Optimized Isolation Forest Based Intrusion Detection System for Heterogeneous and Streaming Data in IIoT Networks

An Isolation Forest tuned with a modified Harris Hawks Optimisation (ERHHO) reduces features and training time and handles streaming data with concept drift.

3.2.9.1 Summary of the Research Item

Elsaid and Binbusayyis [9] proposed OIFIDS, an Isolation Forest based intrusion detection system for Industrial IoT networks. The Isolation Forest is optimised with ERHHO, a modified Harris Hawks Optimisation algorithm, which reduces the number of features and the training time. The system was evaluated on NSL-KDD, UNSW-NB15 and CSE-CIC-IDS2018, reaching accuracies of 95.6%, 94.8% and 99% respectively. On streaming data with concept drift it achieved AUC values of 0.948, 0.922 and 0.97 on the three datasets.

3.2.9.2 Critical Analysis of the Research Item

The strengths and weaknesses of this work are:

- It addresses streaming traffic and concept drift, which are real problems for a deployed detector.
- Feature reduction lowers computational complexity, which suits resource-constrained devices.
- The optimisation step adds complexity to training, and the results are still on public benchmarks rather than a live deployment.

3.2.9.3 Relationship to the Proposed Research Work

The proposed NIDPS also has to cope with traffic that changes over time. Instead of a metaheuristic optimiser, it handles this with feature selection and a periodic recalibration of the Isolation Forest threshold on the local network's normal traffic.

3.2.10 Improved Isolation Forest with Sparse Auto-Encoder Feature Extraction

A sparse auto-encoder extracts features and a new split-point rule improves the Isolation Forest, giving better accuracy, recall, F1-score and AUC on KDDCUP99.

3.2.10.1 Summary of the Research Item

Yu et al. [10] improved the Isolation Forest for intrusion detection in two ways. First, a sparse auto-encoder is used to extract better features from the raw traffic records. Second, a new rule is used to choose the split points of the isolation trees. Evaluated on the KDD-CUP99 dataset, the improved model achieved higher accuracy, recall, F1-score and AUC than the standard Isolation Forest.

3.2.10.2 Critical Analysis of the Research Item

The strengths and weaknesses of this work are:

- It shows that feature quality has a large effect on how well Isolation Forest separates attacks.
- KDDCUP99 is outdated and contains many redundant records, so the results may not hold on modern traffic.
- The auto-encoder adds training and inference cost compared with the plain Isolation Forest.

3.2.10.3 Relationship to the Proposed Research Work

This work confirms that the proposed NIDPS should invest in good flow features for its Isolation Forest. The proposed system obtains them from a consistent flow extractor and feature selection rather than from a neural auto-encoder, keeping the anomaly layer lightweight.

3.2.11 An Improved X-Means and Isolation Forest Based Methodology for Network Traffic Anomaly Detection

X-iForest removes the need to know the anomaly ratio in advance by clustering the suspected anomalies. It improves average AUC by 8.1% over the basic Isolation Forest.

3.2.11.1 Summary of the Research Item

Feng et al. [11] addressed a common problem of Isolation Forest: the contamination parameter (the share of anomalies) must be guessed, although the true ratio is unknown in real traffic. Their method, X-iForest, first runs an Isolation Forest with a deliberately high contamination value, then measures the distance of each suspected anomaly from the centre of the normal data, removes extreme values and uses X-means and 2-means clustering to separate real anomalies from normal points. Across four simulated and four public datasets, the average AUC was 8.1% higher than the basic Isolation Forest, and the method had the best AUC on six of the eight datasets.

3.2.11.2 Critical Analysis of the Research Item

The strengths and weaknesses of this work are:

- It removes the need to guess the contamination parameter, which is a real practical issue.
- The evaluation datasets are general anomaly-detection datasets and simulated Gaussian data, not network intrusion datasets.
- Performance is weaker on small datasets, and the method has not been tested on live traffic.

3.2.11.3 Relationship to the Proposed Research Work

The proposed NIDPS faces the same problem of setting the anomaly threshold without knowing the true anomaly ratio. It solves it by calibrating the Isolation Forest threshold on a period of normal traffic from the local network, which is simpler to run on the device.

3.2.12 Intrusion Detection: A Comparison Study of Machine Learning Models Using Unbalanced Dataset

Five classifiers are compared on the imbalanced UNSW-NB15 dataset. Random Forest and XGBoost give the best overall results.

3.2.12.1 Summary of the Research Item

Ajagbe, Awotunde and Florez [12] compared five machine learning models, Linear SVC, Logistic Regression, Random Forest, Decision Tree and XGBoost, on the UNSW-NB15 dataset, in which about 92.7% of the records are normal and 7.3% are attacks. Random Forest and XGBoost performed best in accuracy, precision, recall and F1-score. The AUC-ROC was 0.91 for Random Forest and Linear SVC, 0.90 for XGBoost, 0.83 for Decision Tree and 0.70 for Logistic Regression.

3.2.12.2 Critical Analysis of the Research Item

The strengths and weaknesses of this work are:

- It evaluates the models on imbalanced data, which reflects real networks where attacks are rare.
- The way oversampling and undersampling were applied is not clearly described.
- The study was not tested in a real-world setting and uses a single dataset.

3.2.12.3 Relationship to the Proposed Research Work

This comparison supports the choice of Random Forest and XGBoost for the attack classifier (Layer 3) of the proposed NIDPS, and highlights class imbalance as an issue that the training pipeline must handle.

3.2.13 Enhanced Intrusion Detection Systems Performance with UNSW-NB15 Data Analysis

Four supervised classifiers are compared after correlation-based feature selection. Random Forest performs best with an F1-score of 97.80%.

3.2.13.1 Summary of the Research Item

More et al. [13] compared Logistic Regression, SVM, Decision Tree and Random Forest on UNSW-NB15 after exploratory data analysis, correlation-based feature selection and random sampling. Random Forest was the best model, with an F1-score of 97.80%, an accuracy of 98.63% and a false alarm rate of 1.36%.

3.2.13.2 Critical Analysis of the Research Item

The strengths and weaknesses of this work are:

- It shows strong Random Forest results with a low false alarm rate after simple feature selection.
- Results come from a single dataset, so they say little about generalisation to other networks.
- Detection latency and resource use are not reported.

3.2.13.3 Relationship to the Proposed Research Work

The study supports the use of Random Forest in Layer 3 of the proposed NIDPS. The proposed system goes further by also measuring latency and resource use and by testing the model across datasets.

3.2.14 An Optimized Gradient Boosting Framework for IoT Intrusion Detection on the CICIoT2023 Dataset

XGBoost, LightGBM and CatBoost are evaluated on CICIoT2023 with a focus on edge deployment. Reducing 46 features to 23 cuts inference latency by 30–51% with almost no loss of accuracy.

3.2.14.1 Summary of the Research Item

Almahaqeri et al. [14] proposed an optimised gradient boosting intrusion detection framework for IoT environments, with an emphasis on practical deployment at the edge. An intrusion detection system on an edge gateway must not only detect attacks accurately but also run efficiently with limited computing power. The framework was evaluated on the CICIoT2023 dataset, comparing XGBoost, LightGBM and CatBoost for binary, attack-family (8-class) and fine-grained (34-class) classification.
Training is performed offline, while the deployed edge system performs online flow classification. To reduce the processing cost, the authors apply gain-based feature selection and reduce the 46 features to 23, which lowered inference latency by around 30–51% and training time by 33–40% with almost no loss in predictive performance. XGBoost achieved 99.61% accuracy and a macro F1-score of 0.995 in binary detection, taking 0.355 microseconds per sample.

3.2.14.2 Critical Analysis of the Research Item

The strengths and weaknesses of this work are:

- It takes into account both the deployment efficiency and detection accuracy, which many intrusion detection works failed to consider.
- It demonstrates that reducing the feature set significantly increases the inference speed without hurting the accuracy, which is particularly suitable for edge computing scenarios.
- The evaluation was restricted to the CICIoT2023, therefore, the extent to which the method can be generalized to other network types has yet to be demonstrated, however, the authors have pointed out that cross-validation on different datasets and streaming conditions are possible future work.
- Edge-suitability was evaluated in terms of latency rather than actual deployment on a low-power gateway and only the flow-level features are used.

3.2.14.3 Relationship to the Proposed Research Work

This work is closely related to the proposed NIDPS, which also performs detection on a resource-constrained Linux gateway. Like the paper, the proposed system separates offline training from online inference and treats latency and resource use as evaluation criteria. Unlike this study, it also includes signature-based and anomaly detection, cross-dataset evaluation, automated blocking, failure handling and testing on traffic from the actual deployment.

3.2.15 Cross-Dataset Generalisation of Deep Learning Intrusion Detection Models

Deep IDS models that score about 99% on the dataset they were trained on drop to roughly 50% when tested on a different dataset.

3.2.15.1 Summary of the Research Item

Xin and Xu [15] examined how well intrusion detection models generalise beyond the dataset they were trained on. They report that deep IDS models which reach around 99% on their own dataset often fall to about 50%, close to random guessing, when evaluated on other datasets. This shows that high benchmark scores do not guarantee good performance on a different network.

3.2.15.2 Critical Analysis of the Research Item

The strengths and weaknesses of this work are:

- It exposes a major weakness of single-dataset evaluation, which most reviewed studies rely on.
- It focuses on detection accuracy and does not propose a deployable system or measure runtime cost.

3.2.15.3 Relationship to the Proposed Research Work

This finding is the main reason why the proposed NIDPS is trained on CIC-IDS-2017, CSE-CIC-IDS2018 and CIC-IoT-2023 with a common flow extractor, evaluated across datasets, and validated on traffic collected from the deployment network itself.

3.3 Literature Review Summary Table

Table 3.1 gives a general overview of the studies reviewed in Section 3.2, listing the method, dataset, main results and limitations of each work.

Table 3.1: Summary of the reviewed literature.

Title	Year	Method	Dataset	Results	Limitations
Performance comparison of IDSs and application of ML to Snort [1]	2018	Snort vs Suricata at 10 Gbps; SVM (firefly-optimised) filtering Snort alerts	Test-bed traffic (10 Gbps)	FPR 8.6%, FNR 2.2%; Suricata handles high load better	Offline SVM; no blocking

Title	Year	Method	Dataset	Results	Limitations
Hybrid IDS integrating anomaly with misuse detection [2]	2014	C4.5 decision tree + multiple one-class SVMs	NSL-KDD	Higher detection of known and unknown attacks; ~50% of training time	Old dataset; offline only
Collaborative hybrid NIDS based on Suricata and Isolation Forest [3]	2019	Suricata + Isolation Forest; alerts shared via central log	Not reported (design proposal)	Lower computational cost (design)	No classifier; limited evaluation
Self-healing hybrid IDS: an ensemble ML approach [4]	2024	C5 decision tree + LSTM; anomalies turned into new signatures	UNSW-NB15, ADFA-LD	C5: ~97% TPR, 8% FPR; LSTM: 90% DR, AUC 0.936	Stages tested separately; costly DL
Comparative evaluation of ML and signature-based NIDS [5]	2025	Snort vs RF, XGBoost, DT	UNSW-NB15	RF 87% vs Snort 56.21% accuracy; FPR 25% vs 62.5%	Single dataset; hybrid not built
Real-time hybrid signature and ML detection [6]	2025	Signature engine + deep learning, late fusion	Not reported	Late fusion better than chaining stages	Heavy DL models; no prevention
Hybrid NIDS based on Isolation Forest and Random Forest [7]	2025	Suricata + Isolation Forest + Random Forest	CIC-IDS-2017, simulated	96.33% (CIC-IDS-2017), 99.86% (simulated)	Small attack subset; no blocking
Extended Isolation Forest for intrusion detection in Zeek data [8]	2024	Extended Isolation Forest, Spark	UWF-Zeek-DataFall22	78.3–82.3% accuracy; level 0 (basic IF) best	Moderate accuracy; big-data stack
Optimized Isolation Forest IDS for IIoT networks [9]	2024	Isolation Forest tuned with ERHHO; streaming data	NSL-KDD, UNSW-NB15, CSE-CIC-IDS2018	Accuracy 95.6%, 94.8%, 99%; streaming AUC up to 0.97	Complex optimisation; no live test
Improved Isolation Forest based on split points [10]	2024	Sparse auto-encoder features + new split rule	KDDCUP99	Higher accuracy, recall, F1 and AUC than IF	Outdated dataset; extra cost
X-means and Isolation Forest for traffic anomaly detection [11]	2022	X-iForest: IF + distance + X-means clustering	4 simulated + 4 public sets	Average AUC +8.1% over IF; best on 6 of 8	Not intrusion data; weak on small sets

Title	Year	Method	Dataset	Results	Limitations
Comparison of ML models using unbalanced dataset [12]	2024	Linear SVC, LR, RF, DT, XGBoost	UNSW-NB15	RF and XGBoost best; AUC 0.91 (RF), 0.90 (XGB)	Single dataset; not tested in real setting
Enhanced IDS performance with UNSW-NB15 data analysis [13]	2024	LR, SVM, DT, RF with correlation feature selection	UNSW-NB15	RF: F1 97.80%, accuracy 98.63%, FAR 1.36%	Single dataset; no latency
Optimized gradient boosting framework on CICIoT2023 [14]	2026	XGBoost, LightGBM, CatBoost; gain-based feature selection	CICIoT2023	XGBoost 99.61%, macro F1 0.995; 30–51% lower latency	One dataset; no device deployment
Cross-dataset generalisation of deep IDS models [15]	2025	Transformer IDS with calibration; cross-dataset evaluation	NSL-KDD, UNSW-NB15, CIC-IDS2017	~99% within-dataset drops to near chance across datasets	No deployable system

3.4 Data Set Summary Table

Table 3.2 gives an overview of the datasets used in the reviewed studies and how relevant each one is to the proposed NIDPS.

Table 3.2: Datasets used in the reviewed studies.

Sr. No	Dataset	Year	Environment / Traffic	Used in	Relevance to the proposed NIDPS
1	KDDCUP99	1999	Simulated military network (DARPA 1998)	[10]	Historical baseline only; outdated and redundant
2	NSL-KDD	2009	Cleaned version of KDDCUP99	[2, 9]	Still based on 1998 traffic; no modern attacks
3	UNSW-NB15	2015	Hybrid real and synthetic traffic, 9 attack families	[4, 5, 9, 12, 13]	Common benchmark; class imbalance
4	ADFA-LD	2013	Host-based Linux system-call traces	[4]	Host-level data; not used for flows
5	CIC-IDS-2017	2017	5 days of enterprise-style traffic	[7]	Used for training and cross-evaluation
6	CSE-CIC-IDS2018	2018	Larger AWS-hosted network, 10 days	[9]	Used for training and cross-evaluation
7	CICIoT2023	2023	105 IoT devices in a smart-home topology	[14]	Closest to home/IoT target; used for training
8	UWF-Zeek-DataFall22	2022	Zeek connection logs labelled with MITRE ATT&CK	[8]	Shows value of real Zeek-style logs

3.5 Comparison with Closest Existing Systems

The systems reviewed above that come closest to the proposed NIDPS are compared in Table 3.3. The table shows which detection layers each system uses and whether it addresses inline blocking, encrypted traffic, evaluation on multiple datasets or real traffic, low-cost hardware and failure handling.

Table 3.3: Comparison of the closest existing systems with the proposed NIDPS.

System	Signature layer	Anomaly layer	Classifier	Inline blocking	Encrypted traffic	Multi-dataset / real traffic	Low-cost hardware	Failure handling
Wang and Avrianto [7]	Yes (Suricata)	Yes (IF)	Yes (RF)	No	No	No	No	No
Chiba et al. [3]	Yes (Suricata)	Yes (IF)	No	Proposed	No	Not evaluated	No	No
Kim, Lee and Kim [2]	Misuse model (C4.5)	Yes (OC-SVM)	Yes (DT)	No	No	No (NSL-KDD)	No	No
Kushal et al. [4]	Learned (C5)	Yes (LSTM)	Yes (C5)	No	No	Partial	No	No
Wen-Udeoji et al. [5]	Yes (Snort)	No	Yes (RF, XGB, DT)	No	No	No	No	No
Naik et al. [6]	Yes	Yes (deep)	n/r	n/r	No	n/r	No	No
Shah and Issac [1]	Yes (Snort)	No	Yes (SVM)	No	No	n/r	No (server)	No
Proposed NIDPS	Yes (Suricata IPS)	Yes (IF, calibrated)	Yes (RF/XGB)	Yes (ipset)	Yes (JA3, flow stats)	Yes (3 public + own)	Yes (Raspberry Pi 4)	Yes (watchdog, 3 modes)

n/r: not reported. IF: Isolation Forest, RF: Random Forest, XGB: XGBoost, DT: Decision Tree, OC-SVM: One-Class SVM.

The existing systems mainly address one or two of these issues, whereas the proposed system improves on them in five ways:

- **Complete system:** Previous works either propose a design or only detect traffic. The proposed system detects, blocks and applies safeguards, while keeping forwarding latency close to that of a regular router.
- **Three layers with fusion:** Suricata catches known attacks first, an Isolation Forest tuned to local traffic spots anomalous behaviour, and a Random Forest/XGBoost classifier identifies the attack family. Their outputs are fused so that the system records what each layer decided and why.
- **Lightweight models:** Unlike many deep learning systems, the proposed system uses lightweight tree models that run beside the forwarding path rather than in it. They check each flow once it is complete, or every few seconds for long-running flows, so they add no delay to forwarded traffic. Latency, throughput and resource use are measured.

- **Cross-dataset evaluation:** Because most models fail on unseen datasets, the system is trained on CIC-IDS-2017, CSE-CIC-IDS2018 and CIC-IoT-2023 using one flow extractor and validated on self-collected household traffic.
- **Encrypted traffic and failure handling:** The system analyses encrypted connections using JA3/JA3S fingerprints and flow metadata without decryption, and uses a watchdog with configurable fail-open, fail-closed and degraded modes. None of the reviewed systems addresses either problem.

The contribution of the proposed project is therefore not the idea of combining signature detection with machine learning, which has been researched extensively, but turning that idea into a practical, deployable system for home and small-office networks.

3.6 Conclusion

The reviewed literature shows a clear direction: signature engines such as Snort and Suricata detect known attacks reliably but miss new ones and raise many false alarms, while machine learning detects unknown behaviour but is rarely deployed inline. Hybrid systems that let each technique handle the traffic it suits best, and that fuse their decisions at the end, give the best results [2, 5, 6]. Isolation Forest is a cheap and effective anomaly detector when it is given good features and a sensible threshold [8, 10, 11], and tree ensembles such as Random Forest and XGBoost are the strongest and fastest choice for classifying known attack types [12, 13, 14].
The review also exposed common gaps. Most studies use a single, often outdated dataset, and models that score close to 99% on one dataset can fall to about 50% on another [15]. Few works report latency or resource use, few implement automated blocking, and none of the reviewed systems analyses encrypted traffic or defines what happens when a component fails. The proposed NIDPS is designed around these gaps: it combines Suricata, a calibrated Isolation Forest and a Random Forest/XGBoost classifier with late fusion, blocks confirmed attackers automatically, analyses encrypted traffic through flow metadata and JA3/JA3S fingerprints, is evaluated across three public datasets and real traffic, and runs on low-cost hardware with defined failure modes.

Chapter 4

Software Requirement Specifications

This part will explain the requirements of the proposed Network Intrusion Detection and Prevention System. It will go through the workings of the system, what its required benchmarks are and the conditions in which it can be used. The level of detail will be such that it can be properly built and tested. We start with a feature list and functional requirements which have been categorized. Further on we discuss quality attributes and non-functional requirements followed by assumptions regarding the system. Interactions between different users and the system are modeled as use cases. The chapter closes with the hardware and software requirements, the design of the graphical user interface, the database design and an analysis of project risks. Requirements are given identifiers (for example FR-2.1) so that they can be traced to the design in Chapter 6 and to the tests in Section 6.6.6.
The system has five classes of users or actors, which are referred to throughout this chapter:

- **Network Administrator:** the owner of the home or small-office network who installs the NIDPS and manages it through the web dashboard. This user is not assumed to be a security expert.
- **Network User (client device):** any phone, laptop or IoT device that connects to the NIDPS access point. These users do not interact with the NIDPS directly; they are protected by it.
- **Attacker:** an external host or a compromised device inside the network that generates malicious traffic. The attacker triggers detection and prevention use cases.
- **System Scheduler / Watchdog:** internal time- and event-driven actors that run scheduled jobs (rule updates, threshold calibration, data purging) and supervise the health of every component.
- **Model Maintainer:** a project team member who trains and validates machine learning models offline and publishes model packages to the device.

4.1 List of Features

The NIDPS provides the following features. Each feature is refined into functional requirements in Section 4.2.

Table 4.1: Features of the proposed NIDPS.

ID	Feature	Description
NIDPS-1	Inline wireless access point	Acts as the visible network in the system. Any device wanting to connect to the protected network will see this and establish a connection to it instead.
NIDPS-2	Detection based on signatures (Layer 1)	Checks every packet that passes through using Suricata.
NIDPS-3	TLS fingerprints and flow extraction	Groups up related packets and assigns them 78 features. It can identify details from encrypted packets as well.
NIDPS-4	Anomaly detection (Layer 2)	An Isolation Forest will be trained on normal traffic relative to the local network it is placed in and will be able to detect an anomaly score for traffic.
NIDPS-5	Classification of attacks (Layer 3)	Once attacks or anomalies have been detected, they are then reused and classified into specific families through Random Forest or XGBoost.
NIDPS-6	Decision fusion	Collects the decisions from each layer and decides whether traffic may pass through or the device should be blocked due to frequent anomalous traffic.
NIDPS-7	Dynamic blocking	It will block identified hosts that are dangerous by blocking their IP address, there will be a specific whitelist as well for unique cases. Features eventual expiry as well.
NIDPS-8	Recovery mechanism	Takes care of all components with a watchdog system, if some system fails the watchdog may close the network, or let traffic pass through free depending on configuration.
NIDPS-9	Monitoring	An interface available on the network's LAN which will show statistics and alerts including all configuration settings.
NIDPS-10	Configurations	The administrator can edit or turn off the layers, change thresholds for detection, change failsafe actions and manage black/whitelists.
NIDPS-11	Logging and storage	Stores alerts, traffic logs, configurations in files and databases.
NIDPS-12	Training and versioning	The system can train and test its models using the collected data. Each version is tagged so it can be returned to.

ID	Feature	Description
NIDPS-13	Auto maintenance	Rules and tracking features are updated daily and thresholds updated. Older data outside of the limit deleted.

4.2 Functional Requirements

The functional requirements below describe what the system must do. They are grouped by the features listed in Section 4.1, and each requirement is numbered according to its feature.

4.2.1 NIDPS-1: Inline Wireless Access Point

FR-1.1 The system shall act as a WAP, and its network name will be configurable along with being password protected.

FR-1.2 The system shall assign IP addresses to any devices that are connected using DHCP.

FR-1.3 The system shall be acting as the DNS server for any connected devices.

FR-1.4 The system shall pass client traffic on to the ISP router using NAT over its wired connection.

FR-1.5 The system shall stop wireless clients from talking to each other directly, so that traffic between them also passes through the NIDPS.

4.2.2 NIDPS-2: Signature-Based Detection (Layer 1)

FR-2.1 The system shall send all forwarded traffic to Suricata so it can be checked before it is passed on.

FR-2.2 The system shall check every packet against the Emerging Threats Open ruleset.

FR-2.3 The system shall drop packets that match rules set to "drop" and allow all other packets through.

FR-2.4 The system shall use custom rules to detect SSH brute force, port scans, DNS spoofing, SYN floods and known malicious TLS fingerprints.

FR-2.5 The system shall detect ARP spoofing by comparing ARP replies with the devices it has assigned addresses to.

FR-2.6 The system shall pass every alert, with its source, destination, rule and severity, on to the other layers.

4.2.3 NIDPS-3: TLS Fingerprints and Flow Extraction

FR-3.1 The system shall group packets into flows, where a flow is a connection/conversation between devices.

FR-3.2 The system shall end a flow when it is closed, when inactive for a set amount of time, or when it has been active for a set amount of time.

FR-3.3 The system shall extract 78 features for flows from the packets that arrive.

FR-3.4 The system shall read identifying details (JA3, JA3S and SNI) from encrypted TLS connections and attach them to the flow, without decrypting the traffic.

4.2.4 NIDPS-4: Anomaly Detection (Layer 2)

FR-4.1 The system shall check that each flow matches the features expected by the current model and reject any that do not.

FR-4.2 The system shall give each flow an anomaly score between 0 and 1 using an Isolation Forest.

FR-4.3 The system shall mark a flow as anomalous when its score is above the anomaly threshold.

FR-4.4 The system shall set the anomaly threshold based on normal traffic from the local network, leaving out traffic linked to alerts or blocked devices.

FR-4.5 The system shall learn how much data each device normally sends out and flag it when a device sends far more than usual (possible data theft).

FR-4.6 The system shall flag devices that contact the same destination at regular intervals with similar-sized messages (possible C2 beaconing).

4.2.5 NIDPS-5: Classification of Attacks (Layer 3)

FR-5.1 The system shall sort every flow into a class using XGBoost or Random Forest. The possible classifications are DoS/DDoS, Botnet/C2, Infiltration, Spoofing, Reconnaissance, Benign.

FR-5.2 The system shall evaluate each classification and give it a confidence score.

FR-5.3 The system shall label a flow as "unknown" when the model's confidence is too low, instead of forcing it into a class.

4.2.6 NIDPS-6: Decision Fusion

FR-6.1 The system shall combine the results of all three layers to decide whether each flow should be passed (PASS), flagged (ALERT) or blocked (BLOCK).

FR-6.2 The system shall record which layers led to each decision and the evidence used.

FR-6.3 The system shall keep a risk score for each device that rises with suspicious activity and drops over time when the device behaves normally.

FR-6.4 The system shall block a device when its risk score reaches the block threshold.

4.2.7 NIDPS-7: Dynamic Blocking

FR-7.1 The system shall block a device that it has deemed risky by placing its IP address/MAC address into a block/blacklist.

FR-7.2 The system shall remove members from the block list once the set cooldown timer has passed.

FR-7.3 The system shall have an allow/whitelist, and it shall never block any devices on it. It must necessarily include the NIDPS itself, any routers, DNS and admin devices as well.

FR-7.4 The system shall double the block time each time the same device is blocked again within 24 hours, up to a set maximum.

FR-7.5 The system shall stop blocking and only raise alerts if too many blocks happen within one minute, and warn the administrator.

FR-7.6 The system shall log every block, skipped block and release.

4.2.8 NIDPS-8: Recovery Mechanism

FR-8.1 The system shall contain a watchdog per component which it must feed every few seconds.

FR-8.2 The system shall assume that a part of the system has failed if it misses some amount of watchdog checks or stops running completely, at which point it will try to restart it.

FR-8.3 The system shall, upon the failure of a component, switch to one of three modes: let all traffic through unfiltered, block all traffic or run with reduced functionality.

FR-8.4 The system shall switch to a low functionality mode when it is overwhelmed and switch back when it can handle the load.

FR-8.5 The system shall, upon failure, create logs that detail when it happened, in what mode and the affected packets before recovery.

4.2.9 NIDPS-9: Monitoring

FR-9.1 The system shall require the administrator to log in before the dashboard can be viewed.

FR-9.2 The system shall show live traffic statistics, alerts, blocks and types of attacks, updated at least every two seconds.

FR-9.3 The system shall show a list of alerts with time, source, destination, attack type, layer and confidence, which can be filtered.

FR-9.4 The system shall show the evidence behind each alert.

FR-9.5 The system shall list blocked devices with the reason and time left, and let the administrator unblock them.

FR-9.6 The system shall show traffic and risk scores for each connected device.

FR-9.7 The system shall show the health of each component, along with CPU, memory, temperature and uptime.

4.2.10 NIDPS-10: Configurations

FR-10.1 The system shall let the administrator turn each detection layer on or off while running.

FR-10.2 The system shall let the administrator choose the failure mode and watchdog settings.

FR-10.3 The system shall let the administrator change detection thresholds, block times and flow timeouts within safe limits.

FR-10.4 The system shall let the administrator add and remove devices from the allowlist.

FR-10.5 The system shall let the administrator reload the rules and recalibrate the anomaly threshold on demand.

FR-10.6 The system shall apply configuration changes without restarting, and log who made each change and what was changed.

4.2.11 NIDPS-11: Logging and Storage

FR-11.1 The system shall store any alert notifications, potential alerts and blocks, calculated risk scores, configuration data and health statuses in a log file or database.

FR-11.2 The system shall make sure not to store actual packet data during operation.

FR-11.3 The system shall let the admin select a block of time and export alerts, blacklists or other operational data in a preferable format such as CSV.

4.2.12 NIDPS-12: Training and Versioning

FR-12.1 The system shall build training data using the same flow extractor that runs on the device.

FR-12.2 The system shall train and test models offline.

FR-12.3 The system shall be able to save each model in a package, which will include what model it is, the extractor version, classes, feature list and training date.

FR-12.4 The system shall not be able to load a model that does not follow the flow extractor rules and architecture.

FR-12.5 The system shall keep the previous model so the administrator can return to it in one step.

4.2.13 NIDPS-13: Auto Maintenance

FR-13.1 The system shall download and test new rules every day, and keep using the old rules if the test fails.

FR-13.2 The system shall recalculate and adjust the anomaly threshold by looking at traffic at the end of a set time such as after a week.

FR-13.3 The system shall delete records about flows after a set time frame has passed, which will be a week by default.

4.3 Quality Attributes

The quality attributes below describe the general qualities the NIDPS must have and the design decisions that provide them. The measurable targets for each attribute are given as non-functional requirements in Section 4.4.

Table 4.2: Quality attributes of the NIDPS.

Attribute	Meaning for the NIDPS	How it is achieved
Performance	The NIDPS must not noticeably slow the network and must react to attacks quickly.	Only the kernel and Suricata sit in the forwarding path; ML runs on flows beside the path; lightweight tree models; ipset lookups in constant time.
Availability	A failure of the NIDPS must not leave the household without internet unless the owner chooses that.	By default, if the NIDPS fails, it will simply let traffic pass through it, it will try to restart any failed components and will have blocks that expire after a timeframe.
Reliability	The NIDPS should be able to work under high network traffic loads and be able to recover from faults.	Implement a watchdog that must be fed, create a degraded mode for heavy loads and log any failure and recovery occurrences.
Security	The NIDPS must be secure enough to survive attacks and must not attack its valid users.	Create a dashboard only accessible when authenticated through a LAN, add a limit to the block rate, models are loaded from a protected directory.

Attribute	Meaning for the NIDPS	How it is achieved
Privacy	Any traffic that is collected during operation should be kept private.	When traffic is passing through, only keep important information such as metadata or alerts with a window size limit.
Accuracy	Attacks should be detected with a good detection rate and few false alarms.	Three-layer design, fusing decisions to reach a conclusion, calibration to the current network.
Usability	Someone who is not well versed in networking should also be able to understand when alerts occur and how to act on them.	Alerts should be explainable in plain language, default settings should be clear and safe, one click to block/unblock threats and health indications.
Maintainability	The models, training thresholds and rules should be easy to build on or update.	Models should be tagged and versioned with the ability to roll back, rule and configuration setting files should be available.
Extensibility	New types of detectors or response types should be easily addable into the system without a full rewrite.	There should be a common interface for detectors along with a publish/subscribe event bus and JA3 extraction should be in its own module to be replaced/changed later.
Testability	Each layer and the full pipeline must be measurable.	Layers can be toggled at runtime for ablation; PCAP replay; fault injection; every decision logged with its evidence.
Portability	The system should run on different low-cost Linux devices.	Only requires a standard Linux install, with Python services. No part of the code requires specialized hardware.

4.4 Non-Functional Requirements

The NFRs will measure the statistics of how the system will perform. The goal is to allow the system to run on a Raspberry Pi with around 4 GB RAM or a mid-tier laptop to serve a home/small office network of 30 or more devices. These are design targets that will be tested, and adjusted if needed, during the evaluation in Section 5.10.

4.4.1 Performance

- **NFR-1** The system shall only add an upper limit of 200 ms of delay onto devices under normal loads.

- **NFR-2** The system shall be able to support 50 Mbps of traffic with all layers active.
- **NFR-3** The system shall analyze each flow and then assign a risk score to its device in no more than 15 ms.
- **NFR-4** The system shall, when it has recognized an anomalous device, block it within 1 second.
- **NFR-5** The system shall come to a decision regarding long-running connections in about a second after it has timed out.

4.4.2 Capacity

- **NFR-6** The system shall support up to 30 devices connected at the same time along with 5 people using the dashboard.

4.4.3 Resource Usage

- **NFR-7** The system shall use around a maximum of 70% of a lower-end Pi CPU and no more than 4 GB of memory at a given time.

4.4.4 Reliability and Availability

- **NFR-8** The system shall be able to detect failure and start resetting within 3 seconds. The system should not take more than 30 seconds to restart in normal conditions.
- **NFR-9** The system shall let all packets pass through it undiagnosed in case fail-open mode is activated to not interrupt the connection.

4.4.5 Accuracy

- **NFR-10** The system shall not raise false positives on more than 1% of normal traffic and not block any devices during a week of testing.
- **NFR-11** The system shall be able to detect more attacks than Suricata alone using its three-layered architecture, measured by macro F1-score.

4.4.6 Security

- **NFR-12** The system shall only allow access to the dashboard from the local network; it will store passwords in salted hashes and block in case of many incorrect login attempts.
- **NFR-13** The system shall be able to run all detection algorithms without admin privileges if they do not require network access.

4.4.7 Privacy

- **NFR-14** The system shall not save any specific data about packets or flows and delete them after a set timeframe (defaults to 1 week).

4.4.8 Usability

- **NFR-15** The system shall be simple enough that a non-technical user can understand what caused a failure and unblock it within a few minutes.
- **NFR-16** The system shall have a simplified explanation for the type of attack found, layer that detected it and other such information in alerts/logs.

4.4.9 Maintainability

- **NFR-17** The system shall allow new models and rules to be installed, or rolled back, without interrupting network traffic.

4.4.10 Extensibility and Reusability

- **NFR-18** The system shall allow a new detector to be added without changing any other component.
- **NFR-19** The system shall allow the flow extractor and training pipeline to be reused separately from the device.

4.4.11 Portability

- **NFR-20** The system shall be installed with a single script on any 64-bit Debian-based ARM or x86 device that meets the hardware requirements in Section 4.7.1.

4.5 Assumptions

The specification is based on the following assumptions. If one of them does not hold, the affected requirements may need to be revisited.

- The NIDPS is the only wireless network that client devices use. The ISP router's own Wi-Fi is disabled, and devices connected by cable directly to the ISP router are outside the protection of the system.
- The host is a mid-range laptop or a Raspberry Pi with at least 4 GB of RAM, a wireless adapter that supports access point mode and an Ethernet port for the uplink.
- The ISP router works correctly and provides internet access, but it is not relied on for any security function.

- The NIDPS host and the administrator's devices are trusted and have not been compromised.
- The administrator is not a security expert. Default settings must therefore be safe without tuning.
- Periodic internet access is available for ruleset updates.
- Most traffic is encrypted with TLS or QUIC, so payload inspection is not possible for the ML layers. Detection of encrypted traffic relies on flow metadata and handshake fingerprints only.
- The public datasets (CIC-IDS-2017, CSE-CIC-IDS2018 and CIC-IoT-2023) and their raw PCAP files remain available for downloading.
- Attack traffic for the self-collected dataset is generated only on an isolated network owned by the project team, against the team's own devices.
- Traffic observed during the calibration window is mostly benign; flows flagged by Suricata during this window are excluded.
- The household traffic volume does not exceed the measured capacity of the device for long periods; short peaks are handled by degraded mode.

4.6 Use Cases

This section describes the main use cases of the NIDPS. Figure 4.1 shows the use case diagram with the actors introduced at the beginning of this chapter. The use cases that represent the central functionality of the system, namely automatic detection and blocking, failure handling and the administrator's main tasks on the dashboard, are then described in detail in tabular form. The tables include a basic flow, which is the path followed in normal circumstances, and an alternate flow, where a suffix will represent replacing a basic flow step with that same number.

Table 4.3: Use case UC-01: Log In to Dashboard.

Name	**UC-01: Log In to Dashboard**
Actors	Network Administrator
Summary	The admin will enter their login credentials on the sign in page. Once verified they will be taken to the dashboard.
Pre-Conditions	The administrator's device is connected to the NIDPS network (LAN side). An administrator account exists. The administrator is not currently logged in.

Post-Conditions		Credentials are authenticated, a new session is established and the home page is displayed.	
Special Requirements		Passwords are stored as salted hashes. The account is locked for 5 minutes after 5 consecutive failed attempts (NFR-12).	
Basic Flow			
Actor Action		**System Response**	
1	The administrator will enter the address of the dashboard in the browser and open the page.	2	The webpage will display the login page, prompting for username and password.
3	The administrator will enter provided valid credentials and click Log In.	4	The system will verify the credentials and create a session. Along with this a record is created in the audit log and the home page is displayed.
Alternative Flow			
3	The administrator enters an invalid username or password.	4-A	The system shows the message "Incorrect username or password" and increments the failed-attempt counter.
3	The administrator enters wrong credentials for the fifth consecutive time.	4-B	The system locks the account for 5 minutes and shows the time remaining.

Table 4.4: Use case UC-02: View Live Network Statistics.

Name	**UC-02: View Live Network Statistics**
Actors	Network Administrator
Summary	The administrator views real-time throughput, flows per second, alert and block counts per detection layer, attack category distribution and per-device traffic.
Pre-Conditions	The administrator is logged in (UC-01).
Post-Conditions	Statistics are displayed and update after a certain period of time.
Special Requirements	Statistics refresh at least every 2 seconds (FR-9.2).
Basic Flow	
Actor Action	**System Response**

1	The administrator opens the Overview page.	2	The system displays current throughput, flows per second, alerts and blocks per layer and the attack category chart for the last hour.
3	The administrator selects a different time window (for example 24 hours).	4	The system reloads the charts and counters for the selected window.
5	The administrator selects a client device from the device list.	6	The system shows that device's traffic summary and current risk score.
Alternative Flow			
2	No traffic has been observed in the selected window.	2-A	The system shows empty charts with the message "No traffic in this period".
	The live connection to the server is lost.		The system marks the data as stale and retries the connection automatically.

Table 4.5: Use case UC-03: Review Alerts and Evidence.

Name	**UC-03: Review Alerts and Evidence**		
Actors	Network Administrator		
Summary	The administrator browses the alert list, filters it and opens an alert to see which layer detected it, the evidence behind the verdict and the originating flow.		
Pre-Conditions	The administrator is logged in (UC-01).		
Post-Conditions	The selected alert and its evidence are displayed. The alert may be marked as reviewed.		
Special Requirements	Each alert includes a plain-language description of the suspected attack (NFR-16).		
Basic Flow			
Actor Action		**System Response**	
1	The administrator will click to open the alerts section.	2	The system will display the most important statistics such as recent alerts, threats, layers detected, confidence and action taken.
3	The administrator filters by severity, layer, host or time range.	4	The system displays only the matching alerts.

5	The administrator selects an alert.	6	The system shows the evidence panel: Suricata signature, anomaly score and threshold, class probabilities, host risk history and the flow record.
7	The administrator marks the alert as reviewed.	8	The system records the review in the audit log.
Alternative Flow			
2	No alerts match the filter.	2-A	The system shows "No alerts match the selected filters".
3	The flow record has been deleted by the retention policy.	3-A	The system shows the stored evidence and notes that the flow details are no longer available.

Table 4.6: Use case UC-04: Detect and Block Malicious Host.

Name	**UC-04: Detect and Block Malicious Host**		
Actors	Attacker (initiator), System		
Summary	Traffic from an attacker is detected by one or more layers. The decision engine issues a BLOCK verdict, and the enforcer adds the attacker's address to the block set for a limited time and notifies the dashboard.		
Pre-Conditions	The NIDPS is running with at least one detection layer enabled. The attacker's traffic passes through the NIDPS.		
Post-Conditions	The attacker's address is in the block set with an expiry time. A record is logged and a notification appears in the dashboard regarding the block.		
Special Requirements	Blocking the malicious IP must happen no more than 1 second after the decision is made (NFR-4). IPs in the allowlist cannot be blocked (FR-7.3).		
Basic Flow			
Actor Action		**System Response**	
1	The attacker attempts to send harmful traffic.	2	Suricata will inspect the packets as they pass inline, the flow extractor will build connections using features and fingerprints.
	—	3	The first layer will activate an alert, the others may score the flow, and this will be published onto the event bus.

	—	4	The algorithm is run to combine the decisions, and it should return BLOCK along with which layers chose it.
	—	5	The enforcement component checks the allowlist and the rate limit, then computes the block duration. The address is added to ipset and the record is logged.
6	The attacker continues sending traffic.	7	Packets from the attacker are dropped in the kernel before reaching Suricata. The dashboard shows the new block.
Alternative Flow			
3	Evidence is weak (for example an anomaly score just above τ only).	3-A	The verdict is ALERT; evidence is added to the host's risk score, and the host is blocked only if its score reaches Θ.
4	The attacker's address is on the allowlist.	4-A	The block is suppressed and logged as "block suppressed"; an alert is still shown.
4	The block rate limit R_max has been reached.	4-B	The verdict is downgraded to ALERT and a "block rate limit" warning is shown on the dashboard.
4	The attacker was blocked before in the last 24 hours.	4-C	The block duration is doubled for each earlier offence, up to the maximum.

Table 4.7: Use case UC-05: Release Blocked Host.

Name	**UC-05: Release Blocked Host**
Actors	Network Administrator
Summary	The administrator reviews the list of blocked hosts and manually releases a host that was blocked by mistake.
Pre-Conditions	The administrator is logged in (UC-01). At least one host is currently blocked.
Post-Conditions	The host's address is removed from the block set and the release is recorded in the audit log.
Special Requirements	The released host may optionally be added to the allowlist (UC-06).
Basic Flow	
Actor Action	**System Response**

1	The administrator will open the page that contains the Blocked Hosts.	2	The system will list every blocked host with their details such as IP address, device name, attack type, layer detected, blocked time and remaining block time.
3	The administrator will pick a particular host and click release/unblock.	4	The system prompts for confirmation.
5	The administrator clicks confirm.	6	The system, upon confirmation, optionally adds the host to the allowlist, removes it from the ipset and marks/logs it as released.
Alternative Flow			
3	The block expired while the dialog was open.	3-A	The system informs the administrator that the block has already expired.
3	The administrator cancels.	3-B	No change is made.

Table 4.8: Use case UC-06: Manage Allowlist.

Name	**UC-06: Manage Allowlist**		
Actors	Network Administrator		
Summary	The administrator adds or removes IP or MAC addresses that must never be blocked, such as the gateway or the administrator's own laptop.		
Pre-Conditions	The administrator is logged in (UC-01).		
Post-Conditions	The allowlist is updated and immediately used by the enforcer.		
Special Requirements	The NIDPS host, gateway and DNS resolver are permanent entries and cannot be removed.		
Basic Flow			
Actor Action		**System Response**	
1	The administrator navigates to the Allowlist section and clicks on Add.	2	A form will be displayed by the system to insert address and description.
3	The administrator fills out the address and description forms and clicks on save.	4	The system validates the address and stores it. The changes are published to the bus and records are logged.
Alternative Flow			
2	The address is invalid or already listed.	2-A	A validation error message is shown and the system does not save anything.
2	The address is currently blocked.	2-B	The system asks whether to release the block as well (UC-05).

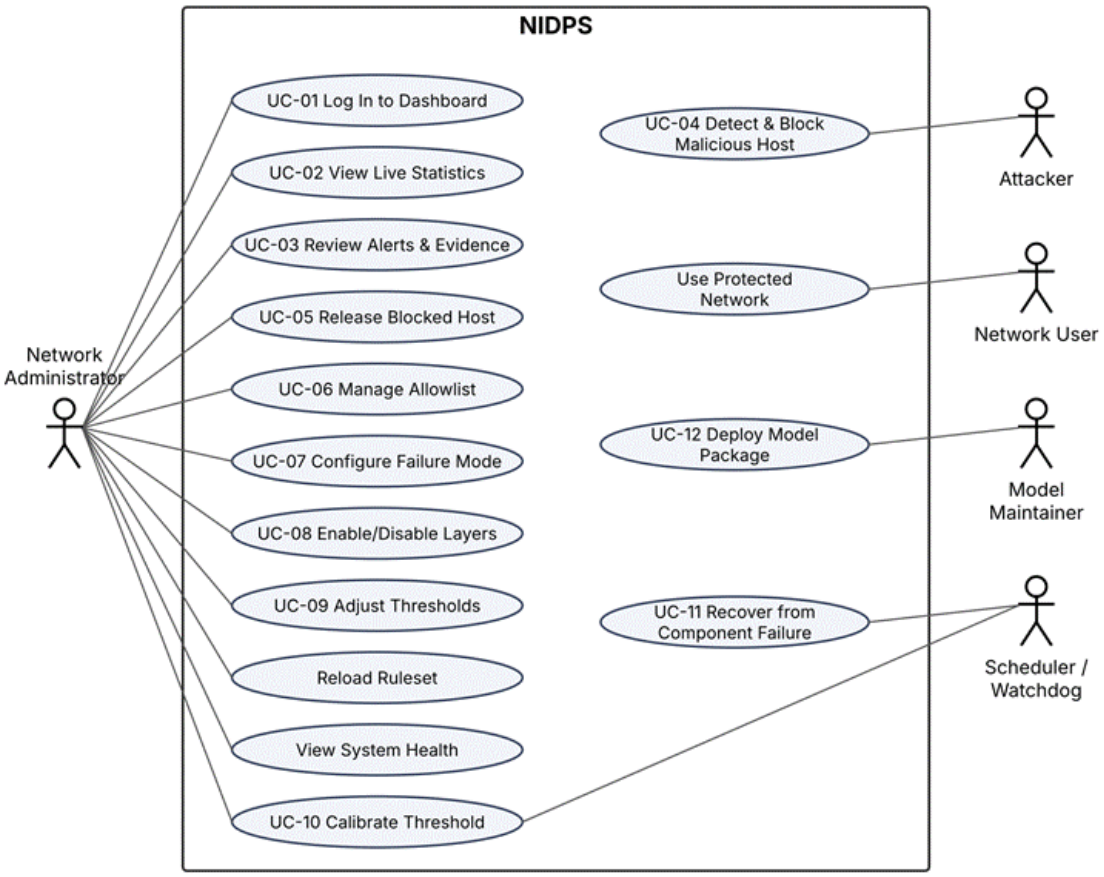


Figure 4.1: Use case diagram of the NIDPS.

Table 4.9: Use case UC-07: Configure Failure Mode.

Name		**UC-07: Configure Failure Mode**	
Actors		Network Administrator	
Summary		The administrator can choose network behavior under failure: either fail-open, fail-closed or degraded mode while under load.	
Pre-Conditions		The administrator is logged in (UC-01).	
Post-Conditions		The decision is recorded and the failure mode is applied to the kernel queue configuration.	
Special Requirements		Changing between fail-open and fail-closed modes will reload the inspection queue rule; packets will still be forwarded during this period.	
Basic Flow			
Actor Action		**System Response**	
1	The administrator clicks on Settings and then opens the Failure Mode tab.	2	The system will display what current mode it is using along with its specifications.
3	The administrator can click another mode to change it and then click save.	4	The system prompts for confirmation and explains the trade-offs of the mode being switched to.
5	The administrator clicks on confirm.	6	The system will update the queue-bypass config and save it. The changes will be logged, and a success message will be shown.
Alternative Flow			
3	The system cannot apply the new rule.	3-A	The previous mode remains active, and an error message is shown.

Table 4.10: Use case UC-08: Enable or Disable Detection Layers.

Name	**UC-08: Enable or Disable Detection Layers**
Actors	Network Administrator
Summary	The administrator turns individual detection layers (Suricata, Isolation Forest, classifier) on or off. This is also used to run the layer ablation experiments.
Pre-Conditions	The administrator is logged in (UC-01).
Post-Conditions	The layers will be either active or inactive, and the dashboard page shows their status.

Special Requirements		At all times, a minimum of one layer must be active unless the admin has authorized monitoring mode.	
Basic Flow			
Actor Action		**System Response**	
1	The administrator clicks on settings and opens the Detection Layers tab.	2	The system shows every layer with its current status and an on/off toggle.
3	The administrator toggles a layer off and clicks on save changes.	4	The system publishes the change; the decision engine stops using that layer's evidence and the dashboard marks it as disabled.
5	The administrator switches the layer back on.	6	The system resumes using the layer's evidence.
Alternative Flow			
2	The administrator tries to disable all layers.	2-A	The system displays a warning message that after this change the network will be fully unprotected and confirmation is required.

Table 4.11: Use case UC-09: Adjust Detection Thresholds.

Name	**UC-09: Adjust Detection Thresholds**		
Actors	Network Administrator		
Summary	The administrator changes detection sensitivity, such as the calibration percentile, classifier confidence thresholds, host block threshold and block durations.		
Pre-Conditions	The administrator is logged in (UC-01).		
Post-Conditions	The thresholds are then saved if they are valid, and all running services will now refer to the updated thresholds.		
Special Requirements	Every threshold has a specific valid range and a default value which can be restored by clicking restore default.		
Basic Flow			
Actor Action		**System Response**	
1	The administrator clicks on settings and then opens the Threshold section.	2	The system will show all the current set thresholds along with their valid ranges and recommended defaults.

3	The administrator changes any of the thresholds and clicks on save.	4	The system will validate the new values, save them along with making logs of the change. New values are made active.
Alternative Flow			
2	The value entered is outside of the acceptable range.	2-A	The system will not allow the change and will show the allowed values.
2	The administrator will click on restore all defaults.	2-B	The system restores all default values after confirmation.

Table 4.12: Use case UC-10: Calibrate Anomaly Threshold.

Name	**UC-10: Calibrate Anomaly Threshold**		
Actors	System Scheduler, Network Administrator		
Summary	The anomaly threshold τ is recalculated from the scores of clean local traffic so that it fits the deployment network. It runs weekly or on demand.		
Pre-Conditions	The Isolation Forest model is loaded. Flow records from the calibration window are available.		
Post-Conditions	A new calibration setting is saved with the threshold, the time logged and sample count. It will be used by Layer 2 from now on.		
Special Requirements	Any flows that Suricata has alerted or that have been blocked are not included. Threshold τ will not differ more than δ from the global threshold (Algorithm 5.4).		
Basic Flow			
Actor Action		**System Response**	
1	The system's scheduler triggers calibration on its own or the administrator clicks on the recalibration option.	2	The system will then gather anomaly scores of detected clean flows from the calibration window.
	—	3	The system will compute the p-th percentile, clip the value if needed to the valid range and then store the new τ value and publish it on the bus for use.
	—	4	The dashboard now shows the new τ value along with the date set and the number of flows used.

Alternative Flow			
2	Fewer than N_min clean flows are available.	2-A	The system keeps the global threshold and reports that more traffic is needed.

Table 4.13: Use case UC-11: Recover from Component Failure.

Name	**UC-11: Recover from Component Failure**		
Actors	Watchdog		
Summary	When any of the system's components crash or freeze, their portion of the watchdog is not reset/unfed. This is detected by the watchdog, and it will apply the configured failure mode, try to restart the component and record the recovery.		
Pre-Conditions	The watchdog is running and receiving heartbeats.		
Post-Conditions	The component will be back to working again, or the system will stay in a failed state, and the administrator will be sent an alert.		
Special Requirements	Detection will be within 3 seconds, and attempted restart should happen within 30 seconds (NFR-8). Fail-open is used by the kernel even if the watchdog has crashed.		
Basic Flow			
Actor Action		**System Response**	
	—	1	A component misses three consecutive heartbeats or its process exits.
	—	2	The watchdog detects a failure; it applies the configured fail mode and then logs the event.
	—	3	systemd successfully restarts the failed component; the watchdog will wait for a healthy heartbeat.
	—	4	The watchdog then marks the component as healthy and restores normal operation mode as well as logging the recovery time and packets missed or dropped.
5	The administrator views System Health.	6	The system shows the failure event and recovery details.
Alternative Flow			

3	systemd fails to restart the component after a while, or the maximum number of attempts is reached.	3-A	The system will stay in its failed state, and a visible warning is shown on the main dashboard.
1	Load exceeds the high watermark without a crash.	2-A	The watchdog enters degraded mode and returns to normal when load falls below the low watermark.

Table 4.14: Use case UC-12: Deploy Model Package.

Name	**UC-12: Deploy Model Package**		
Actors	Model Maintainer		
Summary	The developer trains a new model and validates it completely offline. It gets deployed to the device as a tagged/versioned package. The device will first check compatibility with all libraries/features before starting.		
Pre-Conditions	A package for a model update is produced by the offline training process (FR-12.3). The developer will have access to deploy onto the device.		
Post-Conditions	The new model will be made active on the device, and any previous versions will be kept as packages as a possible rollback option.		
Special Requirements	Any XGBoost models will be stored in a JSON native format. Packages are loaded from only the protected directory and no outside sources.		
Basic Flow			
Actor Action		**System Response**	
1	The maintainer copies the package to the model registry and runs the deploy command.	2	The system checks the feature list and extractor version against the running extractor.
	—	3	The system loads the new model, keeps the previous one, and shows the new version and its metrics on the dashboard.
Alternative Flow			
1	The feature schema does not match.	2-A	The system refuses the package, keeps the current model and reports the mismatch.
2	The maintainer runs the rollback command.	2-B	The system reactivates the previous package.

4.7 Hardware and Software Requirements

The NIDPS has two environments: the deployment environment, which is the device that runs inline on the network, and the development environment, which is used to build the software, train the models and generate test attacks. The requirements of both are listed below.

4.7.1 Hardware Requirements

Table 4.15: Hardware requirements.

Component	**Minimum / reference specification**	**Purpose**
NIDPS host	Any mid-level device, e.g. a laptop with 8 GB RAM or a Raspberry Pi 4 with 4 GB RAM	This device will function as the access point and will run Suricata, flow extraction, the scoring system, the decision system, enforcement and the watchdog, and will serve the dashboard
Storage	32 GB microSD (A2 class) or SSD	Operating system, rulesets, model packages, event database and logs
Wireless adapter	Pre-installed Wi-Fi card or USB Wi-Fi adapter that supports AP mode	Client-facing access point (wlan0)
Wired uplink	Gigabit Ethernet port and Cat5e/Cat6 cable	Connection to the ISP router (eth0)
Cooling and power	Official 5 V/3 A power supply; heatsink or fan case	Stable operation under sustained load without thermal throttling
ISP router / ONT	Existing ISP device (e.g. Huawei HG8546M) with built-in Wi-Fi disabled, or a mobile data source for testing	Upstream internet gateway
Attack host	Laptop or VM running Linux	Generating controlled attacks on the isolated test network
Test client devices	At least two phones/laptops	Benign traffic, victims of lateral movement and spoofing tests
Training workstation	Laptop/PC with $\geq$ 16 GB RAM (32 GB recommended), $\geq$ 200 GB free disk	Regenerating datasets from PCAPs and training models offline

4.7.2 Software Requirements

Table 4.16: Software requirements.

Category	Software (version)	Purpose
Operating system	Raspberry Pi OS 64-bit or another Debian-based Linux with systemd	Base platform for the NIDPS host
Access point	hostapd, dnsmasq	Wi-Fi access point, DHCP and DNS
Firewall and queueing	iptables (nftables backend), ipset, NFQUEUE kernel support	Routing, NAT, inline queue, blocking and failure modes
Signature IDS/IPS	Suricata 7 or later; Emerging Threats Open ruleset; suricata-update	Layer 1 inline detection, JA3/JA3S logging
Programming language	Python 3.11 or later	All custom services and training scripts
ML libraries	scikit-learn (Isolation Forest, Random Forest), XGBoost, NumPy, pandas, imbalanced-learn, joblib	Model training and inference
Packet processing	Scapy / libpcap bindings; CICFlowMeter-compatible feature definitions	Flow extraction and TLS handshake parsing
Event bus and storage	Redis (Streams); SQLite 3	Messaging between services; event database
Dashboard	FastAPI (or Flask) with WebSocket support; HTML/CSS/JavaScript with a charting library	Web interface and API
Supervision	systemd units and hardware watchdog	Process supervision and automatic restart
Testing and evaluation	pytest, tcpreplay, iperf3, Wireshark/tcpdump, Linux attack tools (nmap, hydra, hping3, arpspoof, dnsspoof)	Unit, integration, performance and live attack tests
Development tools	Git and GitHub, VS Code, Jupyter Notebook	Version control, coding and data analysis
Client side	Any modern web browser (Chrome, Firefox, Edge, Safari)	Accessing the dashboard

4.8 Graphical User Interface

The NIDPS has one graphical user interface: the web dashboard used by the Network Administrator. It is served by the NIDPS host over HTTPS and is reachable only from the protected local management network. Network clients have no interface because detection and mitigation operate transparently inline. The interface provides a unified, responsive dashboard layout with a persistent left-hand navigation sidebar, top status header (including system theme toggle and live connectivity indicators), and dedicated functional panels. Figure 4.2 shows the proposed navigation flow between the pages.

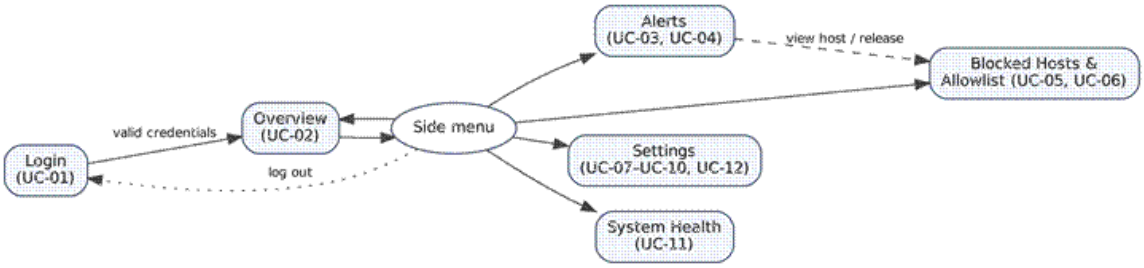


Figure 4.2: Navigation flow of the Network Administrator.

4.8.1 Login Screen

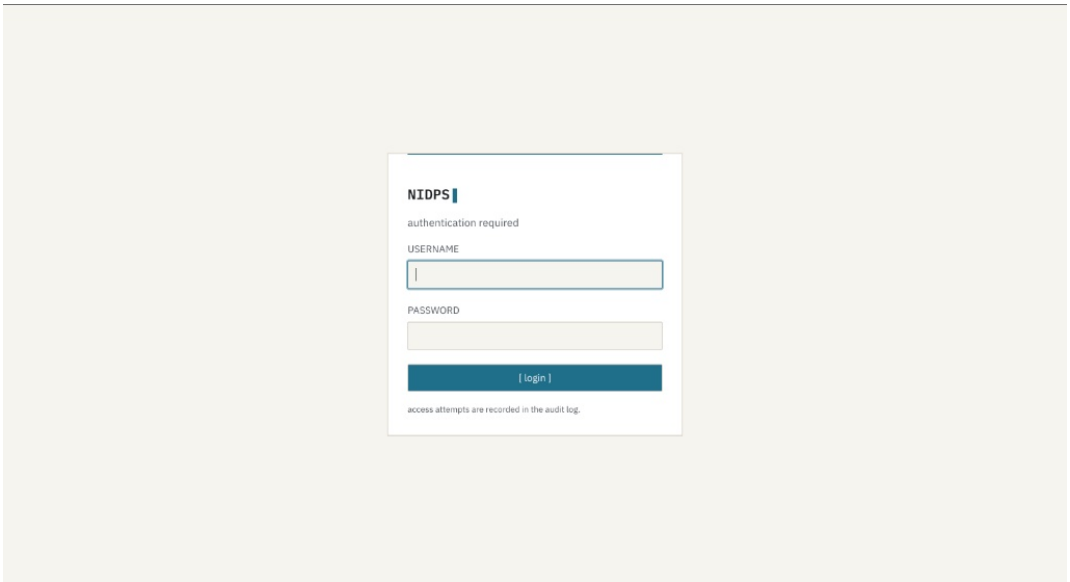


Figure 4.3: Login screen.

Access to the dashboard is strictly gated by administrator authentication (UC-01). The login interface displays username and password input fields and notifies the operator that access attempts are audited. Generic error messages are returned upon unsuccessful attempts to prevent username enumeration, and administrative accounts are temporarily locked after multiple consecutive failures (NFR-12). Successful authentication establishes a secure session cookie, directing the administrator to the Overview page.

4.8.2 Overview Screen

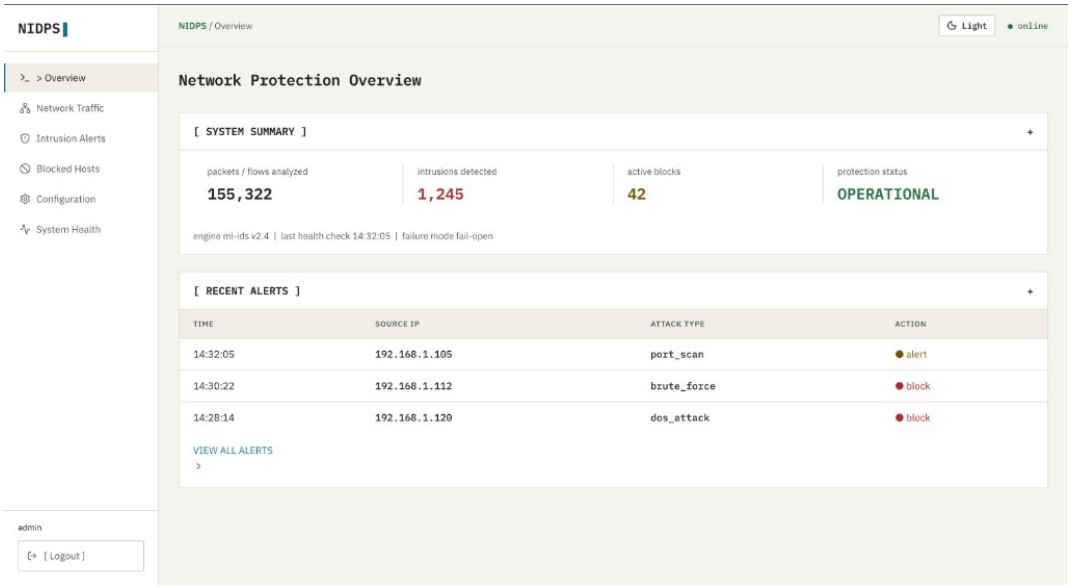


Figure 4.4: Overview screen.

The Overview screen serves as the primary operational dashboard for the network administrator (Figure 4.4). The **System Summary** panel aggregates high-level telemetry, displaying total packets/flows analyzed, total intrusions detected, currently active host blocks, and overall protection state (*OPERATIONAL*), alongside runtime details such as the active ML-IDS engine version, last health check timestamp, and kernel failure mode (*fail-open*). The **Recent Alerts** panel displays a real-time table of recent detection events indicating timestamp, originating source IP, attack classification, and automated enforcement action (*alert* or *block*), with a direct shortcut to view all alerts. (UC-02)

4.8.3 Network Traffic Screen

The Network Traffic screen provides in-depth visibility into network bandwidth and active conversations (Figure 4.5). Live counters display current observed throughput (in KB/s), aggregate bidirectional flows analyzed over the past 24 hours, and the count of connected devices alongside active concurrent flows. An interactive **Traffic History** timeline charts network volume over rolling time windows. The **Connected Devices and Flows** table features real-time search filtering by host or IP, detailing source and destination addresses, transport protocols, cumulative transferred volume, assessed threat risk level, and last activity timestamps. (UC-02)

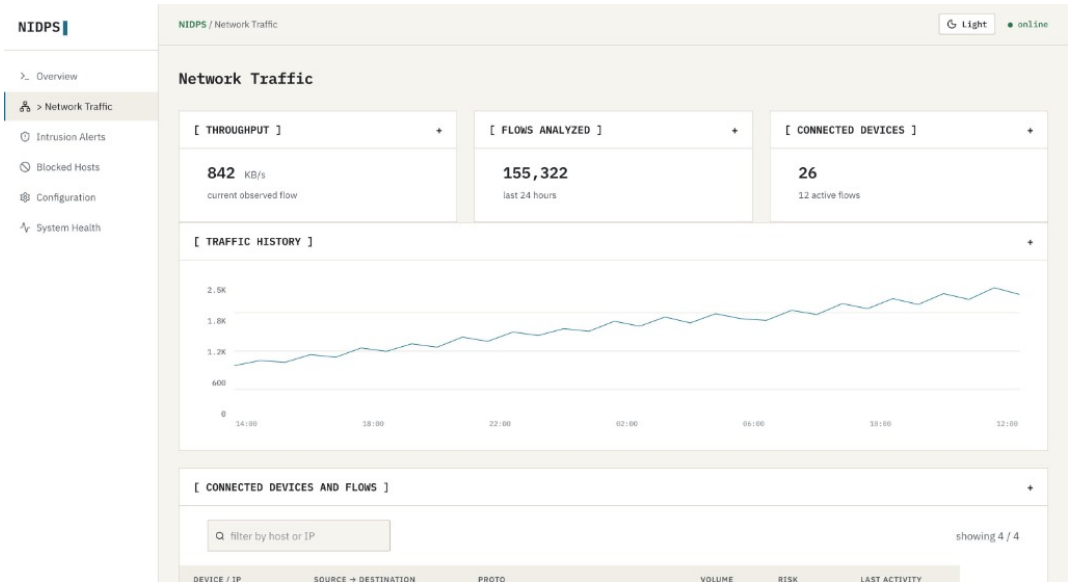


Figure 4.5: Network Traffic screen.

4.8.4 Intrusion Alerts Screen

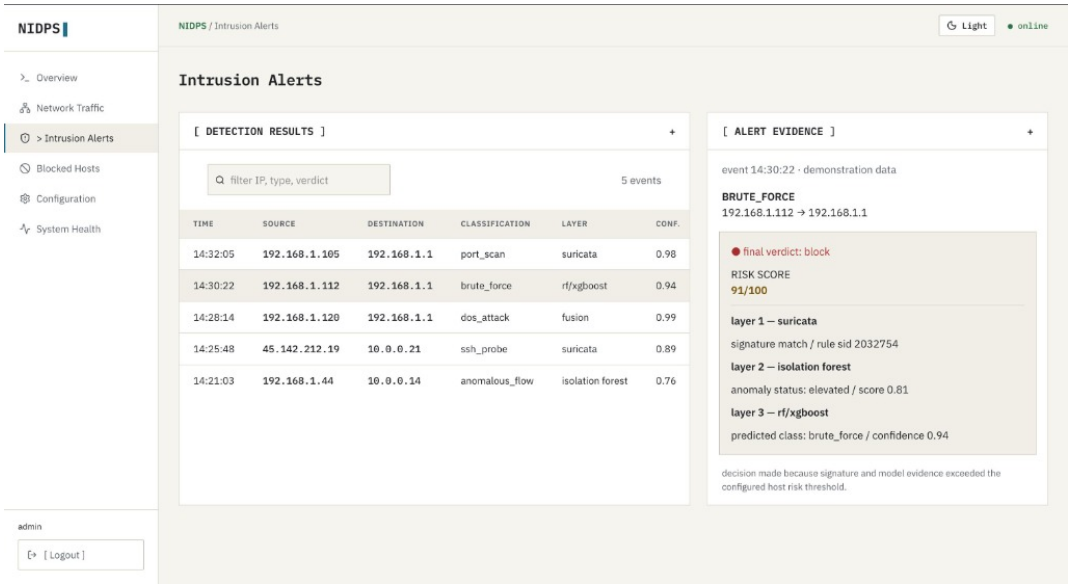


Figure 4.6: Intrusion Alerts screen with the Alert Evidence panel.

The Intrusion Alerts screen functions as the central alert triage console (Figure 4.6). The **Detection Results** table lists identified events with filter controls for IP address, attack category, and verdict. Each entry indicates the timestamp, source IP, destination IP, classification (such as port scan, brute force, DoS, SSH probe, or anomalous flow), detecting layer, and model confidence score. Selecting an alert opens the **Alert Evidence** drawer, which displays the calculated host risk score, final verdict, and an explainable breakdown across all three layers: Layer 1 Suricata signature rule SID, Layer 2 Isolation Forest anomaly score, and Layer 3 Random Forest/XGBoost classification confidence. (UC-03, UC-04)

4.8.5 Blocked Hosts and Allowlist Screen

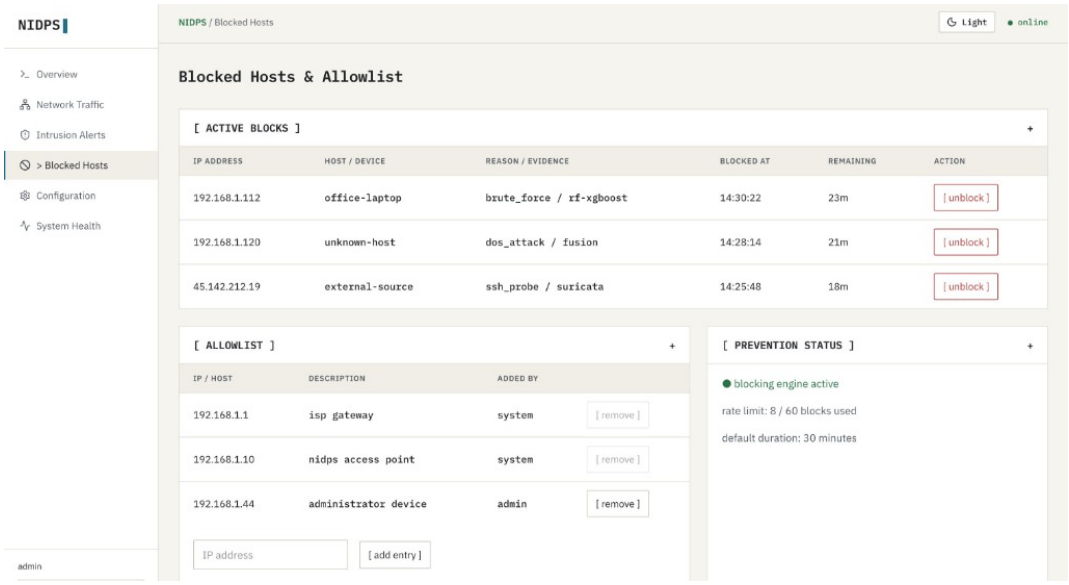


Figure 4.7: Blocked Hosts and Allowlist screen.

The Blocked Hosts and Allowlist screen provides complete administrative control over prevention and exemption policies (Figure 4.7). The **Active Blocks** table lists currently blocked IP addresses, identified host names, detection reason and originating layer, block initiation timestamp, remaining lockout duration, and a manual *[unblock]* action button to release hosts blocked by mistake. The **Allowlist** section manages entities exempt from automated blocking, including permanent infrastructure (ISP gateway, NIDPS access point) and administrator devices, with an input form to add new IP entries. The **Prevention Status** panel reports active blocking engine health, quota consumption against the configured block rate limit, and default quarantine duration. (UC-05, UC-06)

4.8.6 Detection and Prevention Configuration Screen

The Configuration screen allows the administrator to inspect and update all detection, mitigation, and model settings at runtime without requiring a service restart. Changes are validated immediately before persistence, and default values can be restored on demand. The configuration interface is divided into four dedicated tabs shown in Figures 4.8 through 4.11:

- **Detection Layers Tab (Figure 4.8):** Allows the administrator to independently enable or disable individual detection layers—Layer 1 (Suricata signature matching), Layer 2 (Isolation Forest flow deviation scoring), and Layer 3 (Random Forest / XGBoost attack classification). Administrative state is decoupled from runtime daemon health; disabled layers are excluded from decision fusion calculations. (UC-08)
- **Thresholds Tab (Figure 4.9):** Provides controls to tune detection sensitivity, including the Isolation Forest anomaly score cutoff (default 0.75), multi-class classifier confidence

threshold (default 0.80), host risk blocking threshold (default 70), and flow inactivity timeout (default 60 s). (UC-09)

- **Prevention and Failure Tab (Figure 4.10):** Configures active mitigation parameters, including initial block duration (30 min), repeat offender penalty escalation multiplier (2×), and maximum block rate cap (60/min). It also controls the kernel failure policy among *Fail-Open* (passing traffic if inspection daemons fail), *Fail-Closed* (blocking traffic upon component failure, with explicit confirmation prompt), and *Degraded* (operating with a reduced inspection profile), and displays watchdog operational status. (UC-07, UC-10)
- **Rules, Calibration and Models Tab (Figure 4.11):** Displays active Suricata ruleset metadata (Emerging Threats Open) with an on-demand ruleset reload trigger; Isolation Forest baseline calibration date and status with an instant recalibration button; and active machine learning model package versions and training provenance. (UC-12)

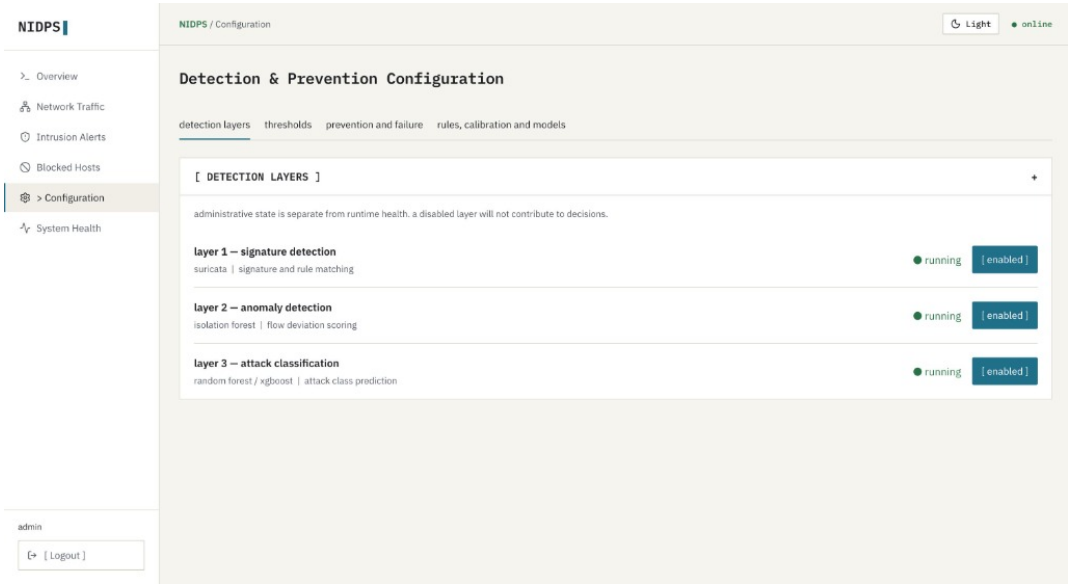


Figure 4.8: Configuration screen – Detection Layers tab.

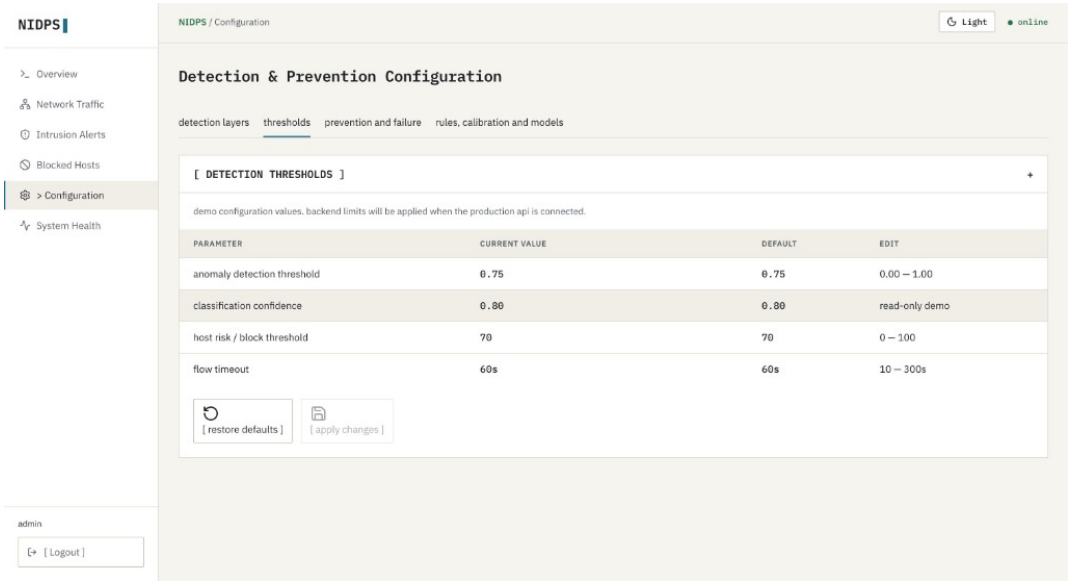


Figure 4.9: Configuration screen – Detection Thresholds tab.

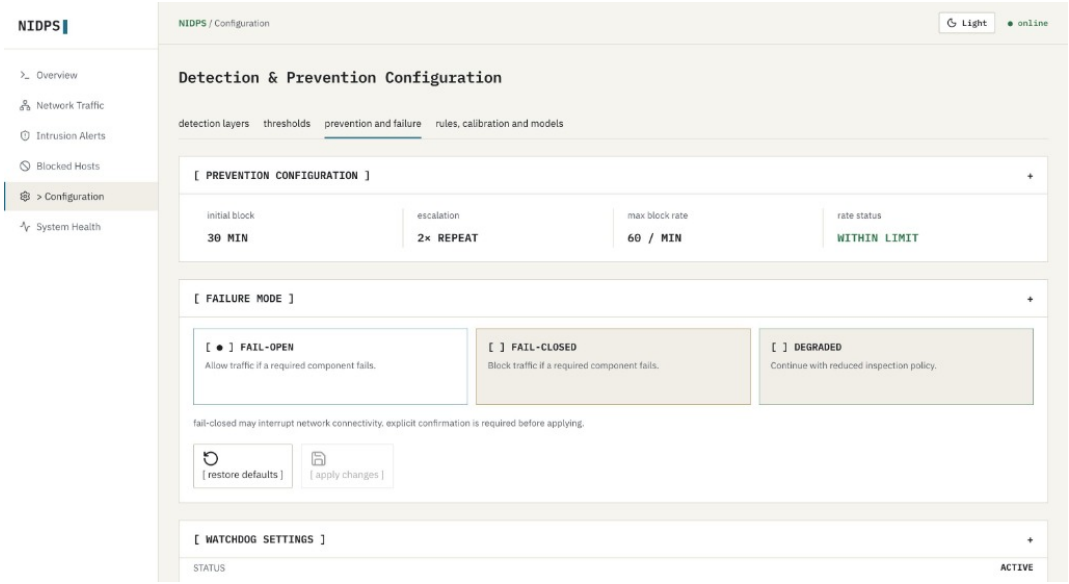


Figure 4.10: Configuration screen – Prevention and Failure tab.

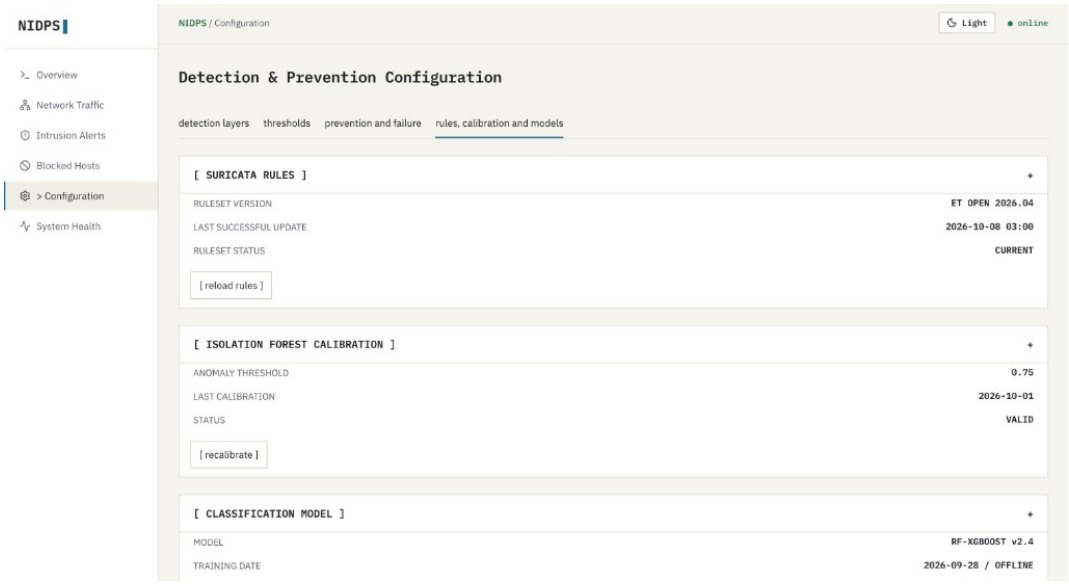


Figure 4.11: Configuration screen – Rules, Calibration and Models tab.

4.8.7 System Health Screen

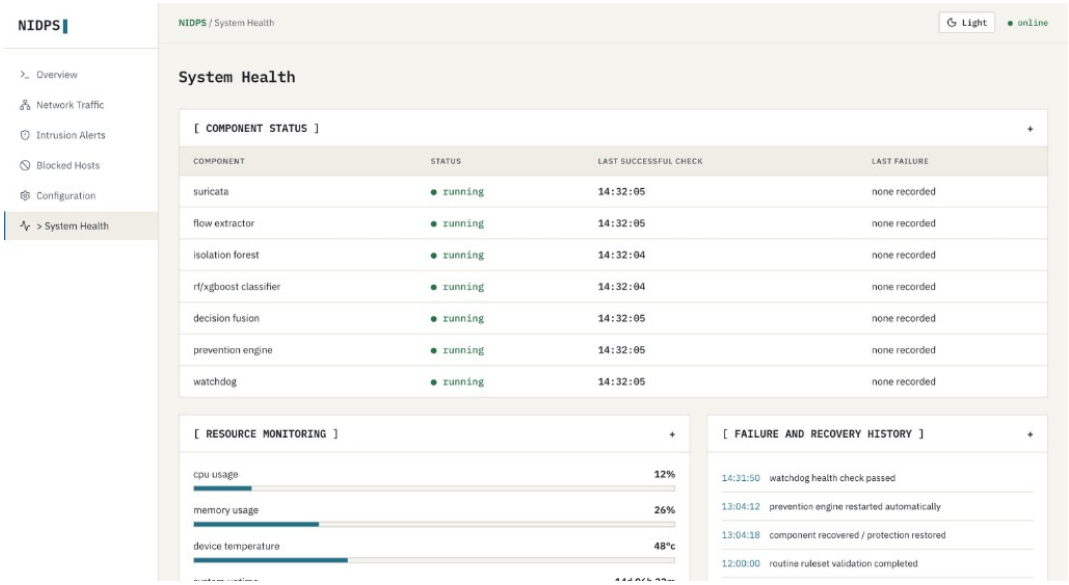


Figure 4.12: System Health screen.

The System Health screen displays real-time diagnostics and supervisory logs across all NIDPS subsystems (Figure 4.12). The **Component Status** panel tracks every background service (Suricata, flow extractor, Isolation Forest, RF/XGBoost classifier, decision fusion, prevention engine, and watchdog), confirming operational status (*running*), last successful heartbeat check timestamp, and failure counters. The **Resource Monitoring** panel displays live gauges for host CPU usage, memory utilization, device operating temperature, and system uptime. The **Failure and Recovery History** panel maintains an auditable chronological record of watchdog validations, automated process restarts, and recovery events. (UC-11)

4.9 Database Design

The NIDPS does not store any direct data such as packets or extra information. It will only store data points such as hosts, flow records, black/white lists, model packages, configuration state, verdicts and logs for a period. An RDBMS is used as the records are linked to each other, e.g. blocks happen in response to a verdict on a flow, and so on. The dashboard view may also need filtered entries. SQLite is chosen because it runs in-process with no server, stores everything in one file and is sufficient for the write rate of a single home device. Raw packets and payloads are never stored. Live counters shown on the dashboard are kept in memory and on the event bus rather than in the database.

4.9.1 ER Diagram

Figure 4.13 shows the entity-relationship diagram. HOST is the central entity: a host has many flow records, evidence items, verdicts and blocks. Each flow record can have several evidence items (one per layer) and at most one final verdict, and a BLOCK verdict creates at most one block entry. Evidence produced by an ML layer refers to the model package that produced it, and calibration states belong to the anomaly model. Administrative actions are linked to the admin user who performed them.

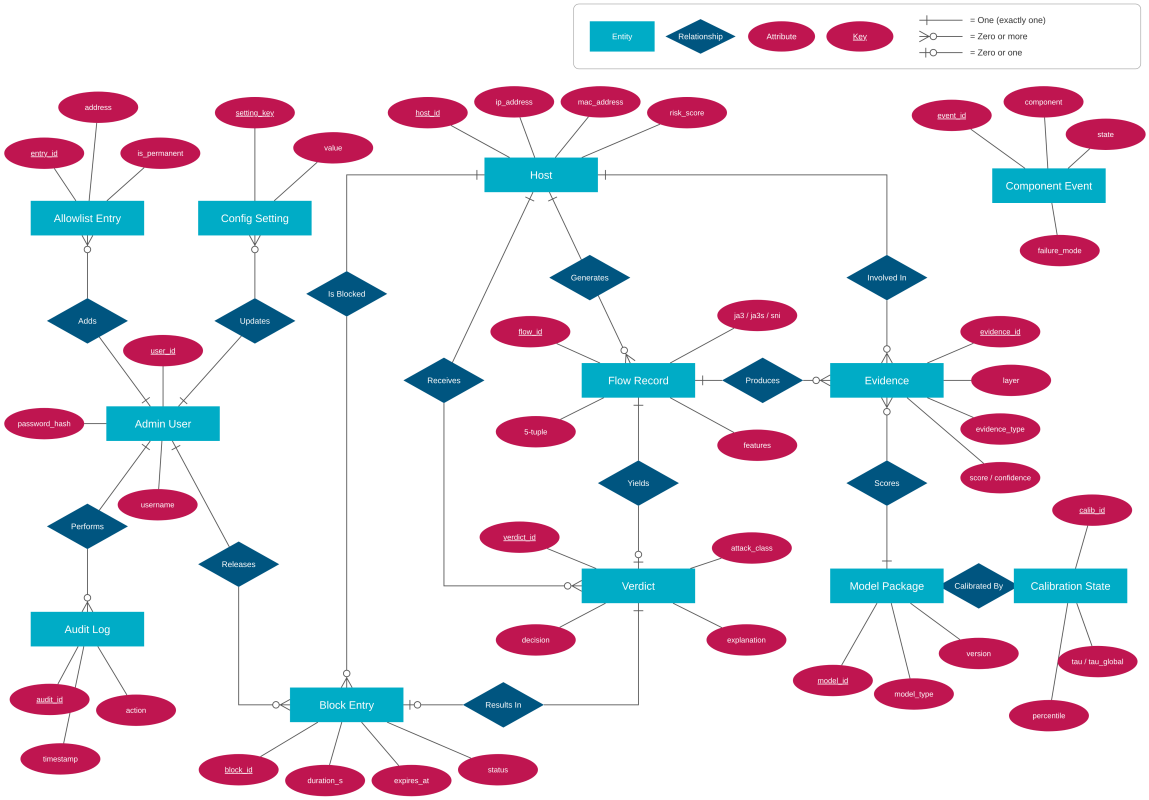


Figure 4.13: Entity-relationship diagram of the NIDPS database.

4.9.2 Data Dictionary

The tables below describe every entity. PK denotes a primary key and FK a foreign key. Timestamps are stored in UTC in ISO 8601 format.

Table 4.17: Data dictionary for ADMIN_USER.

Field	Type	Constraints	Description
user_id	INTEGER	PK	Unique identifier of the administrator
username	TEXT	UNIQUE, NOT NULL	Login name
password_hash	TEXT	NOT NULL	Salted password hash (e.g. Argon2/bcrypt)
failed_attempts	INTEGER	DEFAULT 0	Consecutive failed logins
locked_until	DATETIME	NULL	End of lock-out period, if locked
created_at / last_login	DATETIME		Account creation and last successful login

Table 4.18: Data dictionary for HOST.

Field	Type	Constraints	Description
host_id	INTEGER	PK	Unique identifier
ip_address	TEXT	NOT NULL	Current IPv4/IPv6 address
mac_address	TEXT	NULL	MAC address (internal hosts only)
hostname	TEXT	NULL	Name from DHCP lease or reverse DNS
is_internal	BOOLEAN	NOT NULL	True if the host is a client of the NIDPS
risk_score	REAL	DEFAULT 0	Current decaying evidence score S_h
first_seen / last_seen	DATETIME		First and most recent activity

Table 4.19: Data dictionary for FLOW_RECORD.

Field	Type	Constraints	Description
flow_id	INTEGER	PK	Unique identifier

Field	Type	Constraints	Description
host_id	INTEGER	FK → HOST	Internal host involved in the flow
src_ip, dst_ip, src_port, dst_port, protocol	TEXT / INTEGER	NOT NULL	5-tuple (stored for investigation, not used as model features)
start_time, end_time	DATETIME	NOT NULL	Flow or segment time span
segment_no	INTEGER	DEFAULT 0	Segment number for long flows
ja3, ja3s, sni	TEXT	NULL	TLS fingerprints and server name, if TLS
features	TEXT (JSON)	NOT NULL	78 flow features
extractor_version	TEXT	NOT NULL	Version of the flow extractor

Table 4.20: Data dictionary for EVIDENCE.

Field	Type	Constraints	Description
evidence_id	INTEGER	PK	Unique identifier
flow_id	INTEGER	FK → FLOW_RECORD, NULL	Related flow (NULL for ARP or host-level evidence)
host_id	INTEGER	FK → HOST	Host the evidence is about
layer	TEXT	CHECK IN (L1, ARP, L2, L3, HOST)	Detecting layer or host tracker
evidence_type	TEXT	NOT NULL	e.g. SIGNATURE, ANOMALY, CLASS, EXFIL, BEACON
signature_id	INTEGER	NULL	Suricata rule SID (Layer 1)
score	REAL	NULL	Anomaly or behaviour score
predicted_class, confidence	TEXT, REAL	NULL	Layer 3 class and probability
weight	REAL	NOT NULL	Weight w_v added to host score
model_id	INTEGER	FK → MODEL_PACKAGE, NULL	Model that produced ML evidence

Field	Type	Constraints	Description
created_at	DATETIME	NOT NULL	Time the evidence was produced

Table 4.21: Data dictionary for VERDICT.

Field	Type	Constraints	Description
verdict_id	INTEGER	PK	Unique identifier
flow_id	INTEGER	FK → FLOW_RECORD, NULL	Decided flow (NULL for host-score blocks)
host_id	INTEGER	FK → HOST	Host the verdict applies to
decision	TEXT	CHECK IN (PASS, ALERT, BLOCK)	Verdict
layers	TEXT	NOT NULL	Responsible layer(s), e.g. "L2+L3"
attack_class	TEXT	NULL	Unified attack class
explanation	TEXT	NOT NULL	Plain-language reason shown on the dashboard
reviewed	BOOLEAN	DEFAULT FALSE	Marked as reviewed by the administrator
created_at	DATETIME	NOT NULL	Time of the verdict

Table 4.22: Data dictionary for BLOCK_ENTRY.

Field	Type	Constraints	Description
block_id	INTEGER	PK	Unique identifier
host_id	INTEGER	FK → HOST	Blocked host
verdict_id	INTEGER	FK → VERDICT	Verdict that caused the block
offence_count	INTEGER	NOT NULL	Number of earlier blocks in 24 h (n)
duration_s	INTEGER	NOT NULL	Block duration D0 · 2^n, capped at D_max
start_time, expires_at	DATETIME	NOT NULL	Start and expiry of the block

Field	Type	Constraints	Description
status	TEXT	CHECK IN (ACTIVE, EXPIRED, RELEASED, SUPPRESSED)	Current state
released_by, released_at	INTEGER, DATETIME	FK → ADMIN_USER, NULL	Manual release details

Table 4.23: Data dictionary for ALLOWLIST_ENTRY.

Field	Type	Constraints	Description
entry_id	INTEGER	PK	Unique identifier
address	TEXT	UNIQUE, NOT NULL	IP or MAC address
description	TEXT		Reason, e.g. "Gateway"
is_permanent	BOOLEAN	DEFAULT FALSE	System entries that cannot be removed
added_by, added_at	INTEGER, DATETIME	FK → ADMIN_USER, NULL	Who added it and when

Table 4.24: Data dictionary for MODEL_PACKAGE.

Field	Type	Constraints	Description
model_id	INTEGER	PK	Unique identifier
model_type	TEXT	CHECK IN (IFOREST, RF, XGB)	Algorithm
version	TEXT	NOT NULL	Semantic version of the package
file_path	TEXT	NOT NULL	Location in the model registry
feature_list_hash, extractor_version	TEXT	NOT NULL	Schema compatibility information
class_list	TEXT (JSON)	NULL	Output classes (classifier only)
training_data, metrics	TEXT (JSON)		Datasets used and validation metrics

Field	Type	Constraints	Description
is_active	BOOLEAN	NOT NULL	True for the model currently in use
deployed_at	DATETIME	NOT NULL	Deployment time

Table 4.25: Data dictionary for CALIBRATION_STATE.

Field	Type	Constraints	Description
calib_id	INTEGER	PK	Unique identifier
model_id	INTEGER	FK → MODEL_PACKAGE	Anomaly model calibrated
tau, tau_global	REAL	NOT NULL	Calibrated and global thresholds
percentile	REAL	NOT NULL	Percentile p used
sample_count	INTEGER	NOT NULL	Number of clean flows used
u_max	INTEGER	NULL	Upload-size limit for exfiltration evidence
window_start, window_end, created_at	DATETIME	NOT NULL	Calibration window and run time

Table 4.26: Data dictionary for CONFIG_SETTING.

Field	Type	Constraints	Description
setting_key	TEXT	PK	e.g. failure_mode, active_timeout_s
value	TEXT	NOT NULL	Current value
default_value, min_value, max_value	TEXT		Default and allowed range
updated_by, updated_at	INTEGER, DATETIME	FK → ADMIN_USER, NULL	Last change

Table 4.27: Data dictionary for COMPONENT_EVENT.

Field	Type	Constraints	Description
event_id	INTEGER	PK	Unique identifier
component	TEXT	NOT NULL	Service name

Field	Type	Constraints	Description
state	TEXT	CHECK IN (RUNNING, DEGRADED, FAILED, RECOVERING)	State entered
failure_mode	TEXT	NULL	Mode applied during the failure
detected_at, recovered_at	DATETIME		Detection and recovery time
packets_dropped, packets_bypassed	INTEGER	DEFAULT 0	Traffic affected
details	TEXT		Cause and diagnostic information

Table 4.28: Data dictionary for AUDIT_LOG.

Field	Type	Constraints	Description
audit_id	INTEGER	PK	Unique identifier
user_id	INTEGER	FK → ADMIN_USER, NULL	Administrator (NULL for system actions)
action	TEXT	NOT NULL	e.g. LOGIN, BLOCK, RELEASE, CONFIG_CHANGE
target	TEXT		Affected object
old_value, new_value	TEXT	NULL	Values before and after a change
timestamp	DATETIME	NOT NULL	Time of the action

4.10 Risk Analysis

The possible risks that can be faced in the project are listed below, along with their likelihood, impact, category and mitigation. Technical risks decide whether the system will meet the proposed requirements. The other risks, which are resource, schedule and project risks, concern the team being able to deliver the project. Legal and ethical risks cover how testing and data privacy are maintained. Risks are reviewed and updated as the project carries on.

Table 4.29: Risk analysis.

ID	Risk	Category	Likelihood	Impact	Mitigation
R1	Lower-end PCs or Raspberry Pis may not be able to handle the throughput of a household with all layers enabled	Technical	Medium	High	Only Suricata runs inline; ML checks finished flows; disable unneeded rules; use degraded mode under load; move to a stronger device if needed.
R2	Models trained on various public datasets may perform inaccurately on household traffic	Technical	High	High	Extract features uniformly across datasets; tune thresholds for new networks; keep Suricata as a baseline; collect traffic on home networks and test honestly.
R3	High false positive rate causes legitimate devices to be blocked	Technical / Business	Medium	High	Fusion requires strong or corroborated evidence; allowlist; rate limit; short first blocks; one-week alert-only soak test before enabling blocking.
R4	Wireless adapters or devices do not support AP mode or client isolation	Technical	Medium	Medium	Test hardware functionality first; use a supported device; use Linux bridging if needed.
R5	Traffic encryption and JA3 randomisation reduce the ability to detect attacks properly	Technical	High	Medium	Rely on flow metadata; combine JA3, JA3S and SNI; keep the fingerprinting algorithm modular so it can be upgraded to JA4 later.
R6	Errors in publicly available datasets or the flow algorithm will disrupt training	Technical	Medium	Medium	Rebuild flows from PCAPs; relabel them using attack schedules; prefer corrected or improved dataset versions.
R7	PCAPs from large datasets can exceed memory limits for training	Resource	Medium	Medium	Process captures in chunks; sample benign traffic; use external storage or university lab machines.
R8	If the NIDPS fails, the whole network could go down	Operational	Low	High	Kernel-enforced fail-open by default; watchdog to catch crashes; automatic restart attempts; timed blocks.
R9	An attack could be mistaken for normal traffic during calibration, letting an attacker evade the models	Security	Low	High	Flagged flows are excluded from calibration; threshold shifts are capped; retraining only happens offline; signatures remain active.

ID	Risk	Category	Likelihood	Impact	Mitigation
R10	Attack experiments affect devices or networks outside the project	Legal / Ethical	Low	High	Run attacks only on an isolated lab network owned by the team; capture only the team's own traffic.
R11	Schedule slippage due to integration complexity of many services	Schedule	Medium	Medium	Delivery will happen methodically along the pipeline, with lower layers such as the AP and Layer 1 delivered first.
R12	Damage to hardware such as corruption or overheating	Resource	Medium	Medium	Use a trustworthy storage device and monitor temperatures; employ backups as well.
R13	Emergency unavailability of a team member	Project	Low	Medium	Shared repository and documentation; pair work on critical components; clear task ownership.

The primary risks to be aware of in this project are R2 and R3, as they will decide if the system will be usable and worthwhile in a real network environment. These are countered by design decisions such as local calibration and cautious decision fusion. They are also measured directly while evaluating the system so they can be addressed properly.

4.11 Conclusion

This chapter set out what the NIDPS has to do and how well. The 13 features break down into 63 functional requirements, and 20 non-functional requirements put numbers on qualities such as performance, reliability, accuracy, security and privacy. Twelve use cases show how the administrator, the scheduler and an attacker interact with the system. The hardware and software lists, the seven dashboard screens and the twelve-table database give enough detail to start building. Two risks matter most: R2, models trained on public datasets performing badly on household traffic, and R3, false positives blocking legitimate devices. Every requirement has an identifier, and Table 6.4 in Chapter 6 links each one to the part of the design that meets it.

Chapter 5

Proposed Approach and Methodology

This chapter presents the methodology through which the NIDPS detects and blocks malicious traffic on a network. The proposed approach is a hybrid detection framework which combines three detection layers, whose output goes through a decision fusion algorithm, that provides a verdict and then an enforcement and self recovery mechanism that acts on that decision. This chapter covers the datasets and how they are fused, automated prevention and failure handling and the evaluation protocol.

5.1 Overview of the Proposed Framework

The NIDPS runs on a single Linux host that acts as a routers wireless access point. Therefore, all packets between the client devices and the ISP gateway pass through it. Each packet travels two independent paths, the inline path in which Suricata inspects packets as they are being forwarded and can drop them immediately, and an analysis path, in which packets are grouped into bidirectional flows, identified by 78 distinct features and their TLS fingerprints. These flow records are scored by an unsupervised Isolation Forest (Layer 2) and a supervised Random Forest/XGBoost classifier (Layer 3).
The three layers cover different threat classes:

- **Layer 1 – Signature detection (Suricata):** known attacks with stable signatures, such as SSH brute force, port scanning and DNS spoofing.
- **Layer 2 – Anomaly detection (Isolation Forest):** traffic that deviates from the network's normal profile, including zero-day behaviour, C2 beaconing and data exfiltration.
- **Layer 3 – Supervised classification (Random Forest/XGBoost):** assignment of suspicious flows to known attack families such as DDoS and lateral movement.

A decision-fusion engine combines the three outputs into one verdict per flow (pass, alert or block). Block verdicts are enforced through iptables and ipset. A watchdog supervises every component and applies the configured failure mode when one of them stops. Figure 5.1 summarises this pipeline.
The design separates what must be fast from what can be thorough. Suricata decides per packet and adds only microseconds of forwarding delay. The machine learning layers decide per flow, so their verdicts are applied to the remainder of a long flow and to all later traffic from the offending host.

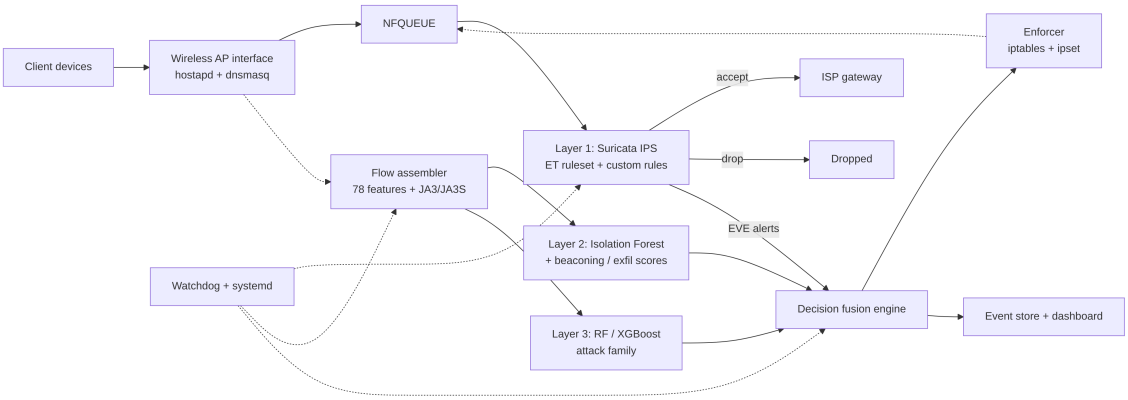


Figure 5.1: Proposed three-layer NIDPS pipeline. The inline packet path is represented by the solid line, whereas the dotted line represents the analysis and the supervision path.

5.2 Datasets

The machine learning layers are trained on three public intrusion detection benchmarks and evaluated additionally on traffic captured from our own deployment. The goal is not the highest score on one benchmark, but a model that still works when the training traffic and the deployment traffic differ.

5.2.1 Public Benchmark Datasets

Table 5.1 lists the public datasets used for training and cross-evaluation.

Table 5.1: Public benchmark datasets used for training and cross-evaluation.

Dataset	**Source**	**Traffic profile**	**Attack families relevant to this project**
CIC-IDS-2017 [16]	Canadian Institute for Cybersecurity (CIC)	5 days of enterprise-style traffic	Brute force (SSH/FTP), DoS/DDoS, port scan, botnet, infiltration
CSE-CIC-IDS2018 [17]	CIC with the Communications Security Establishment	Larger AWS-hosted network, 10 days	Brute force, DoS/DDoS, botnet, infiltration
CIC-IoT-2023 [18]	CIC	105 IoT devices in a smart-home topology	DDoS, recon/scan, spoofing, brute force, Mirai-style attacks

CIC-IoT-2023 is closest to the target environment because home networks are increasingly dominated by IoT devices. CIC-IDS-2017 and CSE-CIC-IDS2018 contribute a wider range

of host-based attacks. Published analyses of CIC-IDS-2017 have reported labelling and flow-construction errors in the original CSV files [19], which is one more reason flows are regenerated from the raw captures (Section 5.2.3).

5.2.2 Self-Collected Real-World Dataset

A fourth dataset is captured on the deployed access point itself. It has two parts:

1. **Benign capture.** Normal household or lab traffic (browsing, streaming, video calls, IoT devices, software updates) is recorded over several days. Flows that Suricata flags during this period are removed before the capture is labelled benign.
2. **Attack capture.** Each in-scope attack is executed from a dedicated attacker machine in an isolated test network that we own, using standard penetration-testing tools. The start and end time, attacker address and target of each run are logged, so every flow in the window can be labelled automatically from these logs.

This dataset is used only for testing, never for training. It measures how well models trained on public data transfer to a real home network.

5.2.3 Feature Harmonisation Across Datasets

The public datasets do not share one feature format. The two CIC-IDS sets ship CICFlowMeter CSVs with about 80 columns whose names and rounding differ between releases, while CIC-IoT-2023 ships a different, smaller feature set computed over packet windows. A model trained on one format cannot be applied to another, and none of them is guaranteed to match what the live system computes.
To remove this mismatch, the following procedure is applied:

1. Download the raw PCAP files of every dataset instead of the provided CSVs.
2. Run the same flow extractor that runs on the device (Section 5.3.2) over every PCAP. All datasets then share an identical 78-feature schema, with identical definitions, timeouts and units. The extractor runs with the same timeouts as the device ($T_a = 10\,\text{s}$, $T_i = 30\,\text{s}$), so a training record and a live record always cover the same span of traffic.
3. Re-label the regenerated flows by matching their 5-tuple and timestamps against each dataset's published attack schedule.
4. Map every dataset-specific label onto one unified attack taxonomy (Table 5.2).

The key advantage is that training-time and serving-time features are produced by the same code, so there is no train/serve skew.

Table 5.2: Unified attack taxonomy used for Layer 3.

Unified class	**Examples of original labels**	**Primary detecting layer**
Benign	BENIGN, BenignTraffic	–
Reconnaissance	PortScan, Recon-PortScan, Recon-OSScan	L1, L3
Brute force	SSH-Patator, FTP-Patator, DictionaryBruteForce	L1, L3
DoS / DDoS	DoS Hulk, DDoS LOIC, DDoS-SYN_Flood, DDoS-UDP_Flood	L1, L3
Botnet / C2	Bot, Mirai variants	L2, L3
Spoofing	DNS_Spoofing, MITM-ArpSpoofing	L1
Infiltration / lateral movement	Infiltration, internal scanning after compromise	L3

5.2.4 Pre-processing

The regenerated flow tables are cleaned before training:

- **CSE-CIC-IDS2018 day selection:** only the Friday-02-03-2018 capture (Bot: Zeus and Ares) is regenerated from this dataset's ten captured days. The remaining nine would add several hundred gigabytes of raw capture for little extra value: two days (Thursday-22-02, Friday-23-02) cover Web/SQL-injection attacks that fall outside the unified taxonomy of Table 5.2, and the rest (Brute force, DoS/DDoS, Infiltration) are already represented in CIC-IDS-2017 and CIC-IoT-2023. The Bot day is kept despite this overlap because it is the only capture, across all three public datasets, with genuine bot-to-C2 check-in traffic rather than a flood proxy for "botnet": CIC-IoT-2023's Mirai sub-attacks (`Mirai-greeth_flood`, `Mirai-greip_flood`, `Mirai-udpplain`) are GRE/UDP floods, not beaconing traffic, so they do not exercise the C2-beaconing detector of Section 5.5.4.
- **Invalid values:** rows with infinite or undefined values (for example, rates computed over a zero-length flow) are dropped, and exact duplicate rows are removed.
- **Identifier removal:** source and destination IP addresses, ports, flow IDs and timestamps are removed from the feature vector. They identify the lab, not the attack, and keeping them lets a model memorise the attacker's address instead of learning behaviour.
- **Constant features:** features with zero variance in the training set are removed.
- **Splitting:** within one dataset, the split into training and test sets is done by capture

day rather than at random, because neighbouring flows of the same attack run are nearly identical and a random split leaks them into both sets.

- **Class imbalance:** class proportions are heavily skewed, in opposite directions: benign flows dominate CIC-IDS-2017, while attack flows dominate CIC-IoT-2023. Layer 3 uses class weights inversely proportional to class frequency; oversampling (SMOTE), if used, is applied only to the training fold.
- **Scaling:** tree-based models do not need feature scaling, so values are kept in their original units. This also keeps the online pipeline simple.
- **Isolation Forest training set:** Layer 2 is trained only on benign flows, since it models normal behaviour.

5.3 Traffic Acquisition and Flow Construction

All detection depends on how packets reach the detectors and how they are summarised. This section describes the inline packet path, the flow assembly algorithm and the extraction of TLS fingerprints from encrypted traffic.

5.3.1 Inline Packet Path

The host runs hostapd to expose a wireless network and dnsmasq to provide DHCP and DNS to clients. IP forwarding is enabled and the uplink interface performs NAT towards the ISP gateway. Forwarded traffic is diverted by an iptables rule in the FORWARD chain to an NFQUEUE, where Suricata runs in IPS mode. For each packet, Suricata returns an accept or drop verdict to the kernel.

A copy of every forwarded packet is also read by the flow assembler through a packet capture socket on the wireless interface. The flow assembler never holds packets back, so a slow ML layer cannot slow down forwarding.

One platform detail matters for lateral movement. By default, the wireless driver and hostapd relay frames between two wireless clients directly, without passing them up to the IP layer, so netfilter and Suricata never see them. To inspect client-to-client traffic, client isolation (`ap_isolate=1`) is enabled and intra-network traffic is routed through the host. Without this setting, lateral movement can only be detected when it crosses the AP's IP layer.

5.3.2 Flow Assembly Algorithm

A flow is the set of packets sharing the same 5-tuple (source IP, destination IP, source port, destination port, protocol) in either direction. The direction of the first packet defines "forward"; replies are "backward". Algorithm 5.1 shows how flows are built and closed.

Algorithm 5.1: Bidirectional flow assembly

Input: packet stream P; active timeout T_a (default 10 s); idle timeout T_i (default 30 s)

Output: completed flow records with 78 features

$FlowTable \leftarrow$ empty hash map keyed by canonical 5-tuple;
foreach *packet* $p \in P$ **do**
  $k \leftarrow$ canonical(p.5tuple) ; `// same key for both directions`
  if $k \notin FlowTable$ **then**
    create flow f with $f.start \leftarrow p.time$ and $f.fwd \leftarrow$ direction of p;
    $FlowTable[k] \leftarrow f$;
  $f \leftarrow FlowTable[k]$;
  update the running statistics of f with p (direction, size, flags, time);
  if *p carries a TLS ClientHello or ServerHello* **then**
    attach JA3/JA3S to f (Algorithm 5.2);
  if *p has FIN in both directions or RST* **then**
    emit(f);
    remove k;
  else if $p.time - f.start \geq T_a$ **then**
    emit(f) as a partial record;
    reset the counters of f but keep k;
`// run periodically`
foreach *flow* f *with* $now - f.last_seen \geq T_i$ **do**
  emit(f);
  remove its key;

The active timeout lets a long-running connection, such as a slow exfiltration channel, be scored in segments while it is still open, instead of only when it closes. Each segment is emitted as its own flow record, after which its counters reset. The idle timeout reclaims memory from flows that stop without a FIN or RST. Table 5.3 gives the default values.

Table 5.3: Flow timeout parameters.

Parameter	Default	Effect
Active timeout T_a	10 s (configurable, e.g. 5–30 s)	An open flow is re-scored every T_a, which caps how long one connection can run unscored
Idle timeout T_i	30 s	A flow with no packets for T_i is closed and scored

Most flows, such as web requests, DNS lookups and scan probes, end in well under 5 seconds and are scored once, when they close. Only long-lived connections are segmented. T_a trades CPU for reaction time: halving it doubles the scoring work for long flows. The

default of 10 s is a conservative starting point for low-cost hardware. T_a is a configuration parameter, so a deployment with spare CPU can lower it, for example to 5 s, to react faster. The evaluation measures both settings (Section 5.10.3).
The 78 features follow the CICFlowMeter definitions: packet and byte counts per direction, packet length statistics, inter-arrival time statistics, TCP flag counts, header lengths, bulk and sub-flow measures, and active/idle periods. To keep memory constant per flow, means and variances are updated incrementally with Welford's method (Equation 5.1) rather than by storing every packet:

$$\begin{aligned} \mu_n &= \mu_{n-1} + \frac{x_n - \mu_{n-1}}{n}, \\ M_n &= M_{n-1} + (x_n - \mu_{n-1})(x_n - \mu_n), \\ \sigma_n^2 &= \frac{M_n}{n-1}. \end{aligned} \tag{5.1}$$

Here x_n is the n-th observation (for example, a packet length), μ_n the running mean and M_n the running sum of squared deviations. Minimum and maximum are tracked directly.

5.3.3 TLS Fingerprint Extraction (JA3/JA3S)

Most traffic is encrypted, so payload cannot be inspected without decryption. Decrypting traffic on the AP would require installing a certificate on every client, would break certificate pinning in many apps and would cost far more CPU than the target hardware can spare. Instead, the system uses fields of the TLS handshake that are sent in clear text [20], as shown in Algorithm 5.2.

Algorithm 5.2: JA3/JA3S fingerprint

Input: TLS ClientHello (or ServerHello)
Output: 32-character MD5 fingerprint
read the TLS version, cipher suites, extension types, supported groups (elliptic curves) and EC point formats;
remove GREASE values (0x0a0a, 0x1a1a, . . . , 0xfafa) from every list;
join the decimal values of each list with "-";
$s_{JA3} \leftarrow$ Version,Ciphers,Extensions,Groups,PointFormats;
$s_{JA3S} \leftarrow$ Version,Cipher,Extensions ; `// ServerHello`
return *MD5(s)*;

The fingerprint identifies the client software that opened the connection, not the user. Malware and C2 frameworks often use TLS libraries with fingerprints that differ from common browsers and operating systems. The pair (JA3, JA3S) is attached to the flow record and used by Layer 2 in two ways: as a rarity signal (how many local hosts have ever used this fingerprint) and as part of the key for beacon detection (Section 5.5.4). Suricata's TLS parser can emit the same hashes in its EVE logs, which lets Layer 1 rules match known-bad fingerprints directly.

5.4 Layer 1: Signature-Based Detection

Layer 1 catches attacks that have a known, stable pattern. It is the only layer that acts on individual packets, so it is the first line of prevention.

5.4.1 Suricata Configuration

Suricata [21] runs in IPS mode on the NFQUEUE with the Emerging Threats Open ruleset. Rule categories that do not apply to a home network (for example, SCADA protocols) are disabled to save CPU. Each rule carries an action (alert or drop) and a severity from 1 (highest) to 3. Rules with severity 1 that target in-scope attacks are set to drop; the rest only alert and are passed on to decision fusion. Every alert is written to Suricata's EVE JSON output, which the alert ingestor reads over a Unix socket.

5.4.2 Custom Rules for In-Scope Attacks

The ET ruleset matches known payloads well, but several in-scope attacks are defined by rate rather than content. These are covered by custom threshold rules, which count matching events per source address within a time window (Table 5.4).

Table 5.4: Custom Layer 1 rules for the in-scope attacks.

Attack	**Detection logic**	**Example threshold (tunable)**
SSH brute force	New TCP connections (SYN) to port 22 from one source	≥ 5 in 60 s
Port scan	SYN packets from one source to many distinct destination ports	≥ 20 ports in 10 s
DNS spoofing	DNS responses to clients whose source is not the AP's own resolver	any occurrence
Known C2 TLS client	JA3 hash on a blocklist of published malicious fingerprints	any occurrence
SYN flood (DoS)	SYN packets to one destination without completed handshakes	rate above a calibrated limit

The thresholds are starting values that are tuned during evaluation. The DNS spoofing rule relies on the AP design. Clients receive the AP's address as their only DNS server through DHCP, so a legitimate DNS answer always comes from the AP. Any DNS response arriving at a client from another address is forged or comes from a rogue resolver.

5.4.3 ARP Spoofing Monitor

Suricata inspects IP traffic and does not match rules against ARP frames, which sit below the IP layer. ARP spoofing is therefore detected by a small companion sensor that belongs to Layer 1 and reports into the same alert stream. It uses the DHCP lease table of dnsmasq as the ground truth for which MAC address owns which IP address (Algorithm 5.3).

Algorithm 5.3: ARP consistency check

Input: ARP frames on the wireless interface; DHCP lease table L (IP $\rightarrow$ MAC)
Output: ARP spoofing alerts

```
1 Bindings ← L ;                                   // trusted IP-to-MAC map
2 foreach ARP reply or gratuitous ARP a do
3     if a.sender_ip ∈ Bindings and Bindings[a.sender_ip] ≠ a.sender_mac then
4         raise alert ARP_SPOOF (attacker a.sender_mac, victim a.sender_ip);
5     if a.sender_ip = gateway_ip and a.sender_mac ≠ AP_mac then
6         raise alert GATEWAY_IMPERSONATION with severity 1;
7     if more than N unsolicited replies from one MAC within window W then
8         raise alert ARP_FLOOD;
9 on every DHCP lease change, refresh Bindings from L;
```

Impersonating the gateway is the classic step before a man-in-the-middle attack, so that check raises the highest severity.

5.5 Layer 2: Unsupervised Anomaly Detection

Layer 2 flags traffic that looks unlike the network's normal behaviour, without needing a label for the attack. It covers what signatures miss: new attacks, variants that change their signature, C2 beaconing and data exfiltration.

5.5.1 Isolation Forest

Isolation Forest [22] is based on a simple observation: anomalies are few and different, so they are easier to separate from the rest of the data than normal points are. The model builds an ensemble of random binary trees. Each tree is grown on a random subsample of ψ flows by repeatedly choosing a random feature and a random split value between that feature's minimum and maximum, until every point is isolated or a height limit of $\lceil \log_2 \psi \rceil$ is reached.

An anomalous flow is isolated after few splits, so its path length $h(x)$ from root to leaf is short. Averaging over all trees gives $E[h(x)]$, which is normalised by the average path length of an unsuccessful search in a binary search tree of ψ points:

$$c(\psi) = 2H(\psi - 1) - \frac{2(\psi - 1)}{\psi}, \qquad H(i) \approx \ln(i) + 0.5772 \tag{5.2}$$

$$s(x,\psi) = 2^{-\frac{E[h(x)]}{c(\psi)}} \tag{5.3}$$

The anomaly score s in Equation 5.3 lies between 0 and 1. Scores close to 1 indicate clear anomalies; scores well below 0.5 indicate normal points. The model uses $t = 100$ trees and $\psi = 256$, the values recommended by the original authors, and is trained only on benign flows from the harmonised datasets.

Isolation Forest suits the target hardware because training and scoring are cheap. Scoring a flow costs t tree walks of at most $\lceil \log_2 \psi \rceil = 8$ steps each, regardless of how much training data was used.

5.5.2 Adaptive Threshold Calibration

The model outputs a continuous score, so a threshold τ is needed to turn it into a decision. A threshold taken from benchmark traffic does not fit every home network: a household with game consoles and security cameras produces traffic that a benchmark lab never saw, which shows up as false positives. The calibration scheme in Algorithm 5.4 adjusts only the threshold to each deployment; it does not retrain the model.

Algorithm 5.4: Per-network threshold calibration

Input: trained Isolation Forest M; global threshold τ_g; calibration window W (e.g. 24–72 h); percentile p (e.g. 99.5); minimum sample count N_{min}; maximum shift δ

Output: calibration state $\{\tau, \text{timestamp}, n\}$

$S \leftarrow \emptyset$;
foreach *flow x completed during W* **do**
  if *x matched any Suricata alert* ***or*** *came from a blocked host* **then**
    skip x ; `// keep the baseline clean`
  add M.score(x) to S;
if $|S| < N_{min}$ **then**
  $\tau \leftarrow \tau_g$; `// too little local data: fall back`
else
  $\tau_l \leftarrow p$-th percentile of S;
  $\tau \leftarrow \text{clip}(\tau_l,\ \tau_g - \delta,\ \tau_g + \delta)$;
store $\{\tau, now, |S|\}$ as calibration metadata;
repeat periodically (e.g. weekly) as the network's traffic changes;

Two safeguards protect the calibration. The exclusion step leaves out flows that Layer 1 already considers malicious. The clipping step limits how far the local threshold may move from the global one, so an attacker who is active during calibration cannot raise the threshold far enough to hide.

The percentile p directly sets the expected false positive rate on the calibration traffic: with $p = 99.5$, about 0.5% of benign local flows would exceed τ. This makes the trade-off between missed attacks and alert volume an explicit, tunable parameter.

5.5.3 Data Exfiltration Score

Exfiltration is a property of a host over time, not of one flow. For each internal host h, the system keeps an exponentially weighted moving average and variance of its outbound bytes per time bucket b_t (60 s by default):

$$\bar{b}_t = \alpha\, b_t + (1-\alpha)\,\bar{b}_{t-1}, \qquad \sigma_t^2 = \alpha\,(b_t - \bar{b}_{t-1})^2 + (1-\alpha)\,\sigma_{t-1}^2 \tag{5.4}$$

A bucket is flagged when b_t exceeds the host's baseline by k standard deviations ($k = 4$ as a starting value) and the outbound-to-inbound byte ratio of the host's flows is also unusually high. The second condition prevents large uploads that are part of normal two-way activity, such as video calls, from triggering the alert.

Bucket statistics react only once per bucket, so a second, faster check runs on every active-timeout segment. If one segment carries more than U_{max} bytes outbound to a destination the host has never contacted before, a high-weight evidence item is raised immediately. U_{max} is set during calibration (Algorithm 5.4) as a high percentile, for example the 99.9th, of segment upload sizes in local benign traffic. When this evidence leads to a block, leakage over a single connection is bounded by roughly the host's upload rate $\times$ (T_a + scoring delay). At 20 Mbps and the default $T_a = 10$ s, that is about 25 MB.

5.5.4 C2 Beaconing Score

Infected devices typically contact their command and control server at regular intervals. Flows are grouped by the key (internal host, destination IP or TLS server name, JA3). For each group with at least m connections in the observation window, the inter-arrival times $\Delta t_1, \ldots, \Delta t_k$ between consecutive connections are computed, along with their coefficient of variation:

$$CV = \frac{\sigma_{\Delta t}}{\mu_{\Delta t}} \tag{5.5}$$

Human-driven traffic is irregular (high CV), while beacons are close to periodic (low CV), even with added random jitter. The beacon score combines three signals: a low CV, a low variance in bytes per connection, and a rare JA3 fingerprint on the local network. A group is reported when all three point the same way, which keeps legitimate periodic traffic, such as software update checks from common browsers, from being flagged.

5.6 Layer 3: Supervised Attack Classification

Layer 3 assigns each flow to one of the unified classes of Table 5.2. Where Layer 2 answers "is this unusual?", Layer 3 answers "which known kind of attack does this look like?". The class label decides the response and makes alerts readable on the dashboard. Two tree ensembles are trained and compared, and the better one under the hardware budget is deployed.

5.6.1 Random Forest

Random Forest [23] trains many decision trees, each on a bootstrap sample of the training flows, and at each split considers only a random subset of $\sqrt{d}$ of the d features. Splits are chosen to minimise Gini impurity:

$$G(S) = 1 - \sum_{k=1}^{K} p_k^2 \tag{5.6}$$

where p_k is the share of class k among the flows S at a node. The forest predicts by averaging the class probabilities of all trees. Bootstrapping and feature subsampling make the trees disagree in useful ways, so the ensemble is less prone to overfitting than a single tree. This matters because the model is trained on lab traffic and deployed on a different network.

5.6.2 XGBoost

XGBoost [24] builds trees one after another, each new tree correcting the errors of the current ensemble. It minimises a regularised objective:

$$\mathscr{L} = \sum_i l(y_i, \hat{y}_i) + \sum_m \Omega(f_m), \qquad \Omega(f) = \gamma T + \tfrac{1}{2}\lambda \|w\|^2 \tag{5.7}$$

Here l is the multi-class log loss, T is the number of leaves of tree f and w its leaf weights. The γ and λ terms penalise large, complex trees. XGBoost is usually more accurate than Random Forest on tabular data, but it is more sensitive to hyperparameters and can fit dataset-specific artefacts more closely, which the cross-dataset evaluation (Section 5.10.2) is designed to expose.

5.6.3 Training and Model Selection

1. Train on the harmonised flows with identifiers removed (Section 5.2.4) and using class weights to counter the imbalance.
2. Tune hyperparameters (number of trees, maximum depth, learning rate for XGBoost, minimum samples per leaf) with stratified 5-fold cross-validation on the training days only.
3. Select by macro-averaged F1-score rather than accuracy.
4. Apply a hardware budget: among models within one F1 point of the best, choose the one with the lowest per-flow inference time and memory on the target device.
5. Export the chosen model with its feature list and version number, so the runtime refuses a model whose features do not match the extractor.

5.6.4 Confidence Handling

The classifier produces a probability vector p over K classes. The predicted class is $\hat{y} = \arg\max p$, with confidence $c = \max p$. If the confidence is low then the flow is unlikely to fit into any of the classes particularly if the flow is an attack that was not seen during the model training. For flows with c below a confidence threshold θ_c , instead a laebl "unknown" is assigned. The Layer 2 score is passed to the decision fusion module. This allows refinement of any detected anomalies whilst still allowing a high confidence anomaly to stand untouched.

5.7 Decision Fusion

The layers may differ in their opinions. An action from a single layer would either block too much (anomalies alone are prone to noise) or too little (signatures alone fail to detect novel attacks). Evidence from each layer is combined by host and by flow in the decision fusion module. The underlying motivation is that if one layer provides strong evidence for a block, then it will be blocked, but if the evidence is weak then either evidence must be provided by at least a second layer or the evidence must be accumulative.

5.7.1 Verdict Algorithm

Algorithm 5.5 gives the per-flow verdict, and Figure 5.2 shows the same logic as a flowchart.

Algorithm 5.5: Per-flow verdict

Input: flow f; Suricata alerts $A(f)$; anomaly score $s(f)$; threshold τ; class probabilities $p(f)$; thresholds $\theta_{block} > \theta_{alert} > \theta_c$

Output: verdict in {PASS, ALERT, BLOCK} and the attributed layer(s)

if $f.src \in$ *allowlist* **then return** *PASS*;
if $A(f)$ *contains a drop rule or severity 1* **then return** *BLOCK, layer L1*;
$y \leftarrow \arg\max p(f)$; $\quad c \leftarrow \max p(f)$;
if $y \neq$ *Benign* ***and*** $c \geq \theta_{block}$ **then return** *BLOCK, layer L3*;
if $s(f) \geq \tau$ ***and*** $y \neq$ *Benign* ***and*** $c \geq \theta_{alert}$ **then return** *BLOCK, layers L2+L3*;
if $s(f) \geq \tau$ ***or*** $A(f) \neq \emptyset$ ***or*** $(y \neq$ *Benign* ***and*** $c \geq \theta_{alert})$ **then**
    add evidence to the host score of $f.src$ (Section 5.7.2);
    return *ALERT*;
return *PASS*;

A confident known attack is blocked on Layer 3 alone, while a less confident classification is blocked only when Layer 2 agrees.

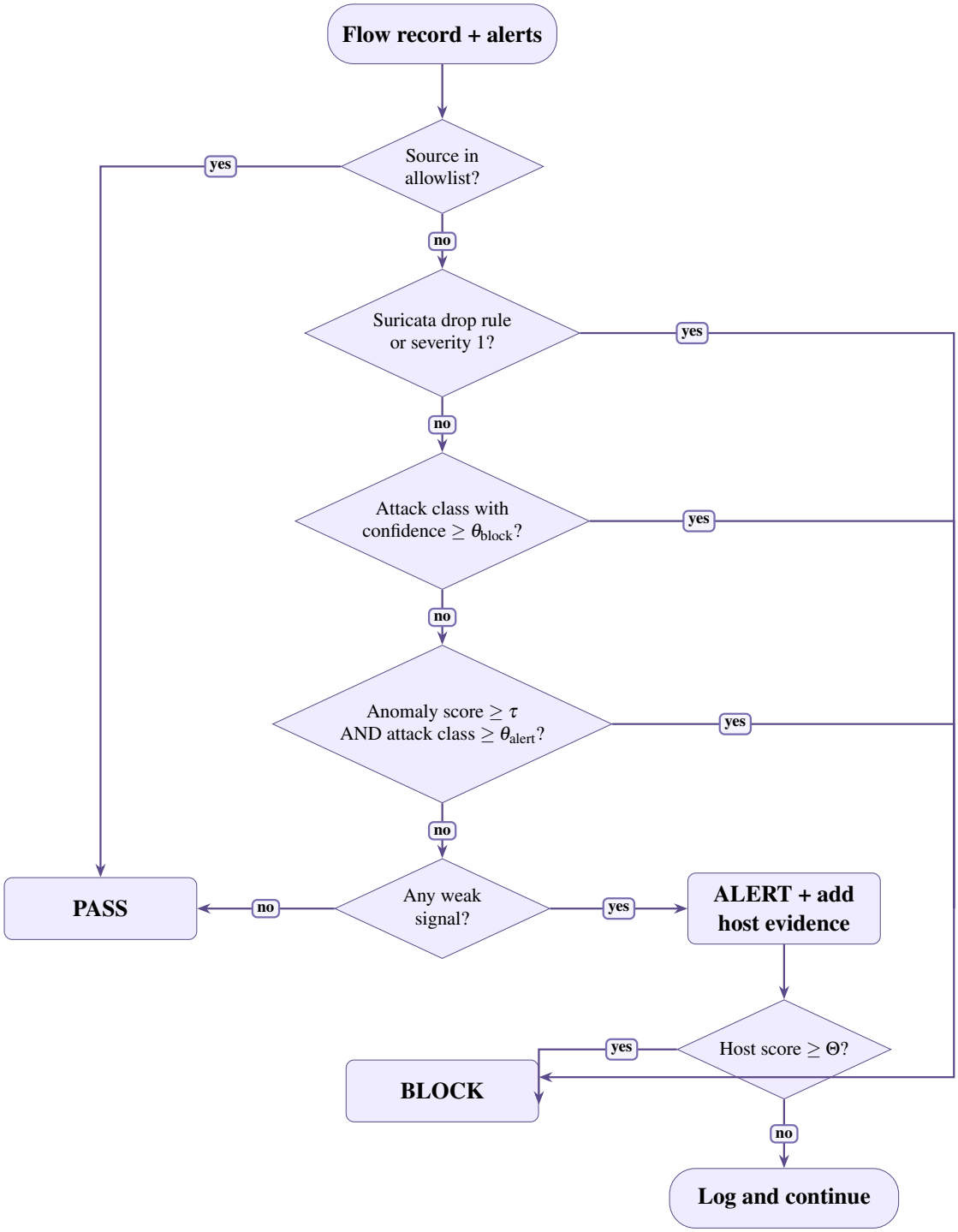


Figure 5.2: Decision-fusion flow for one flow record.

5.7.2 Host Evidence Accumulation

Some attacks, such as a slow port scan or low-volume exfiltration, never produce one strong signal. Their individual flows only trigger alerts. The host score S_h collects this weak evidence and lets it fade over time, so that a single stray alert does not lead to a block but a steady stream of alerts does:

$$S_h(t) = S_h(t_{prev})\, e^{-\lambda (t - t_{prev})} + w_v \tag{5.8}$$

Here w_v is the weight of the new alert (for example, higher for a Suricata alert than for an anomaly score just above τ) and λ is a decay rate. With a half-life of 10 minutes, $\lambda = \ln 2/600\ \mathrm{s}^{-1}$. When S_h reaches the block threshold Θ, the host is blocked. The same score is shown on the dashboard as the host's current risk level.

5.7.3 Timing of ML Verdicts

Layers 2 and 3 score completed flows or active-timeout segments (Algorithm 5.1), so their verdict comes after part of the traffic has passed. Their blocks therefore apply to the rest of a long flow and to all later traffic from that source. This is acceptable for the attacks they target: beaconing, exfiltration, DDoS and lateral movement all unfold over many flows. Attacks that must be stopped at the first packet are the responsibility of Layer 1.
With the default $T_a = 10\,\mathrm{s}$, the worst-case delay for an ML verdict on a long flow is about 10 seconds plus scoring time, which is milliseconds for tree models. The data that can leave over one connection before a block is therefore roughly the upload rate $\times$ that delay. This exposure is measured directly in the live exfiltration tests as "bytes leaked before block" (Table 5.7), rather than estimated.

5.8 Automated Prevention

A BLOCK verdict is turned into a firewall entry by the enforcer. The enforcer must stop attacks quickly without ever locking the owner out of their own network, so blocking follows a strict procedure.

5.8.1 Blocking Mechanism

Blocked addresses are stored in an ipset with a per-entry timeout, referenced by a single iptables rule placed before the NFQUEUE rule in the FORWARD chain (and in INPUT, to protect the AP itself). Adding or removing an address changes only the set, not the rule list, so a block takes effect immediately and costs the same regardless of how many addresses are blocked. Entries expire automatically, so a block never outlives its intended duration even if the enforcer crashes.

5.8.2 Blocking Procedure

Algorithm 5.6 gives the procedure followed for every BLOCK verdict.

Algorithm 5.6: Enforce a BLOCK verdict

Input: offending address a; attack class y; attributed layer(s)
Output: ipset entry and audit record

if $a \in$ *allowlist (AP, gateway, DNS resolver, admin devices)* **then**
    log "block suppressed";
    return;
if *blocks issued in the last minute* $\geq R_{max}$ **then**
    downgrade to ALERT;
    raise a "block rate limit" warning;
    return;
$n \leftarrow$ number of earlier blocks of a within 24 h;
$duration \leftarrow \min(D_0 \cdot 2^n,\ D_{max})$; // e.g. $D_0 = 5\,\mathtt{min}$, $D_{max} = 24\,\mathtt{h}$
add a to the ipset with timeout $duration$;
write audit record $\{a, y, \text{layers}, duration, \text{evidence}, \text{time}\}$;
push the event to the dashboard;

The allowlist check prevents the system from cutting off its own infrastructure, even when an attacker forges packets from those addresses. The rate limit stops a flood of false positives, or an attacker spoofing many sources, from turning the NIDPS into a denial-of-service tool against its own users. Exponential back-off means a first offence is blocked briefly, in case it was a false positive, while repeat offenders are blocked for longer. Every block can be lifted manually from the dashboard.

5.9 Fault Tolerance and Failure Handling

An inline security device is itself a single point of failure: if Suricata crashes while traffic is queued to it, the whole network can go down. The NIDPS therefore defines exactly what happens when each component fails, and lets the owner choose between availability and security.

5.9.1 Health Monitoring

Every component (Suricata, flow assembler, ML scorer, decision engine, enforcer) runs as a separate systemd service. Each one sends a heartbeat to its own watchdog every second, along with load indicators such as queue depth and processing lag. A component is declared failed after $k = 3$ missed heartbeats or if its process exits. systemd restarts failed services automatically, and its own watchdog timer restarts services that are running but frozen.

5.9.2 Failure Modes

Table 5.5 lists the configurable failure modes.

Table 5.5: Configurable failure modes.

Mode	When Layer 1 fails	When an ML layer fails	Suited to
Fail-open	Packets bypass the queue and are forwarded uninspected	Remaining layers continue; affected flows are logged as unscored	Homes where internet access matters most
Fail-closed	Forwarded traffic is dropped until recovery; admin access to the AP stays open	Flows from unknown hosts are held at ALERT and new block decisions rely on Layer 1 only	Small offices handling sensitive data
Degraded (load-shedding)	Non-essential rule groups are disabled to reduce load	Only a sample of flows (1 in k) is scored, prioritising new and high-risk hosts	Temporary overload rather than a crash

Fail-open is implemented in the kernel rather than in software: the NFQUEUE rule is created with the queue-bypass option, so when no process is attached to the queue, the kernel accepts packets instead of dropping them. Fail-closed omits this option. Because the kernel enforces the mode, it holds even when the watchdog itself has crashed.

5.9.3 State Machine

Figure 5.3 shows the states each supervised component moves through.
Two thresholds (high and low watermarks) are used for entering and leaving the degraded state. Without this gap, a load hovering around one value would cause the system to switch modes back and forth many times per second. During every failure, the watchdog records when it was detected, which mode was applied, when service resumed and how many packets were dropped or bypassed. These records produce the recovery measurements reported in the evaluation.

5.10 Evaluation Methodology

The system is evaluated on two questions: how well it detects attacks, and whether it can do so inline on constrained hardware without harming the network.

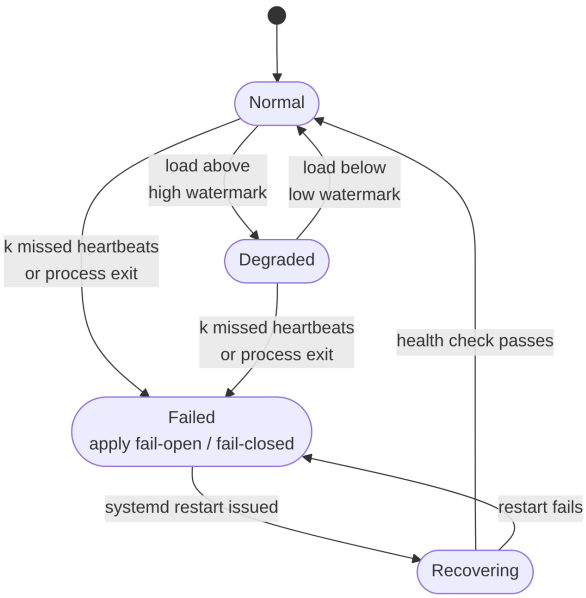


Figure 5.3: Watchdog state machine for each supervised component.

5.10.1 Detection Metrics

For every layer and for the fused system, precision, recall, F1-score and false positive rate (FPR) are reported per class and macro-averaged:

$$\begin{aligned} \text{Precision} &= \frac{TP}{TP+FP}, & \text{Recall} &= \frac{TP}{TP+FN}, \\ F_1 &= \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}, & \text{FPR} &= \frac{FP}{FP+TN} \end{aligned} \tag{5.9}$$

False positive rate receives particular attention. On a home network producing thousands of flows per hour, even a 1% false positive rate means dozens of wrong alerts or blocks every hour. For Layer 2, the area under the precision-recall curve is also reported, since it summarises performance across all thresholds and is more informative than ROC-AUC when attacks are rare.

5.10.2 Cross-Dataset Generalisation

Each model is trained on one dataset and tested on every other dataset, giving the matrix in Table 5.6. The diagonal (train and test on the same dataset, split by day) shows in-distribution performance. The off-diagonal cells show how much performance drops when the network changes, which is the realistic situation for a deployed device. Finally, models trained on all three public datasets combined are tested on the self-collected dataset.

Table 5.6: Cross-dataset evaluation matrix of macro F1-scores, to be filled from experiments.

Train / Test	**CIC-IDS-2017**	**CSE-CIC-IDS2018**	**CIC-IoT-2023**	**Self-collected**
CIC-IDS-2017	in-distribution			
CSE-CIC-IDS2018		in-distribution		
CIC-IoT-2023			in-distribution	
All three combined				

Only classes that exist in both the training and the test dataset are scored in each cell. The adaptive threshold of Section 5.5.2 is evaluated by comparing the false positive rate on the self-collected benign traffic with the global threshold and with the calibrated threshold.

5.10.3 System Performance on Constrained Hardware

Table 5.7 defines the system-level measures and how each is obtained.

Table 5.7: System performance measures.

Measure	**Definition**	**How it is measured**
Forwarding latency	Extra round-trip time added by the NIDPS	Ping RTT through the AP with the NIDPS off, then with each layer enabled in turn
Throughput	Sustained forwarding rate	iperf3 TCP and UDP tests through the AP, layers enabled incrementally
Detection latency	Time from the first attack packet to the alert	Attack start logged on the attacker machine vs. alert timestamp
Time to block	Time from the first attack packet to the ipset entry	Same, against the audit record
Bytes leaked before block	Data the attacker transferred before the block took effect	Bytes received at the lab exfiltration server before the block, repeated at $T_a = 5$ s and 10 s
Resource use	CPU, memory and temperature of the host	Sampled every second during each test, at $T_a = 5$ s and 10 s
Recovery behaviour	Time to detect a failure, time to restore service, packets lost or bypassed	Components killed deliberately under load, in each failure mode

Every attack in the scope is replayed live against test devices on the isolated lab network while background benign traffic runs. This end-to-end test checks the complete chain, from detection through fusion to blocking, rather than each model in isolation.

5.11 Conclusion

The proposed Hybrid framework and the algorithms of the backend were introduced in this chapter. Known attacks are detected at the packet level, and at the fastest rate, by Suricata and an ARP consistency check. Isolation Forest with per network and with Host level scores for exfiltration and C2 beaconing detects unusual behaviour. A Random Forest or XGBoost classifier is used to classify the flows into known attack families. The analysis of the encrypted traffic is done using JA3/JA3S fingerprints and flow stats, without decrypting it, and all datasets are reconstructed with the device's flow extractor, so that features are identical during training and runtime. These signals are converted to verdicts by a decision-fusion algorithm, which are enforced by a host evidence accumulation, using ipsets with timing constraints and a mechanism to prevent self-inflicted outages, and applied by a watchdog, with a configurable failure mode backed by the kernel. The evaluation protocol not only tests the quality of the detection per dataset, but also the latency, throughput and recovery of the system on real hardware.

Chapter 6

High-Level and Low-Level Design

This chapter describes how the methodology of Chapter 5 is realised as software. It moves from the high level to the low level: first an overview of the system and the considerations that shaped its design, then the architecture of the whole system and of each subsystem, the strategies behind the main architectural decisions, the class diagram, and finally the policies and tactics that govern implementation, testing and maintenance.

6.1 System Overview

The NIDPS is a self-contained security appliance built from software on a single Linux host. The host replaces the role of the home router's wireless network: clients connect to the NIDPS access point, and the NIDPS routes their traffic, through its own NAT, to the existing ISP router, which keeps handling the internet connection. No change is needed on client devices or on the ISP router.
Functionally, the system does five things:

1. **Provides network access:** A wireless access point with DHCP, DNS and routing.
2. **Inspects traffic:** Inline signature inspection of every packet, plus flow-level analysis of all traffic, including encrypted traffic through metadata.
3. **Decides:** Fuses signature, anomaly and classification results into a per-flow verdict and a per-host risk score.
4. **Responds:** Blocks offending addresses automatically for a limited, escalating time.
5. **Reports and stays available:** Shows live statistics and alerts on a web dashboard, and keeps the network in a defined state when a component fails.

The system is made up of independent services connected by a local event bus, rather than as one large program. The packet-forwarding path is kept as short as possible (kernel plus Suricata), and everything slower like machine learning, the database and the dashboard, sits beside that path instead of in it. Sections 6.3 and 6.4 explain this choice in detail.

6.2 Design Considerations

This section records the assumptions, constraints, goals and development methods that shaped the design.

6.2.1 Assumptions and Dependencies

- **Hardware:** The host has at least a quad-core 64-bit ARM or x86 CPU, 4 GB of RAM, a wireless adapter that supports AP mode and an Ethernet port for the uplink.
- **Operating system:** Debian based Linux distribution with systemd, iptables (use of nftables is accepted, but not required), ipset (use of NFQUEUE is accepted, but not required) and support in the kernel.
- **Network position:** NIDPS is the only wireless network clients access. Devices which are connected directly to the ISP router via cable or directly by the ISP router's Wi-Fi are not protected. Turning off the ISP router's WiFi is a part of the set-up process.
- **Third-party software:** Suricata 7 or later with Emerging Threats Open ruleset, hostapd, dnsmasq, Python 3.11+ with scikit-learn and XGBoost. Periodic Internet access is required for ruleset updates.
- **End users:** Owner is NOT a security expert. Defaults should be safe and not require tuning, and dashboard should provide information about alerts in easy-to-understand terms.
- **Trust:** The NIDPS host and the admin's devices are trusted. The ISP router is assumed to be working, but is not trusted to provide any security.
- **Likely changes:** Changes are likely to include new attack types and new rules, models may be retrained and the TLS fingerprinting method could change from JA3 to its successor JA4, as TLS client libraries start to shuffle the order of extensions. Each of these is isolated by the design behind its component.

6.2.2 General Constraints

Table 6.1 lists the global constraints and their impact on the design.

Table 6.1: General constraints and their impact on the design.

Constraint	Source	Impact on the design
Limited CPU and memory	Low-cost target hardware	Tree-based models instead of deep networks; per-flow rather than per-packet ML; incremental feature statistics; rule groups trimmed to the home-network scope
Inline position	The NIDPS forwards all traffic	Forwarding path kept to kernel + Suricata; ML runs off-path; failure mode enforced in the kernel
Encrypted traffic	Most traffic uses TLS/QUIC	No payload inspection for ML; detection from flow metadata and JA3/JA3S only
Privacy	Household traffic is personal	No decryption; no payload storage; only flow metadata and alerts are kept, with a retention limit
Availability	A home network must not go down	Configurable fail-open default; block rate limits; allowlist; ipset entries that expire
Interoperability	Training data and live traffic must match	One flow extractor used offline and online; model files carry their feature schema
Security of the device	The NIDPS is itself a target	Dashboard bound to the LAN only, authenticated, management traffic allowlisted; services run without root where possible
Testability	Attacks cannot be run on real users	Isolated lab network and attacker machine for all attack tests

6.2.3 Goals and Guidelines

1. **Never make the network worse than a plain router:** Every design choice is checked against this. It is why fail-open is the default, why blocks expire and why the number of blocks per minute is capped.
2. **Keep the fast path short:** Only the kernel and Suricata sit in the forwarding path. Anything that can be done after the fact is moved off it.
3. **Defence in depth:** Each layer covers a weakness of another layer as no single detector is trusted to catch everything (Section 5.1).
4. **Speed over memory where the two conflict, within a fixed budget:** In order to fit the hardware, the model sizes are capped and loaded fully in memory to avoid disk access during anlysis.

5. **Explainable decisions:** Every block records which layer decided, why, and with what evidence, so the owner can trust or override it.
6. **Same code for training and serving:** Features are computed by one implementation to rule out train/serve skew.
7. **Configurable, safe defaults:** Thresholds, failure mode and block durations are configurable, but the defaults work by default.

6.2.4 Development Methods

The project follows an iterative, incremental approach in short sprints of two weeks, aligned with the FYP milestones. Each increment delivers a working vertical slice: first the access point with Suricata (Layer 1, completed), then flow extraction and Layer 2, then Layer 3, then fusion and enforcement, and finally the watchdog and dashboard. This order means the system protects the network from the first increment onwards and every later layer can be measured against a working baseline.
The machine learning work follows the CRISP-DM cycle (data understanding, preparation, modelling, evaluation, deployment), because model quality depends on several rounds of data cleaning and re-evaluation. A plain waterfall model was considered and rejected: detection performance on real traffic cannot be known in advance, so requirements such as thresholds and feature sets must be allowed to change as results come in.

6.3 System Architecture

The system is decomposed into five runtime subsystems on the NIDPS host and one offline subsystem used for training. Figure 6.1 shows the external architecture (how the NIDPS sits between clients, the ISP router and the administrator) together with the internal subsystems and their main data flows.
The responsibilities are partitioned as shown in Table 6.2.
There are two ways in which the subsystems work together. The forwarding path is entirely in kernel and Suricata and there is only one feedback loop from the enforcer to ipset. Everything else is communicated via the local event bus: detectors publish evidence, decision engine consumes it and publishes verdicts, and the rest (dashboard and event store) subscribe to it.

6.3.1 Subsystem Architecture

This section describes each subsystem in more detail.

6.3.1.1 Network Access and Data Plane

This subsystem is mostly configuration of existing Linux components; the new parts are the netfilter rule layout and its management. Figure 6.2 traces one packet from a client to the

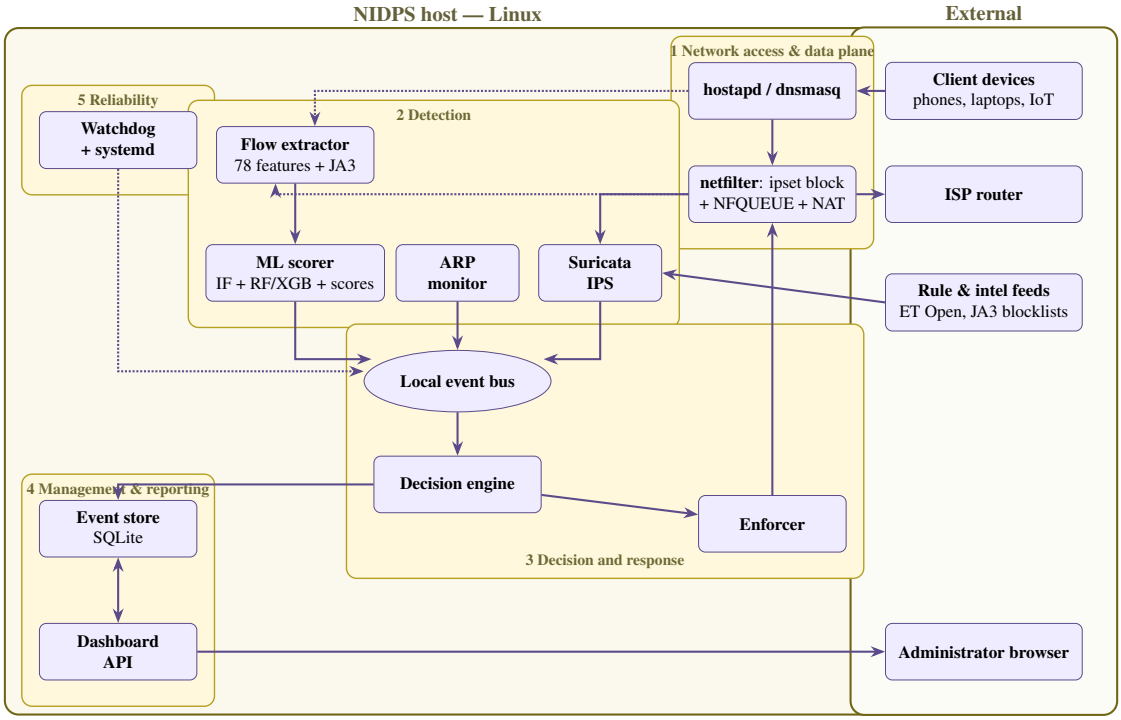


Figure 6.1: System architecture of the NIDPS.

Table 6.2: Subsystem responsibilities.

Subsystem	Responsibility	Main components
1. Network access and data plane	Provide Wi-Fi, addressing and routing; hold the inline queue; enforce blocks and failure mode in the kernel	hostapd, dnsmasq, netfilter (iptables, ipset, NFQUEUE, NAT)
2. Detection	Produce evidence: signature alerts, ARP alerts, flow records, anomaly scores, class probabilities	Suricata, ARP monitor, flow extractor, ML scorer
3. Decision and response	Turn evidence into verdicts and host risk; apply and expire blocks	Decision engine, enforcer
4. Management and reporting	Store events; expose live statistics, alerts and configuration to the admin	Event store, dashboard API and web UI, configuration manager
5. Reliability	Detect failed or overloaded components; restart them; apply failure mode; record recovery metrics	Watchdog, systemd units
Offline: training	Regenerate datasets, train and validate models, publish versioned model files	Dataset builder, trainer, model registry

internet.

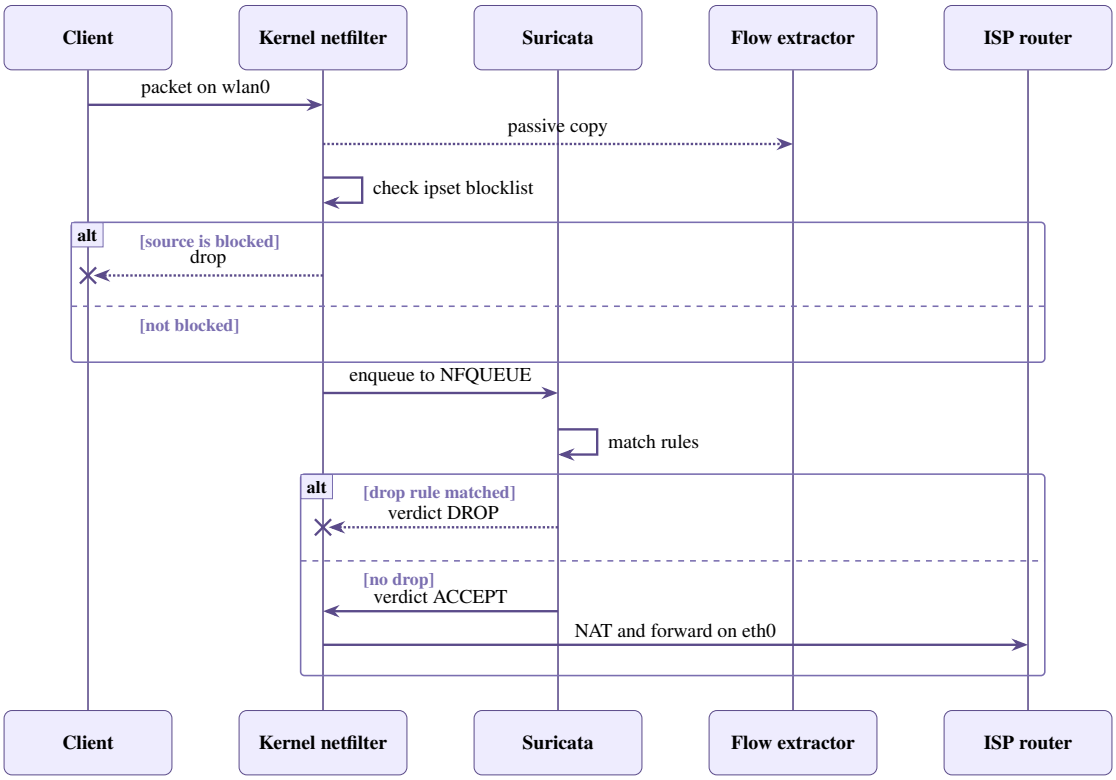


Figure 6.2: Packet path through the data plane.

The rule order matters: the ipset check comes before the queue, so packets from blocked hosts are discarded in the kernel and never cost Suricata any CPU. During a DDoS from a blocked source, this keeps the inspection engine free for other traffic.

6.3.1.2 Detection Subsystem

There are four producers in the detection subsystem. Suricata and the ARP monitor issue alerts directly. The flow extractor will publish flow records, and the ML scorer will enhance the flow records and publish them. The ML scorer consists of three sub-components internally:

- **Feature validator:** Validates each flow record against the feature schema contained in the model and rejects the flow record if it does not match or is malformed.
- **Flow models:** Once loaded at start-up, they are applied to each flow record, according to the Isolation Forest and the RF/XGBoost classifier.
- **Host behaviour tracker:** Maintains per-host state (baseline of exfiltrated bytes, timestamps of connections, local JA3 counts) and calculates the exfiltration and beaconing scores of Sections 5.5.3 and 5.5.4 respectively. Its state is bounded: When inactive host/keys are entered, they expire.

6.3.1.3 Decision and Response Subsystem

The decision engine consumes all evidence from the bus, joins Suricata alerts with flow records by flow identifier and time, runs Algorithm 5.5 and maintains the host scores of Section 5.7.2. The enforcer receives BLOCK verdicts and runs Algorithm 5.6. The two are kept separate so that the component allowed to change firewall state is small, simple and easy to audit.

6.3.1.4 Reliability Subsystem

While the watchdog has application-level health (queue depth, processing lag) information and determines when to switch to degraded mode, systemd has process level supervision (automatic restart, start order, resource limits). The Failure mode at the kernel level (queue bypass on or off) is set during start up of the data plane and is not dependent on the presence of the watchdog.

6.3.1.5 Management and Reporting Subsystem

The Dashboard API is only available on the LAN side. It counts live flows per second, alerts and blocks per layer, shows an alert list with the evidence used for the verdict, has a list of currently blocked hosts with an unblock action, presents the host risk scores, displays the health status of the components and allows you to set thresholds, failure mode and an allowlist. Configurations are checked, saved and posted on the bus for running services to get.

6.3.1.6 Offline Training Pipeline

Figure 6.3 shows the offline pipeline and the hand-over of models to the device.

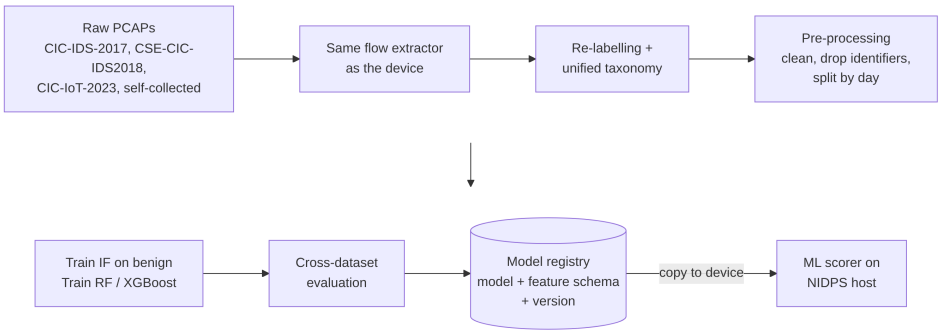


Figure 6.3: Offline training pipeline and model hand-over to the device.

Training runs on a workstation, not on the NIDPS host, because it needs far more memory than inference. The only hand-over to the device is a model package containing the model file, its feature schema and a version manifest.

6.4 Architectural Strategies

Each strategy below notes the decision made, the rationale for the decision and what was rejected.

6.4.1 Layered Defence with a Short Fast Path

Decision: Detection is divided into a packet-level layer on the forwarding path (Suricata) and flow-level layers off the forwarding path (ML), with a fusion layer.
Reasoning: ML inference on each of the forwarded packets would make it impossible to stay within the CPU budget, and would introduce additional latency to each connection, contradicting the first objective of Section 6.2.3. The only forwarding latency cost for flow-level ML outside the path will be the cost of the Suricata rule that must be applied to the first packet.
Alternatives rejected: ML-only inline detection (too slow, and no immediate protection against known attacks); Suricata-only detection (misses zero-day and behavioural attacks).

6.4.2 Metadata-Only Analysis of Encrypted Traffic

Decision: Encrypted traffic analysed only using JA3/JA3S fingerprints and flow statistics.
Reasoning: TLS interception would involve trusted root certificate on each client, would not be compatible with certificate pinning, would break some IoT devices and would increase CPU load. It would also allow for all the traffic to be displayed in plain text from all devices in the household, which would be a huge privacy hazard.
Alternatives rejected: Full TLS interception (above); ignoring encrypted traffic (most C2 now uses TLS). Known limitation: browsers now randomise TLS extension order making JA3 less stable for browser traffic. Since the fingerprinting is separated from the rest of the flow extractor, other components can remain the same and JA4 can be changed to sort extensions before hashing.

6.4.3 Offline Training, Online Inference

Decision: Models to be trained on a workstation and deployed as read-only versioned packages. The device can only do inference and threshold calibration.
Reasoning: A training process of hundreds of millions of flows requires much more memory than the device available. Online retraining on a device will also enable an adversary to slowly condition the model to think that malicious traffic is acceptable (model poisoning). Calibrating just the threshold (Section 5.5.2) adapts to the local network with limited risk.
Alternatives rejected: Continuous online learning (poisoning risk, cost of hardware); a fixed global threshold (false positives with real home traffic).

6.4.4 Local Services Connected by an Event Bus

Decision: Each component is its own systemd service and message exchange is handled via a local publish/subscribe bus. Redis Streams is the proposed implementation.
Reasoning: The failure modes require service isolation, which is where one service can crash without impacting the others, and where systemd can restart each service independently. The bus separates the producers from the consumers. Redis Streams is lightweight, has a bounded backlog which a restarted consumer can resume from and allows for consumer groups that run the dashboard and the decision engine independently, reading the same events.
Alternatives rejected: Direct function calls in one process (no fault isolation); Apache Kafka (too heavy for the hardware); plain unix sockets between each pair of components (simple, but every new consumer requires new wiring, and while a consumer restarts, messages are lost).

6.4.5 Kernel-Enforced Blocking and Failure Semantics

Decision: Blocking and failure (ipset with timeouts, and NFQUEUE bypass or not) are both handled by the kernel and not user space software.
Reasoning: User space programs can crash but the kernel continues to run. While the queue-bypass flag applies the selected mode regardless of whether the watchdog is down or not, timed ipsets entries expire even if the enforcer fails.
Alternatives rejected: Add iptables rules one per blocked host (slow when many hosts are blocked and iptables rules are still there after a crash); A user-space proxy that drops traffic (adds latency and is itself a crash risk).

6.4.6 Choice of Languages and Products

Table 6.3 summarises the main product and language choices.

Table 6.3: Product and language choices.

Need	Choice	Main reason	Alternative considered
Signature IPS	Suricata	Multi-threaded, native NFQUEUE IPS mode, EVE JSON output, TLS/JA3 support, maintained ET Open rules	Snort 3 (similar capability; Suricata chosen for its JSON output and threading)
Services and ML	Python 3	scikit-learn and XGBoost available natively; fast development	C++ (faster, but much slower to develop; reconsidered only if profiling shows Python is the bottleneck)
Models	scikit-learn Isolation Forest and Random Forest, XGBoost	Small, fast tree models suited to tabular flow data	Deep autoencoders and LSTMs (higher compute cost, no clear gain on flow features)
Event store	SQLite	Zero administration, single file, enough for one device	PostgreSQL (needs a server process)
Dashboard	Python web API with WebSocket push and a lightweight web front end	Live updates with little code; reuses the Python stack	Grafana (heavy and less suited to showing verdict evidence and controls)
Firewall	iptables + ipset	Stated in the proposal; well documented; ipset handles large blocklists efficiently	nftables sets (modern equivalent; migration path kept open)

6.4.7 Error Detection and Recovery

There are three levels at which errors are dealt with.

1. **Record level:** Malformed flow records or alerts are rejected by the feature validator and counted, never passed to models.
2. **Component level:** Heartbeats and systemd supervision detect crashes and hangs and restart the component (Section 5.9).
3. **System level:** The system level (kernel-enforced failure mode) determines the fate of traffic when a component is down. All three levels will log to the event store and hence the dashboard will display when the system was not completely protected.

6.4.8 Data Storage, Persistence and Retention

We only store derived data: block information, risk history per host, calibration state and configuration, decisions, alarms and verdicts. No packet data or payload is stored on normal operation. Flow data is stored in a relatively short rolling window (e.g. 7 days) for analysis and calibration purposes before being purged. This provides an effective level of privacy for domestic users whilst placing a reasonable amount of data on a small storage device.

6.4.9 Future Extension

The system is structured to make the most probable extensions accomplished by adding functionality rather than modifying functionality. A new detector emits to the bus. A new response (e.g. notification to the owner's phone) subscribes to decisions. A new extractor for support of a new JA4 or new model is added.

6.5 Domain Model/Class Diagram

Figure 6.4 shows the main classes of the runtime system. A Flow is the core class that passes along its properties and TLS fingerprint if present. Each detector creates Evidence and all implement the same Detector interface. Evidence is converted into a Verdict by the DecisionEngine and BLOCK verdicts are converted into BlockEntry entries by the Enforcer.
The Detector interface is the main extension point. A new detection technique is added by implementing it and publishing its Evidence, without changes to the DecisionEngine. The Enforcer is the only class that changes firewall state, which keeps that responsibility in one small, auditable place.

6.6 Policies and Tactics

The following policies only influence aspects of the implementation and do not change the overall structure of the system.

6.6.1 Rule and Threshold Management

A scheduled job updates the ET Open ruleset every day, it is syntax tested with Suricata's test mode and loaded with a live rule reload, so there will never be any inspection that doesn't get done. In case of failure, the last ruleset is still in effect. Custom rules are stored in a separate file in version control, and are never overwritten by changes to ET rules. All numeric thresholds (rule thresholds, flow timeouts T_a and T_i, τ, θ values, Θ, block durations) are read from one configuration file with documented defaults.

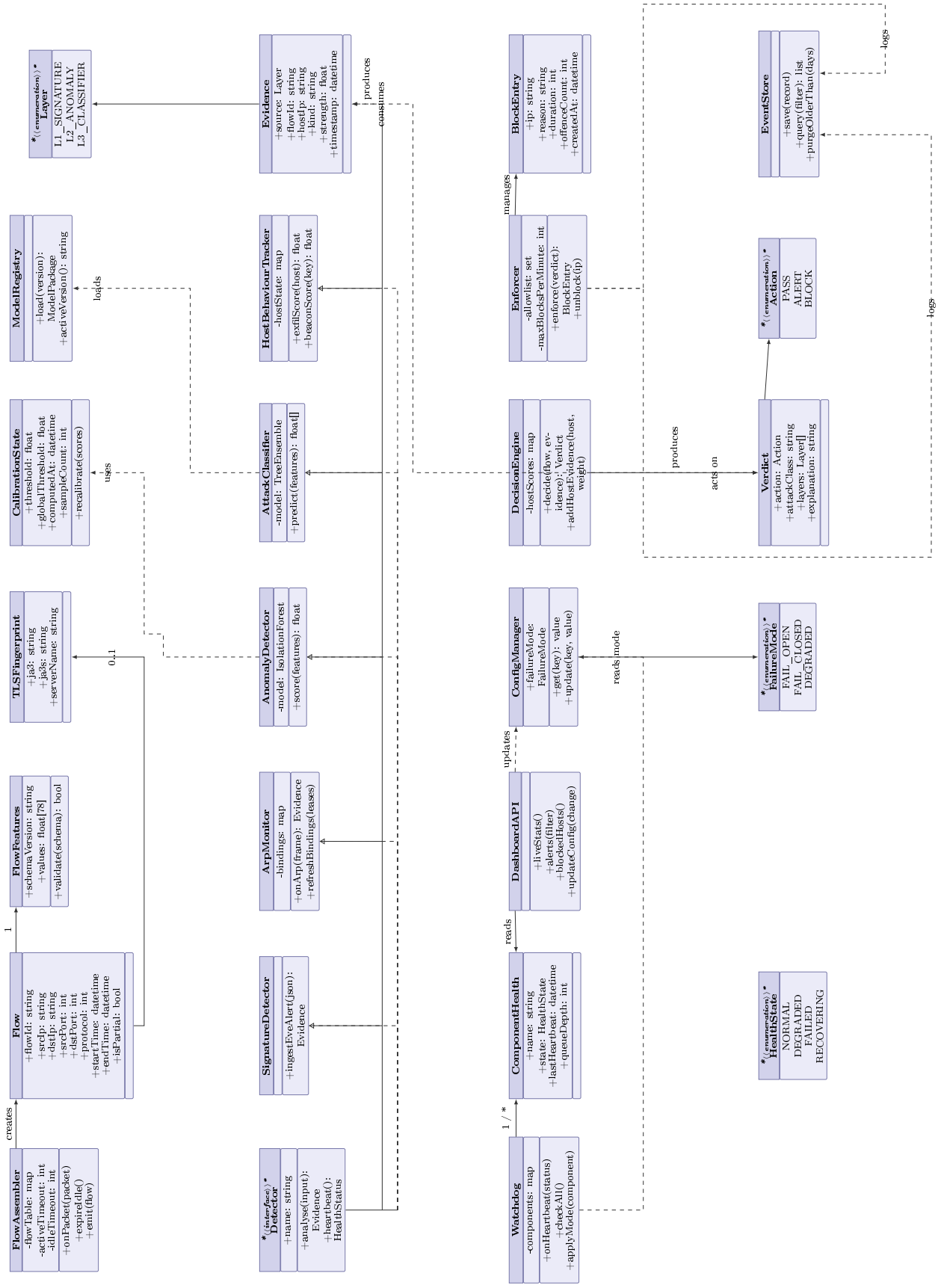


Figure 6.4: Class diagram of the NIDPS runtime.

6.6.2 Model Packaging and Versioning

Each model package contains the model file, the list of features ordered by version of the extractor, the class list and the training date and metrics. The ML scorer rejects any package, whose feature list is incompatible with the running extractor, at load time. XGBoost models are stored in their native JSON format, not as Python pickles, as loading a pickle can run arbitrary code. scikit-learn models are stored in joblib files, and only loaded from the registry directory which is writable only by the deployment user. The last package is preserved for rollback in case of a bad model.

6.6.3 Engineering Trade-offs

- **Detection speed vs. accuracy:** ML verdicts are per flow, not per packet (Section 5.7.3), sacrificing a short delay for ML-detected attacks and eliminating any additional forward delay.
- **False positives vs. missed attacks:** The calibration percentile p and the block thresholds are the explicit controls. Defaults prefer fewer false positives as the loss of trust on a home network is fast.
- **Availability vs. security:** Selected at deployment time (failure mode), default is fail-open for homes.

6.6.4 Coding Guidelines and Conventions

Python code adheres to PEP 8, uses type hints and runs through a linter and type-checker as part of the continuous integration. Each service is a separate package with one exposed entry point. All event bus messages are versioned JSON with a schema. This allows producers and consumers to be upgraded separately. All logging is structured JSON with timestamp, component name and identifier for either a flow or host. This enables events to be followed through services.

6.6.5 Requirements Traceability

Every goal from Chapter 2 is mapped to the components that implement it and the tests that verify it, in a traceability matrix kept in the repository and updated at every sprint review (Table 6.4).

Table 6.4: Requirements traceability matrix.

Proposal objective	Implemented by	Verified by
Transparent inline NIDPS between clients and ISP gateway	Data plane (Section 6.3.1)	Connectivity and latency tests (Table 5.7)
Suricata with ET ruleset for known attacks	SignatureDetector, custom rules (Section 5.4)	Live attack replay: SSH brute force, port scan, DNS spoofing
ML pipeline for anomaly detection and classification	AnomalyDetector, AttackClassifier, HostBehaviourTracker (Sections 5.5–5.6)	Detection metrics and live replay: C2 beaconing, exfiltration, DDoS, lateral movement
Cross-evaluation on three public datasets	Offline training pipeline (Figure 6.3)	Cross-dataset matrix (Table 5.6)
Automatic blocking via iptables	Enforcer (Section 5.8)	Time-to-block measurement; allowlist and rate-limit tests
Watchdog and configurable failure handling	Watchdog, systemd units (Section 5.9)	Fault-injection tests in each failure mode
Web dashboard	DashboardAPI (Section 6.3.1)	Functional UI tests; usability check with non-expert users

6.6.6 Testing Plan

1. **Unit tests for each algorithm:** Flow assembly on crafted PCAPs, JA3 computation against published reference hashes, threshold calibration, fusion rules and the enforcer's allowlist, rate limit and back-off.
2. **Integration tests:** Replay labelled PCAPs around the entire system on a test system and compare results to the label.
3. **Live attack tests:** Involve launching each in-scope attack from the attacker side onto our test systems on the isolated test network.
4. **Performance tests:** For latency, throughput and resource use (Table 5.7).
5. **Fault-injection tests:** Each service is killed or frozen under load in each failure mode, and recovery time and traffic impact are recorded.
6. **Soak test:** The system runs for at least one week on a real home network in alert-only mode to measure false positives before blocking is enabled.

6.6.7 Hierarchical Organisation of the Source Code

The source code is organised as follows:

```
nidps/
  dataplane/      hostapd, dnsmasq and netfilter configuration + setup scripts
  suricata/       suricata.yaml, custom.rules, rule-update job
  services/
    arpmon/       ARP consistency monitor
    flowextract/  flow assembler, feature computation, JA3/JA3S
    mlscorer/     feature validator, models, host behaviour tracker, calibration
    decision/     decision engine and host scores
    enforcer/     ipset management, allowlist, back-off
    watchdog/     heartbeat monitoring, state machine, failure modes
    dashboard/    web API and front end
  common/         bus client, message schemas, config loader, logging
  training/       dataset builder, labelling, training and evaluation scripts
  models/         model registry (versioned packages)
  deploy/         systemd unit files, install and upgrade scripts
  tests/          unit, integration, fault-injection tests and test PCAPs
  docs/           traceability matrix, design notes
```

6.6.8 Build and Deployment

The system is installed on a clean Debian image with one installation script, which installs system packages (Suricata, hostapd, dnsmasq, ipset, Redis), creates a Python virtual environment with pinned versions of dependencies, copies the configuration and active model package, and enables the systemd units in dependency order: data plane first, then Suricata, and then the rest. Only one service is upgraded at a time and that one service is restarted. Each service is supervised and the failure mode is enforced at kernel level, so that an upgrade does not leave the network with no behaviour.

6.6.9 Maintenance

Routine maintenance is automated: daily rule updates, weekly threshold recalibration (see Algorithm 5.4), and purging logs and flow-records outside of the retention window. Models are retrained offline when new datasets are available, or the dashboard indicates that the alert rate is increasing that is not explained by calibration, or when new classes of attacks become available. The health history report on the Dashboard indicates to the owner when the system was operating in a degraded or failed condition.

6.7 Conclusion

In this chapter we translated the methodology of Chapter 5 into a concrete design. The NIDPS is a single Linux host, with five runtime subsystems (data plane, detection, decision and response, management and reporting, and reliability), and an offline training pipeline. The main architectural decisions are that the forwarding path is short, encrypted traffic is analysed only via metadata, that training is done offline and thresholds are be calibrated online, that components are separated with the supervised services communicating with

each other via an event bus, and that blocking and failure behaviour is controlled by the kernel. The class diagram establishes a common Detector interface which allows quick addition of new detectors, and restricts the number of firewall changes to only one Enforcer. The policies relating to rule and model management, testing, traceability and deployment complete the design, and link each policy goal with the component responsible for it and the test for it.

Chapter 7

Implementation and Test Cases

This chapter shows how the design in Chapter 6 became working software. Most of the system is finished: the shared foundations used by every service, the flow extractor, both Layer 1 detectors (the Suricata bridge and the ARP monitor), the Layer 2 anomaly service with its Isolation Forest model, and the scripts for building the self-collected dataset. The decision engine and the dashboard are partly built. The classifier, enforcer and watchdog are not written yet. Everything unfinished is listed in its own section, so no part of this chapter describes code that does not exist.

7.1 Implementation

Each subsection describes what a component does, which libraries it uses and why we built it that way. Program code is left out.

7.1.1 Platform and Tools

The services are written in Python 3.11 and run as separate processes. Scapy captures the packets, Redis Streams carries messages between services, SQLite stores flow records, settings and calibration state, and scikit-learn provides the Isolation Forest. Suricata is the only part not written in Python: it runs as its own process and the NIDPS reads its alert log. Table 7.1 lists the tools. Capturing packets needs raw socket access, so the flow extractor and the ARP monitor run as root or with the CAP_NET_RAW capability.

Table 7.1: Platform and tools used by the implementation.

Need	Tool	Used for
Packet capture	Scapy	Sniffing frames on the wireless interface. TLS handshakes are parsed by hand from the raw bytes.
Signature detection	Suricata	Layer 1 rules. Alerts are read from its JSON log.
Messaging	Redis Streams (redis-py)	Publish and subscribe between the services.
Storage	SQLite	Flow records, settings, calibration state.
Anomaly model	scikit-learn, NumPy, joblib	Isolation Forest scoring and model loading.

7.1.2 Implementation Status

Table 7.2 shows what is built. The access point itself (hostapd, dnsmasq and the netfilter rules) was set up by hand on the device, so it is not in the repository.

Table 7.2: Implementation status of the NIDPS components.

Component	**Status**
Shared foundations (bus, database, settings)	Implemented
Flow extractor	Implemented
ARP monitor	Implemented
Suricata bridge	Implemented
Layer 2 anomaly service and Isolation Forest model	Implemented
Self-collected dataset tooling	Implemented
Decision engine	Partly implemented
Dashboard	Partly implemented
Layer 3 classifier	Planned
Enforcer	Planned
Watchdog	Planned
Offline training pipeline	Planned. Only the dataset capture tools exist.
Access point data plane	Configured on the device

7.1.3 Communication Between Services

The services never call each other. Everything goes through Redis Streams, of which there are five: flow-ready notices, evidence from the detectors, verdicts, configuration changes and heartbeats. The decision engine is partly built, and it reads the evidence stream and writes verdicts. Nothing reads the heartbeats yet because the watchdog does not exist, and the configuration stream is reserved for later. A small bus class wraps the Redis calls. A service publishes to a stream, reads through a consumer group and acknowledges a message only after handling it, so a service that restarts picks up where its group stopped instead of losing messages.

Every message is a JSON envelope holding a schema version, a unique id, a UTC timestamp, a type and a payload. A reader rejects any schema version it does not know, which lets us upgrade producers and consumers one at a time. A flow-ready message carries only the flow id, the host address and a partial-flow flag. The 78 features stay in SQLite and are read back by id, which keeps messages small.

7.1.4 Shared Foundations

7.1.4.1 Database

The schema has the twelve tables of the data dictionary (Section 4.9.2), plus indexes on the columns queried most often: host and start time for flows, host for evidence, and host and status for blocks. Every service opens the database through the same function. It turns on foreign keys, creates any missing tables and checks the schema version. Because creating tables is safe to repeat, whichever service starts first creates the file. Timestamps are stored as UTC text in ISO 8601 form, so comparing them as strings gives time order, and the retention purge depends on that.

7.1.4.2 Configuration

The 40 tunable settings are declared in one schema, each with a type, a default and an allowed range or set of values. Seeding copies the defaults into the database but never overwrites a value the administrator has changed. A change is checked against its range, saved with the user and time, and written to the audit log in the same transaction (FR-10.6). Services cache settings for five seconds, so a new threshold takes effect within seconds and needs no restart.

7.1.4.3 Detector Interface and Logging

One module defines the value types shared by the services: flow, features, evidence, verdict, block entry and component health. Evidence has a strength between 0 and 1 and a separate weight, which the decision engine adds to a host's risk score. Each service fills in the weight from the settings before it publishes. Every detector implements the same interface, which has two methods: one analyses an input and returns zero or more pieces of evidence, the other reports the detector's health. The ARP monitor, the Suricata bridge and the anomaly detector all implement it, so a new detector can be added without touching the decision engine (Figure 6.4). Each service writes one JSON object per line to standard output, and the systemd journal collects them in one format.

7.1.5 Flow Extraction Service

7.1.5.1 Packet Capture

The flow extractor captures with Scapy on the configured interface and keeps no packets in memory. It reduces each IPv4 TCP or UDP packet to a small record: timestamp, addresses, ports, protocol, sizes, TCP flags and window. Scapy is used only in this step, so the rest of the extractor can be tested with hand-made records. A flow is identified by its 5-tuple, with the two endpoints ordered so that both directions of a conversation share one key. The first packet seen sets the forward direction.

7.1.5.2 Flow Assembly

Active flows sit in a table keyed by the 5-tuple. Each packet updates its flow, and the assembler then applies the rules of Algorithm 5.1. A flow ends when both sides have sent FIN or when an RST arrives. If a flow stays open past the active timeout (10 s by default), it is emitted as a partial flow and a new segment starts under the same key. That way a long upload is analysed while it happens, not after it ends. About once a second, an idle check emits every flow that has been silent for the idle timeout (30 s by default). These are marked partial because they never closed cleanly. If a segment starts after a rollover and then receives no more packets, it is dropped, so the service never emits an all-zero record. An error on one packet is logged and the loop carries on.

7.1.5.3 Feature Computation

The 78 features are computed in a single pass as packets arrive, and the packets themselves are not stored (NFR-14). For packet lengths, inter-arrival times, header lengths and active and idle periods, the extractor keeps running statistics updated with Welford's method (Equation 5.1), so memory per flow stays constant. It also counts TCP flags and records the initial window sizes. When a flow is emitted, the values are put in a fixed order. Rates are divided by the flow duration, with a floor of one microsecond so that single-packet flows do not divide by zero. A gap of one second or more between packets ends an active burst and counts as an idle period.
The feature names and groups are modelled on CICFlowMeter so the inputs look familiar. Our values are single-pass statistics and will not match the published dataset files exactly. That matters little, because we regenerate the public datasets from their raw captures with this same extractor (Section 5.2.3).

7.1.5.4 TLS Fingerprints

The extractor reads TLS hellos by hand from the raw bytes. When a TCP payload starts with a handshake record, it checks for a ClientHello or ServerHello and parses it with Python's struct module. This avoids Scapy's TLS layer and the extra dependencies it needs. A hello split across several segments is not reassembled. JA3 and JA3S fingerprints follow Algorithm 5.2: GREASE values are removed, each field list is written as decimal numbers joined by hyphens, the fields are joined by commas, and the result is hashed with MD5. The ClientHello parser also reads the server name. A truncated or malformed hello gives an empty default rather than an exception, so a damaged packet cannot stop capture. All of this sits behind one function, so moving from JA3 to JA4 means changing one file.

7.1.5.5 Storage and Retention

Each emitted flow is stored with its 5-tuple, UTC start and end times, segment number, fingerprints, server name, feature list and extractor version. A feature vector without exactly 78 values is rejected. The 5-tuple is kept for investigation and is not a model input. After

storing, the service publishes a flow-ready message. It deletes records older than the retention window (7 days by default) about once a minute (FR-13.3), and it publishes a heartbeat at a set interval.
Two limits apply. The idle check, the deletion and the heartbeat are all triggered by arriving packets, so on a silent network none of them run. And the host record is created from the source address of the first packet, which for an inbound connection is an outside host.

7.1.6 Layer 1: Signature Detection and ARP Monitoring

7.1.6.1 Suricata Rules

The Suricata setup in the repository is the one we used for the lab captures. It watches the lab's host-only network in passive IDS mode, writes alerts and flow records to a JSON log, and loads only our custom rules. The Emerging Threats ruleset can be added with suricata-update but is switched off by default. Inline blocking through NFQUEUE belongs to the data plane on the device and is not part of the repository.
Four custom rules are active, and each counts events per address inside a time window. SSH brute force is five SYNs to port 22 from one source within 60 s. A port scan is SYNs to twenty ports within 10 s. A SYN flood is 100 SYNs to one destination within 10 s. DNS spoofing is any DNS reply from a source other than the access point's resolver. That rule contains the lab resolver's address, so it must be edited for each deployment. A fifth rule, for known-bad JA3 hashes, is written but commented out until we record a lab hash.

7.1.6.2 Suricata Bridge

The bridge turns Suricata's log into evidence. It polls the log once a second and remembers both its position and the identity of the file, so it starts over if the log is rotated or truncated. It reads only complete lines, which means a line Suricata is still writing is picked up on the next poll. Blank and malformed lines are skipped, and only alert events are kept. Each alert becomes Layer 1 evidence with the alert's source address as the host. Strength follows severity: 1.0 for a drop action or severity 1, 0.6 for severity 2, 0.3 for severity 3 and 0.25 otherwise. The signature weight is 3.0 by default. Alerts are not tied to a flow record, so this evidence carries no flow id and is matched to hosts later.

7.1.6.3 ARP Monitor

The ARP monitor follows Algorithm 5.3. It looks only at ARP replies and gratuitous requests, meaning requests whose sender and target address are the same. It raises evidence of strength 1.0 in three cases: an address with a known MAC shows up with a different one; a host claims the gateway address with a MAC other than the access point's; or one MAC sends more than ten replies in ten seconds. A second window across all MACs catches a storm in which every packet uses a new random MAC.
The known address-to-MAC bindings come from the dnsmasq lease file, which the monitor rereads every five seconds. If a reread returns nothing after bindings were loaded, it

keeps the old ones and logs a warning, so a read failure cannot quietly switch detection off. Memory is bounded because old entries are removed as they age. The gateway address and the access point MAC are required settings, and the service will not start without them.

7.1.7 Layer 2: Anomaly Detection Service

7.1.7.1 Scoring and Calibration

The anomaly detector wraps a scikit-learn Isolation Forest. It first checks that each feature vector has 78 values under the expected schema version, and it skips the flow if not. scikit-learn returns the negative of the anomaly score in the original paper, so the result is negated and clipped to the range 0 to 1 to match Equations 5.2 and 5.3. A flow produces anomaly evidence only when its score is above the calibrated threshold.
The model is loaded from the database: the newest active package of the Isolation Forest type, which also supplies the global threshold. If the load fails, the service logs a warning, scores nothing and tries again every 60 seconds. The host-behaviour checks need no model, so they keep working in the meantime.
The threshold is recalibrated as in Algorithm 5.4. With fewer than 2000 scores, the global value is kept. Otherwise the 99.5th percentile of the local scores is used, limited to within 0.15 of the global threshold. Recalibration runs once seven days have passed. It leaves out every flow from a host that has Layer 1 evidence or an active block, so an attacker present during calibration cannot raise the threshold. From the same flows it also computes the upload limit used by the exfiltration check, which is the 99.9th percentile of forward bytes per flow segment. Taking this limit from the same sample is an addition to the algorithm as designed.

7.1.7.2 Host Behaviour: Exfiltration and Beaconing

Exfiltration is checked per host, in 60-second buckets of outbound bytes. When a bucket closes and the host already has a baseline, the bucket is flagged if its bytes exceed the host's moving average by four standard deviations and outbound traffic is at least three times inbound. The bucket is then folded into the average with a smoothing factor of 0.3 (Equation 5.4). A host's first bucket never raises evidence, because there is no baseline yet. Flows are reported when they close, roll over or expire, so observations do not arrive in time order. Closing a bucket as soon as a later one appears would split one window into incomplete pieces. The tracker therefore keeps several open buckets per host and closes one only when the newest end time seen across all hosts has passed it by the idle timeout plus one bucket length. Older observations are counted and dropped. If no flow arrives for the idle timeout plus two bucket lengths, all open buckets are flushed so the last window is still checked.
The fast path does not wait for a bucket to close. If a partial segment sends more than the upload limit to a destination the host has not contacted before, exfiltration evidence is raised at once. The fast path stays off until an upload limit has been calibrated.

Beaconing is checked by grouping flows by host, destination (the server name if there is one, otherwise the address) and JA3. Once a group has five connections, we compute the coefficient of variation (Equation 5.5) for the gaps between connections and for the bytes per connection. Evidence is raised only if both are at most 0.15 and the JA3 is rare, meaning no more than two hosts have used it. Flows without a JA3 are never flagged. Requiring all three conditions keeps regular but common traffic, such as browser update checks, from being reported.

7.1.7.3 Service Loop

The service reads flow-ready messages through its own consumer group. For each one it loads the flow record and passes it to the host-behaviour tracker and, if a model is loaded, to the anomaly detector. Each piece of evidence gets its weight from the settings (anomaly 2.0, exfiltration 2.0, beaconing 2.5 by default) and is published. Between messages the main loop retries a failed model load, checks whether recalibration is due, flushes quiet exfiltration buckets and publishes a heartbeat. Each job has its own error handler, so a failure in one cannot stop the heartbeat, which needs neither a model nor a calibration.

7.1.8 Self-Collected Dataset Tooling

7.1.8.1 Attack Scripts

Ten scripts generate the attacks for the self-collected dataset: port scan, SSH brute force, SYN flood, DDoS, ARP spoofing, DNS spoofing, C2 beaconing, C2 with a chosen JA3, data exfiltration and lateral movement. Seven wrap standard tools (nmap, hydra, hping3, arpspoof and dnsspoof) and three are small Python TLS clients. All of them run through one runner. It prints a reminder that attacks may only be run on an isolated lab network, offers a dry-run mode, and records the run even if it is interrupted. For every run it appends one JSON line with the attack, tool, attacker and victim addresses, UTC start and end time, status and parameters. These records let us label every flow in a capture automatically (Section 5.2.2).
Two scripts have limits worth stating. The DDoS script is a single-source flood, because we have only one attacker machine, and each record says so. DNS spoofing works only while ARP spoofing of the same victim is running, and the two are started separately.

7.1.8.2 Capture Sessions

A capture session is a directory named after its UTC start time and a label, and tcpdump writes the pcap file into it. A benign session captures for a set time and then writes a manifest of the Suricata alerts that fell inside that window, so traffic Suricata already finds suspicious can be left out before the capture is labelled benign (Section 5.2.2). An attack session starts tcpdump, runs one attack with the session directory as its output folder, waits three seconds for the last packets and stops the capture. None of this produces labelled flow

tables. That step runs the flow extractor over each pcap and matches the flows to the attack records, and it belongs to the offline training pipeline.

7.1.9 Components Under Development

The decision engine and the dashboard are partly built. The other components are designed but not written.

- The decision engine is partly built. It combines evidence into a decaying risk score for each host and decides whether to pass, alert or block (Algorithm 5.5). The rest of the logic and its link to the enforcer are still to do.
- The dashboard is partly built. It shows the data already written to SQLite, and the remaining screens of Section 4.8 are still to do.
- The Layer 3 classifier will use Random Forest or XGBoost on the same 78 features. Below a confidence threshold, a flow is labelled unknown.
- The enforcer will turn block decisions into ipset entries, with an allowlist, a rate limit and exponential back-off (Algorithm 5.6).
- The watchdog will read the heartbeats that every service already publishes and apply the configured failure mode.
- The offline training pipeline will regenerate flows from the public datasets with the flow extractor and train the models.

7.1.10 Setup and Deployment

Running the system takes several manual steps. Python 3.11 or later, Redis and Suricata have to be installed, and the Python dependencies go into a virtual environment from the requirements file. Nothing detects the capture interface, so its name is found by hand. A short Python call creates the database and seeds the default settings. Once the environment variables in Table 7.3 are exported, Redis and Suricata are started, followed by the four services in separate terminals. The flow extractor and the ARP monitor need root or CAP_NET_RAW. There is no combined launcher or systemd unit yet and the dashboard is incomplete, so the most reliable ways to observe the system are the database and the JSON logs.

Table 7.3: Environment variables read by the services.

Variable	**Used by**	**Default**
NIDPS_DB_PATH	All services	/var/lib/nidps/nidps.sqlite3
NIDPS_REDIS_URL	All services	redis://localhost:6379/0
NIDPS_CAPTURE_ INTERFACE	Flow extractor, ARP monitor	wlan0
NIDPS_GATEWAY_IP	ARP monitor	Required
NIDPS_AP_MAC	ARP monitor	Required
NIDPS_DHCP_LEASE_PATH	ARP monitor	/var/lib/misc/dnsmasq.leases
NIDPS_EVE_JSON_PATH	Suricata bridge	/var/log/suricata/eve.json

Chapter 9

Experimental Results and Discussion

This chapter covers the experimental work carried out during the development of the NIDPS.

9.1 Experimental Setup

The experiments described in this chapter were conducted on a small isolated test network set up for this project and kept separate from any production or household network. The NIDPS device itself is a single Linux host, running hostapd to act as the wireless access point (Section 5.1), with Suricata, the ARP monitor and the Layer 2 anomaly service all running on it. One laptop was used as the attacker machine and executed the scripts described in Section 7.1.8.1. A second laptop and a phone were connected through the access point as ordinary clients, generating background traffic and serving as victims. For each of the ten attack scripts listed in Table 9.1, the following steps were followed:

1. Suricata, the ARP monitor and the Layer 2 anomaly service were started and left to run for a short warm-up period so that the host-behaviour tracker of Section 7.1.7.2 had a baseline.
2. The attack script was run once from the attacker machine, through the attack runner described in Section 7.1.8.1, which records the attack name, its start and end time, and the attacker and victim addresses.
3. After the run, the Suricata `eve.json` log, the evidence table written by the services, and the anomaly service's own log were inspected by hand to see whether the attack produced any evidence, and from which detector.

Table 9.1: Attack scripts used in the experiments (Section 7.1.8.1).

Attack	**Tool / script**
Port scan	nmap
SSH brute force	hydra
SYN flood	hping3
DDoS (single source)	hping3, run from one attacker machine
ARP spoofing	arpspoof
DNS spoofing	dnsspoof (run together with ARP spoofing of the same victim)
C2 beaconing	custom Python TLS client
C2 beaconing with a chosen JA3	custom Python TLS client
Data exfiltration	custom Python TLS client
Lateral movement	custom Python TLS client

For the Isolation Forest used in Section 9.3, scikit-learn was used to fit a model on a benign capture from the test network, using the same 78-feature flow records the live extractor produces. Flows were then scored through the scoring and calibration code described in Section 7.1.7.1.

9.2 Signature-Based Detection: Suricata Against the Self-Collected Attacks

Only four custom Suricata rules are enabled: SSH brute force, port scan, SYN flood and DNS spoofing (Section 7.1.6.1). Running the ten attacks against the live system produced the pattern shown in Table 9.2.

Six of the ten attacks generated no Suricata alert at all, and a seventh, DDoS, was detected only some of the time. This result is consistent with the prediction made in Chapter 5: signature detection covers only attacks with a stable packet-level pattern, and the remaining attacks were selected precisely because they do not have one. ARP spoofing is the one exception to this pattern. Suricata itself missed it, but the ARP monitor caught it, which is the reason that detector exists as a separate component rather than as another Suricata rule.

Table 9.2: Suricata's response to each self-collected attack.

Attack	**Alert raised**	**Notes**
Port scan	Yes	Matched the twenty-ports-in-10 s rule every time it was run.
SSH brute force	Yes	Matched the five-SYNs-to-port-22 rule as soon as the fifth connection attempt landed inside the 60 s window.
SYN flood	Yes	Matched the 100-SYNs-in-10 s rule.
DNS spoofing	Yes	Matched because the forged reply came from an address other than the lab resolver.
DDoS (single source)	Partially	Crossed the SYN-flood threshold when the flood rate was high enough, but a slower run of the same script stayed under 100 SYNs in 10 s and produced no alert. A genuine distributed flood, coming from many sources at once, would likely never trip this single-source rule at all.
ARP spoofing	No	No Suricata rule inspects ARP traffic; this attack was instead picked up by the ARP monitor (Section 7.1.6.3), which raised evidence for the spoofed address as designed.
C2 beaconing	No	No packet-level signature exists for a client that just repeats regular TLS connections to one address; this is the behaviour Layer 2 is meant to catch.
C2 beaconing with a chosen JA3	No	The JA3-blacklist rule is written but left commented out (Section 7.1.6.1) because no bad JA3 hash had been recorded for the lab yet, so the attack produced no Suricata alert.
Data exfiltration	No	A single large upload to a new destination has no signature; this is exactly the behaviour the exfiltration check in Section 7.1.7.2 is designed to flag instead.
Lateral movement	No	Internal connections between lab hosts matched no active rule.

9.3 Anomaly Detection: Isolation Forest Against the Self-Collected Attacks

The four attacks that left no trace in the Suricata log, but that still leave a flow-level footprint, are C2 beaconing, C2 beaconing with a chosen JA3, data exfiltration and lateral movement. These were scored using the provisional Isolation Forest and the host-behaviour checks described in Section 9.1. The anomaly service logged the evidence it generated for each run, and Table 9.3 summarises the results.

Table 9.3: Isolation Forest and host-behaviour results on the four attacks Suricata missed.

Attack	**Observation**
Data exfiltration	The outbound-bytes burst exceeded the host's rolling baseline and tripped the exfiltration check of Section 7.1.7.2 on every run; this check does not depend on the Isolation Forest model itself.
C2 beaconing (fixed JA3)	Once five or more connections to the same destination and JA3 had been seen, the beaconing check flagged the host, because the gaps between connections and the bytes per connection stayed regular, exactly the pattern the coefficient-of-variation test is built to catch.
C2 beaconing with a chosen JA3	Behaved the same way as the plain beaconing run; changing the JA3 string did not by itself change the regularity of the timing or byte pattern, so it was still flagged.
Lateral movement	The flows involved did not fit the exfiltration or beaconing patterns. The Isolation Forest scored a few of these flows above the ad hoc threshold used for this test, but not consistently, and several lateral-movement flows scored close to the benign flows from the warm-up capture.

Benign flows from the warm-up capture scored consistently lower than the attack flows, though the sample on both sides was small. Precision, recall and false positive rate are not reported here, since a meaningful figure would require far more flows and the cross-dataset comparison described in Section 5.10.2, neither of which this round of testing attempted. Lateral movement produced the weakest result, and detecting it reliably will likely require the Layer 3 classifier, once it exists, rather than the Isolation Forest on its own.

9.4 Discussion

The following experiments were used to validate the division of labour between all three layers of Section 5.1. Suricata successfully blocked all the attacks that had a fixed packet-

level pattern and failed to block all attacks that were not fixed in their packet-level pattern (such as the JA3-blacklist case, where in principle there was a signature but it was not yet populated). That's why the one attack doesn't appear as a rule, but rather as a separate attack type detected by the ARP monitor. The two behaviours they were designed for, exfiltration and beaconing, were clearly picked up by the Isolation Forest and the host-behaviour checks, respectively. Lateral movement was weaker, and is probably more appropriate for the Layer 3 classifier, after it has been constructed.

9.5 Limitations of the Current Experimental Work

- The Layer 3 classifier, the enforcer and the watchdog have not yet been built (Section 7.1.2), so none of the system-performance measures described in Section 5.10.3, such as forwarding latency, throughput, time to block and recovery behaviour, could be measured.
- Each attack was run only a small number of times, which is enough to observe whether a detector fires but not enough to establish a stable false positive or detection rate.
- The DDoS script used here is a single-source flood rather than a true distributed attack, so its partial detection by the SYN-flood rule should not be taken as representative of how that rule would perform against a genuine distributed flood.

9.6 Conclusion

This chapter presented a small set of experiments against the parts of the NIDPS that have already been built. Suricata detected the attacks it has rules for and missed the rest, as expected, while the ARP monitor detected the one attack Suricata cannot structurally see. The Isolation Forest and the host-behaviour checks performed well on exfiltration and beaconing and performed poorly on lateral movement. Taken together, these results offer an early indication that the layered design works as intended.

Bibliography

[1] S. A. R. Shah and B. Issac, "Performance comparison of intrusion detection systems and application of machine learning to Snort system," *Future Generation Computer Systems*, vol. 80, pp. 157–170, 2018.

[2] G. Kim, S. Lee, and S. Kim, "A novel hybrid intrusion detection method integrating anomaly detection with misuse detection," *Expert Systems with Applications*, vol. 41, no. 4, pp. 1690–1700, 2014.

[3] Z. Chiba, N. Abghour, K. Moussaid, A. El Omri, and M. Rida, "Newest collaborative and hybrid network intrusion detection framework based on Suricata and Isolation Forest algorithm," in *Proc. 4th Int. Conf. Smart City Applications (SCA)*, pp. 1–11, ACM, 2019.

[4] S. Kushal, B. Shanmugam, J. Sundaram, and S. Thennadil, "Self-healing hybrid intrusion detection system: an ensemble machine learning approach," *Discover Artificial Intelligence*, vol. 4, 2024. Art. no. 28.

[5] S. C. Wen-Udeoji, M. K. Muyeba, and A. Mohammadi, "Comparative evaluation of machine learning and signature-based NIDS for multi-class and binary threat detection," in *Proc. Int. Conf. Data Science, AI and Applications*, Springer, 2025.

[6] Naik *et al.*, "Hybrid intrusion detection system for network security: real-time signature and machine learning-based anomaly detection," in *Proc. IEEE Int. Conf. CICT*, 2025.

[7] R. C. Wang and R. P. Avrianto, "Improving detection accuracy of network intrusions using a hybrid network intrusion detection system based on Isolation Forest and Random Forest algorithms," *Jurnal Teknik Informatika (JUTIF)*, vol. 6, no. 6, pp. 5371–5385, 2025.

[8] F. Moomtaheen, S. S. Bagui, S. C. Bagui, and D. Mink, "Extended Isolation Forest for intrusion detection in Zeek data," *Information*, vol. 15, no. 7, 2024. Art. no. 404.

[9] S. A. Elsaid and A. Binbusayyis, "An optimized isolation forest based intrusion detection system for heterogeneous and streaming data in the industrial Internet of Things (IIoT) networks," *Discover Applied Sciences*, vol. 6, 2024. Art. no. 483.

[10] Yu *et al.*, "Improving network intrusion detection methods in isolated forests based on split points," *Computer Engineering*, 2024.

[11] Y. Feng, W. Cai, H. Yue, J. Xu, Y. Lin, J. Chen, and Z. Hu, "An improved X-means and isolation forest based methodology for network traffic anomaly detection," *PLoS ONE*, vol. 17, no. 1, 2022. Art. no. e0263423.

[12] S. A. Ajagbe, J. B. Awotunde, and H. Florez, "Intrusion detection: a comparison study of machine learning models using unbalanced dataset," *SN Computer Science*, vol. 5, 2024. Art. no. 1028.

[13] S. More, M. Idrissi, H. Mahmoud, and A. T. Asyhari, "Enhanced intrusion detection systems performance with UNSW-NB15 data analysis," *Algorithms*, vol. 17, no. 2, 2024. Art. no. 64.

[14] S. A. Almahaqeri, M. H. Almourish, A. A. Nasser, A. S. A. Alghawli, A. A. K. Elsayed, and A. N. Alhejoj, "An optimized gradient boosting framework for IoT intrusion detection: a comprehensive evaluation on the CICIoT2023 dataset," *Scientific Reports*, vol. 16, 2026. Art. no. 16909.

[15] C. Xin and K. Xu, "Cross-dataset Transformer-IDS with calibration and AUC optimization (evaluated on NSL-KDD, UNSW-NB15, CIC-IDS2017)," *Journal of Cyber Security*, vol. 7, no. 1, pp. 483–503, 2025.

[16] I. Sharafaldin, A. Habibi Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Information Systems Security and Privacy (ICISSP)*, pp. 108–116, 2018.

[17] Canadian Institute for Cybersecurity, "CSE-CIC-IDS2018 dataset." University of New Brunswick. [Online]. Available: `https://www.unb.ca/cic/datasets/ids-2018.html`.

[18] Canadian Institute for Cybersecurity, "CIC IoT dataset 2023." University of New Brunswick. [Online]. Available: `https://www.unb.ca/cic/datasets/iotdataset-2023.html`.

[19] G. Engelen, V. Rimmer, and W. Joosen, "Troubleshooting an intrusion detection dataset: The CICIDS2017 case study," in *Proc. IEEE Security and Privacy Workshops (SPW)*, 2021.

[20] J. Althouse, J. Atkinson, and J. Atkins, "JA3 – a method for profiling SSL/TLS clients." Salesforce Engineering, 2017. [Online]. Available: `https://github.com/salesforce/ja3`.

[21] Open Information Security Foundation (OISF), "Suricata: Network IDS/IPS engine." [Online]. Available: `https://suricata.io`.

[22] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. 8th IEEE Int. Conf. Data Mining (ICDM)*, pp. 413–422, 2008.

[23] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[24] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, pp. 785–794, 2016.